

Requirement & Design Specification

SoundWave - Website Streaming Music (SW)

Version: 1.7

– Can Tho, September 2026 –

Record of Changes

Version	Date	A* M, D	In charge	Change Description
V1.0	15/9	A	DuyLH	Add (I) Overview - 2. Overall Functionalities (Screens Flow, Descriptions, Authorization, Non-UI)
V1.0	15/9	A	HaoTNT	Add (I) OverView - 1.2 Use Cases - a. Diagram(s) for Guest, Listener, Staff, and Admin
V1.0	15/9	A	TuanTA	Add (I) Overview - 1.1 Actors
V1.0	15/9	A	PhucDTT	Add (IV) Appendix - 3. Business Rules: 50 Business Rules (BR-01 to BR-50)
V1.0	15/9	A	TriNM	Add (I) Overview - 3. System High Level Design, 3.1 ERD, and 3.2 Database Design
V1.1	18/9	A	DuyLH	Add (II) Requirement Specifications: UC-01, UC-02, UC-03, UC-04, UC-05, UC-12.1, UC-12.2, UC-27.1, UC-27.2, UC-28, UC-29.1 to 29.4, UC-30
V1.1	18/9	A	HaoTNT	Add (II) Requirement Specifications: UC-09, UC-10, UC-11, UC-13, UC-14.1, UC-14.2, UC-14.3
V1.1	18/9	A	TuanTA	Add (II) Requirement Specifications: UC-06, UC-07, UC-08, UC-15.1, UC-15.2, UC-15.3, UC-15.4
V1.1	18/9	A	PhucDTT	Add (II) Requirement Specifications: UC-16, UC-17, UC-18, UC-19.1 to UC-19.5, UC-20, UC-21.1 to UC-21.4
V1.1	18/9	A	TriNM	Add (II) Requirement Specifications: UC-22.1 to 22.4, UC-23.1 to 24.2, UC-25, UC-26.1 to 26.4, UC-31.1 to 31.3
V1.2	22/9	M	PhucDTT	Modify (IV) Appendix - 3. Business Rules: Refine and reduce from 50 to 37 rules
V1.2	22/9	M	DuyLH	Modify (I) Overview - 2. Overall Functionalities (Add missing details and update screen flows)
V1.2	22/9	M	TuanTA	Modify (I) Overview - 1.2 Use Cases - a. Diagram(s)
V1.2	22/9	M	HaoTNT	Modify (I) Overview - 1.2 Use Cases - a. Diagram(s)
V1.2	22/9	A	TuanTA	Add (IV) Appendix - 1. Assumptions & Dependencies, 2. Limitations & Exclusions
V1.3	23/9	A	DuyLH	Add (III) Design Specifications: 1. User Authentication, 4.1.a Manage Personal Profile, 7. System Administration
V1.3	23/9	A	HaoTNT	Add (III) Design Specifications: 3. Music Playback, 4.1.b Listening History, 4.2 Favorite Management
V1.3	23/9	A	TuanTA	Add (III) Design Specifications: 2. Music Discovery, 4.3 Playlist Management
V1.3	23/9	A	PhucDTT	Add (III) Design Specifications: 4.4.b Custom Lyric Editor, 5.1 Track Management, 5.2 Album Management
V1.3	23/9	A	TriNM	Add (III) Design Specifications: 4.4.a Lyrics Viewer, 5.3 Content Report Management, 6. Staff Moderation, 8. Non-UI Functions
V1.4	26/9	M	DuyLH	Modify (II) Requirement Specifications: UC-01, UC-02, UC-03, UC-04, UC-05, UC-12.1, UC-12.2, UC-27.1 to UC-30

V1.4	26/9	M	HaoTNT	Modify (II) Requirement Specifications: UC-09, UC-10, UC-11, UC-13, UC-14.1, UC-14.2, UC-14.3
V1.4	26/9	M	TuanTA	Modify (II) Requirement Specifications: UC-06, UC-07, UC-08, UC-15.1, UC-15.2, UC-15.3, UC-15.4
V1.4	26/9	M	PhucDTT	Modify (II) Requirement Specifications: UC-16, UC-17, UC-18, UC-19.1 to UC-19.5, UC-20, UC-21.1 to UC-21.4
V1.4	26/9	M	TriNM	Modify (II) Requirement Specifications: UC-22.1 to 22.4, UC-23.1 to 24.2, UC-25, UC-26.1 to 26.4, UC-31.1 to 31.3
V1.5	27/9	A	TriNM	Add (I) Overview - 3.2 Code Packages
V1.6	2/10	D	TriNM	Delete (I) Overview - 1.2 Use Cases - a.Diagram(s) - UC26: Manage Lyric Delete (II) Requirement Specifications: UC-26.1, UC-26.2, UC-26.3, UC-26.4
V1.6	2/10	D	DuyLH	Delete (I) Overview - 2 Overall Functionalities - 1,2,3: UC-26
V1.7	8/10	D	HaoTNT	D (I) Overview - 1.2 Use Cases - a.Diagram(s) - UC-09: Listen to public Song, UC-27: Assign User Role D (II) Requirement Specifications: UC-09, UC-27
V1.7	8/10	A	HaoTNT	A (I) Overview - 1.2 Use Cases - a.Diagram(s) - UC-09: View Ranking Board, A (II) Requirement Specifications: UC-09
V1.7	8/10	M	HaoTNT	M (I) Overview - 1.2 Use Cases - a.Diagram(s) - UC-23: Submit System Review, UC24, UC25, UC26. UC27 M (II) Requirement Specifications: UC24, UC25, UC26. UC27
V1.7	9/10	D	DuyLH	Delete (I) Overview - 2 Overall Functionalities - 1,2,3: UC-9: Listen to public Song, UC-27: Assign User Role
V1.7	9/10	A	DuyLH	A (I) Overview - 2 Overall Functionalities - 1,2,3: UC-9: Listen to public Song
V1.7	9/10	M	DuyLH	A (I) Overview - 2 Overall Functionalities - 1,2,3: UC24, UC25, UC26. UC27

*A - Added M - Modified D - Deleted

Contents

Record of Changes	2
I. Overview	9
1. User Requirements	9
1.1 Actors	9
1.2 Use Cases	9
a. Diagram(s)	9
b. Descriptions	11
2. Overall Functionalities	15
2.1 Screens Flow	15
2.2 Screen Descriptions	16
2.3 Screen Authorization	18
2.4 Non-UI Functions	19
3. System High Level Design	21
3.1 ERD	21
3.2. Database Design	21
a. Database Schema	21
b. Table Descriptions	23
3.3 Code Packages	26
II. Requirement Specifications	28
1. User Authentication Feature	28
1.1 UC-1: Login	28
1.2 UC-2: Register Account	29
1.3 UC-3: Verify Email	30
1.4 UC-4: Forgot and Reset Password	32
1.5 UC-5: Logout	34
2. Music search & Browsing Feature	35
2.1 UC-6: View Recommended Song	35
2.2 UC-7: Search Public Catalog	36
2.3 UC-8: Filter Public Catalog	37
3. Audio Playback Feature	39
3.1 UC-9: Listen to Public Song	39
3.2 UC-10: Stream Music	40
3.3 UC-11: Control Playback	42
4. Profile Management	43
4.1 UC-12.1: Manage Personal Profile (Read)	43
4.2 UC-12.2: Manage Personal Profile (Update)	44
5. Music Interaction Feature	46
5.1 UC-13: View Listening History	46
5.2 UC-14.1: Manage my Favorite Tracks (Create)	47
5.3 UC-14.2: Manage my Favorite Tracks (Read)	48

5.4 UC-14.3: Manage my Favorite Tracks (Delete).....	50
6. Playlist Management.....	51
6.1 UC-15.1 Manage my Playlist (Create).....	51
6.2 UC-15.2 Manage my Playlist (Read).....	52
6.3 UC-15.3: Manage my Playlist (Update).....	53
6.4 UC-15.4 Manage my Playlist (Delete).....	54
7. Lyrics Interaction.....	56
7.1 UC-16 View Lyrics.....	56
7.2 UC-17 Switch Lyrics Language.....	57
7.3 UC-18 Create Custom Lyric.....	58
8. Track Management.....	59
8.1 UC-19.1 Manage my Tracks (Create).....	59
8.2 UC-19.2 Manage my Tracks (View).....	60
8.3 UC-19.3 Manage my Tracks (Update Status).....	61
8.4 UC-19.4 Manage my Tracks (Delete).....	62
8.5 UC-19.5 Submit Track for Moderation.....	63
8.6 UC-20 View Rejection Reason.....	64
9. Album Management.....	66
9.1 UC-21.1 Manage my Album (Create).....	66
9.2 UC-21.2 Manage my Album (Read).....	67
9.3 UC-21.3 Manage my Album (Update).....	67
9.4 UC-21.4 Manage my Album (Delete).....	69
10. Content Reporting Feature.....	70
10.1 UC-22.1 Manage Content Report (Create).....	70
10.2 UC-22.2 Manage Content Report (Read).....	71
10.3 UC-22.3 Manage Content Report (Update).....	72
10.4 UC-22.4 Manage Content Report (Delete).....	73
11. Content Moderation.....	75
11.1 UC-23.1: View Pending Tracks.....	75
11.2 UC-23.2 Update Track Moderation.....	76
11.3 UC-24.1: View Content Reports.....	78
11.4 UC-24.2 Update Content Report Resolution.....	80
11.5 UC-25: Take Down Published Track.....	82
12. System Administration.....	85
12.1 UC-26.1: Manage User Account (Ban).....	85
12.2 UC-26.2 Manage User Account (Unban).....	86
12.3 UC-27 Assign User Roles.....	88
12.5 UC-28.1 Manage Genre (Create).....	89
12.4 UC-22.2 Manage Genre (Read).....	91
12.6 UC-28.3 Manage Genre (Update).....	92
12.7 UC-28.4 Manage Genre (Delete).....	93

12.8 UC-29 View System Statistics.....	95
13. System Process.....	96
13.1 UC-30.1: Process Cloud Media (Create).....	96
13.2 UC-30.2: Process Cloud Media (Read).....	98
13.3 UC-30.3: Process Cloud Media (Delete).....	99
III. Design Specifications.....	100
1. User Authentication.....	100
1.1 System Access.....	100
a. User Login.....	100
b. Register Account.....	102
c. Verify Email.....	104
d. Forgot Password.....	105
e. Reset Password.....	107
f. Logout.....	108
2. Music Discovery.....	110
2.1 Public Catalog.....	110
a. Explore Music.....	110
b. Search Results.....	111
c. Filtered Public Catalog.....	112
d. Track Details.....	113
3. Music Playback.....	114
3.1 Audio Playback.....	114
a. Music Player.....	114
b. Playback Queue.....	116
4. Listener Profile and Library.....	117
4.1 Personal Profile.....	117
a. Manage Personal Profile.....	117
b. Listening History.....	118
4.2 Favorite Management.....	119
a. Favorite Tracks.....	119
4.3 Playlist Management.....	120
a. Playlist List.....	120
b. Playlist Details.....	121
c. Create or Edit Playlist.....	123
4.4 Lyrics.....	124
a. Lyrics Viewer.....	124
b. Custom Lyric Editor.....	124
5. User Content Management.....	126
5.1 Track Management.....	126
a. Track Management.....	126
b. Upload or Edit Track.....	127

c. Track Rejection Details.....	128
5.2 Album Management.....	129
a. Album Management.....	129
b. Create or Edit Album.....	131
5.3 Content Report Management.....	132
a. Content Report Management.....	132
b. Create Content Report.....	133
6. Staff Moderation.....	134
6.1 Staff Dashboard.....	134
a. Staff Dashboard.....	134
6.2 Track Moderation.....	135
a. Track Moderation.....	135
b. Track Review.....	136
6.3 Report Moderation.....	137
a. Report Moderation.....	137
b. Content Report Review.....	139
c. Track Takedown Confirmation.....	140
6.4 Official Lyrics.....	143
a. Official Lyrics Management.....	143
7. System Administration.....	144
7.1 Admin Dashboard.....	144
a. Admin Dashboard.....	144
7.2 User Management.....	145
a. User Management.....	145
b. User Details.....	146
c. Assign Role.....	147
7.3 Genre Management.....	148
a. Genre Management.....	148
b. Create or Edit Genre.....	149
7.4 Statistics.....	150
a. System Statistics.....	150
8. Non-UI Functions.....	151
a. Process Cloud Media.....	151
IV. Appendix.....	152
1. Assumptions & Dependencies.....	152
2. Limitations & Exclusions.....	152
3. Business Rules.....	152

I. Overview

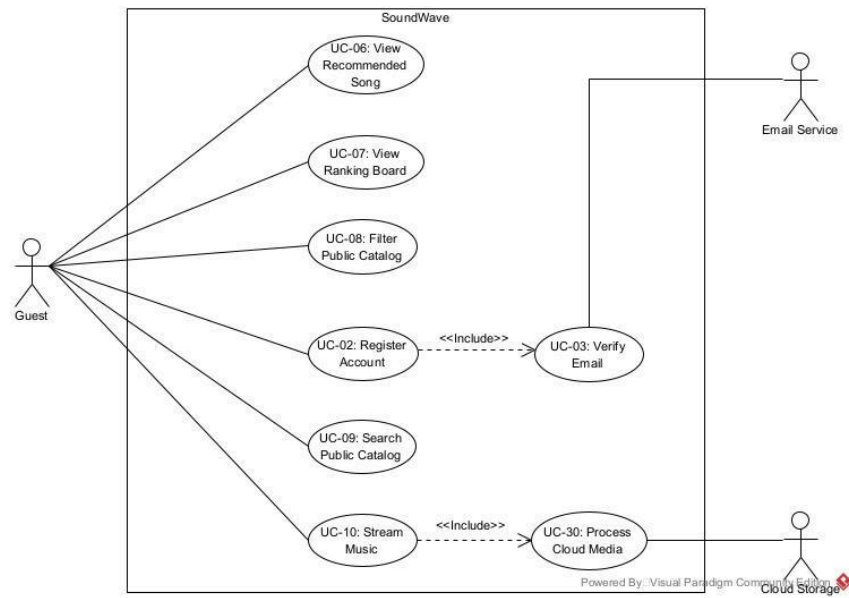
1. User Requirements

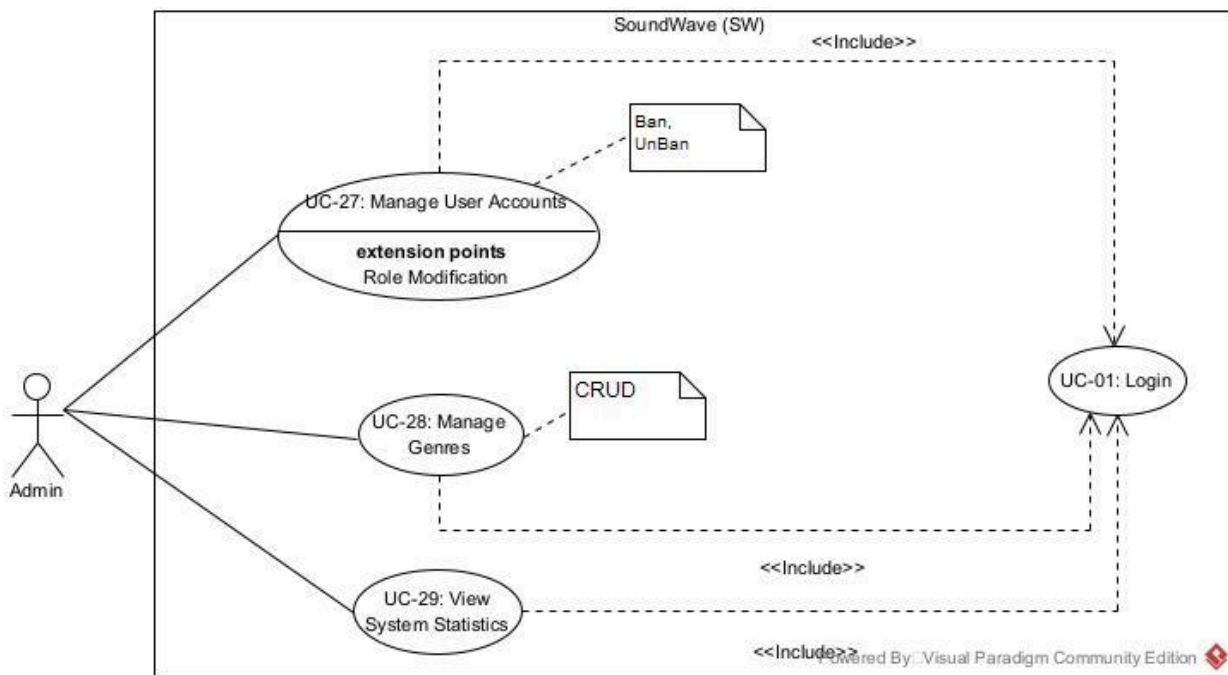
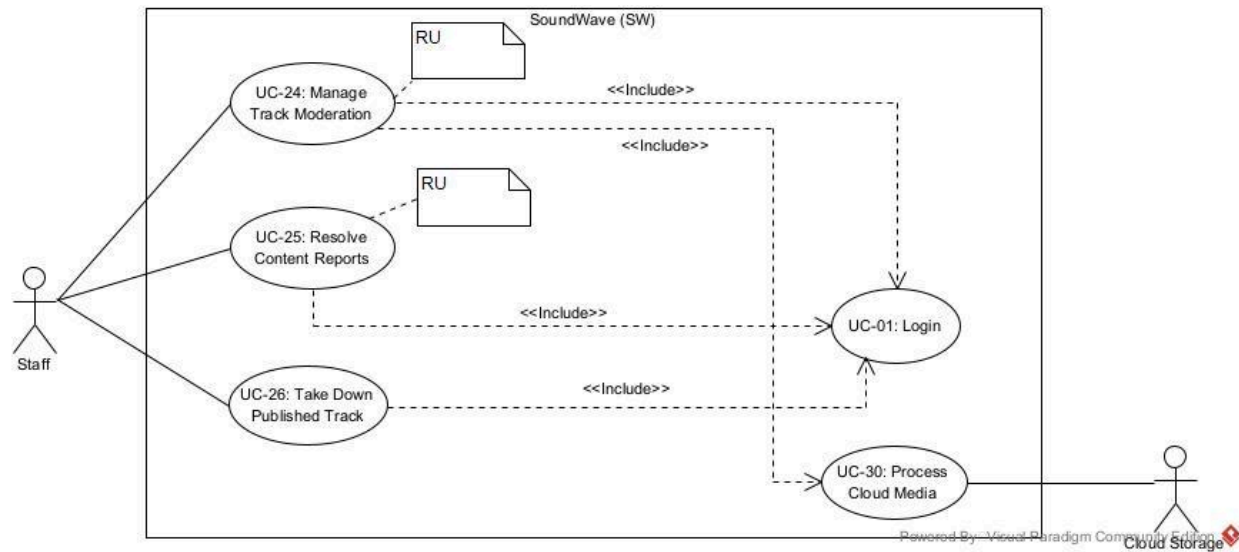
1.1 Actors

#	Actor	Description
1	Guest	An unauthenticated user who can explore, search, and filter the public music catalog, view public track details, and stream complete published tracks. After each track finishes, the system displays a login or registration prompt. A Guest cannot use authenticated features such as favorites, playlists, listening history, content upload, custom lyrics, or content reporting.
2	Listener	Authenticated general user. Has full access to advanced playback tools (queue, shuffle, repeat), manages playlists, writes personal (private) translated/transliterated lyrics, and uploads personal audio files to submit to the moderation queue.
3	Staff	Content management staff. Responsible for previewing and making approval/rejection decisions (with mandatory reasons) for songs uploaded by Listeners, as well as uploading official lyric files.
4	Admin	The highest-level system administrator. Has the authority to manage all accounts (role assignment, ban/unban), add/edit/delete core categories (such as Music Genres), and monitor system statistics.
5	Email Service	Automated email delivery system (Secondary Actor). Receives signals from the system to send welcome emails upon new account creation and distributes authentication codes/links during the password recovery flow.
6	Cloud Storage	Cloudinary cloud storage platform (Secondary Actor). Responsible for receiving, securely storing, and directly distributing multimedia files (MP3, cover art, avatars) to the user's browser, supporting byte-range data transmission techniques (HTTP Range Requests).

1.2 Use Cases

a. Diagram(s)





b. Descriptions

ID	Feature	Use Case	Use Case Description
UC-01	User Authentication	Login	Allows users to authenticate and log into the system.
UC-02	User Authentication	Register Account	Allows Guests to register a new account; includes a mandatory email verification step. (Includes: UC-03).

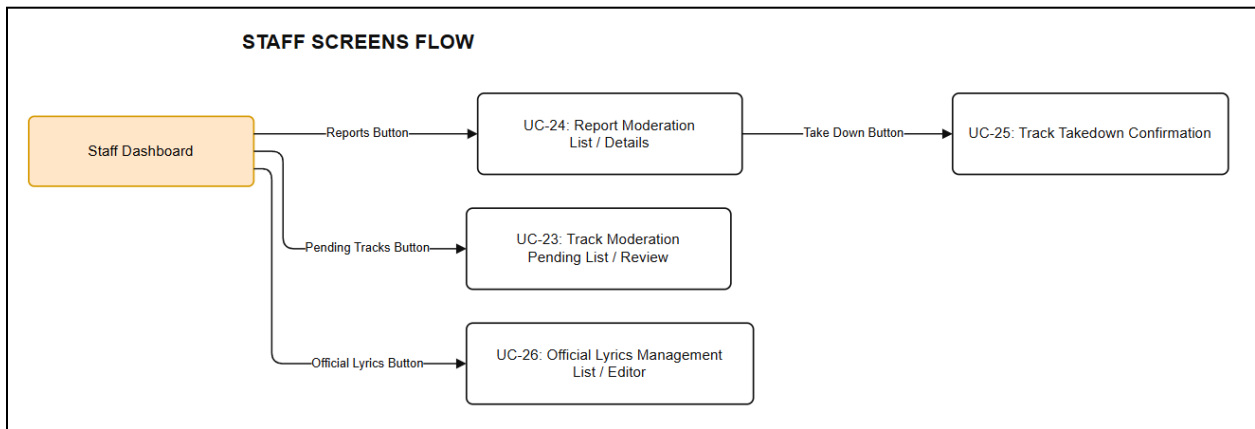
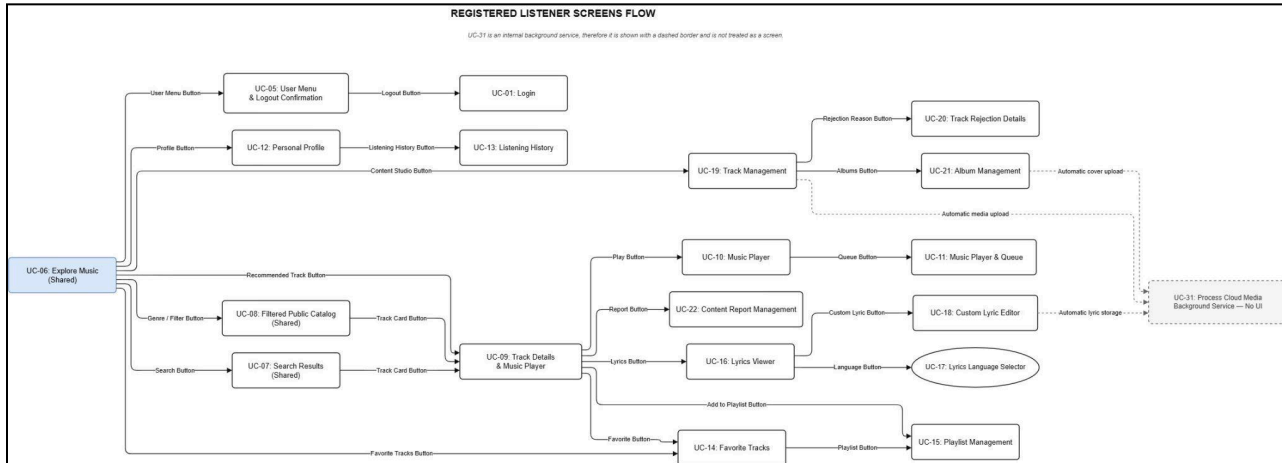
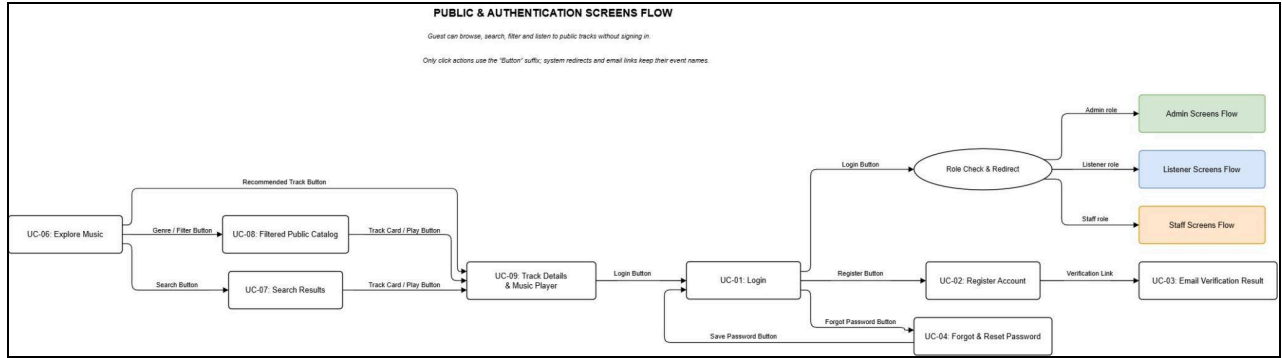
UC-03	User Authentication	Verify Email	Validates a secure, one-time token sent to the user's email address. This generalized use case handles two scenarios: (1) activating a newly registered account by verifying against email_verification_tokens, and (2) authorizing a password reset request by verifying against password_reset_tokens.
UC-04	User Authentication	Forgot and Reset Password	Allows an account owner to request a password-reset link, validate a single-use password-reset token, and securely create a new password.
UC-05	User Authentication	Logout	Allows authenticated users to end their current active session.
UC-06	Music Search & Browsing	View Recommended Song	Displays curated recommended, trending, and newly published public tracks on the Explore Music screen.
UC-07	Music Search & Browsing	View Ranking Board	Displays the ranked leaderboard of published tracks with the highest play counts on the platform.
UC-08	Music Search & Browsing	Filter Public Catalog	Filters published tracks by genre and supported sorting criteria.
UC-09	Music Search & Browsing	Search Public Catalog	Allows users to search published tracks, albums, and creator profiles by keyword.
UC-10	Audio Playback	Stream Music	Allows authenticated Listeners to stream full audio tracks continuously via HTTP Range Requests through Cloud Storage. <i>(Includes: UC-31).</i>
UC-11	Audio Playback	Control Playback	Allows Listeners to manage their active playback with queue, shuffle, and repeat functions.
UC-12	Profile Management	Manage Personal Profile	Allows Listeners to view and update their personal account details, biography, and profile avatar. <i>(Extension Point: Profile Dashboard Displayed).</i>
UC-13	Music Interaction	View Listening History	Allows Listeners to review their chronological history of streamed tracks. (Extends: UC-12).

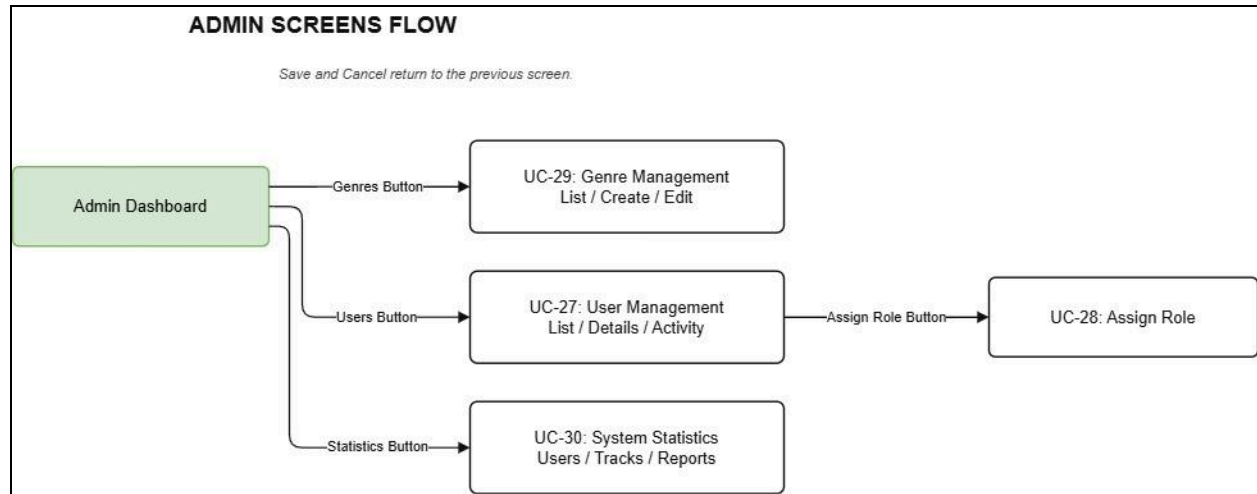
UC-14	Music Interaction	Manage My Favorite Tracks	Allows Listeners to perform CRD operations (add to favorites, view favorite list, remove from favorites) on tracks.
UC-15	Playlist Management	Manage My Playlist	Allows Listeners to perform full CRUD operations (create, view, edit title/description, delete) on custom playlists.
UC-16	Lyrics Interaction	View Lyric	Displays time-synchronized lyrics during track playback. (<i>Extension Point: Lyrics Displayed</i>).
UC-17	Lyrics Interaction	Switch Lyrics Language	Allows Listeners to toggle between multiple available language translations of the active synchronized lyrics. (<i>Extends: UC-16</i>).
UC-18	Lyrics Interaction	Create Custom Lyric	Allows Listeners to write, time-stamp, and save personal private lyric translations or transliterations for any track.
UC-19	Track Management	Manage My Tracks	Allows Listeners to perform CRUD operations (upload draft, update track metadata, delete unapproved tracks) on their own music tracks. (<i>Includes: UC-31</i>).
UC-20	Track Management	View Rejection Reason	Allows Listeners to view detailed feedback and official rejection reasons provided by moderation Staff for unapproved tracks.
UC-21	Album Management	Manage My Album	Allows Listeners to perform CRUD operations (create new EP/album, view catalog, update track order, delete album) to group their published songs.
UC-22	Content Reporting	Manage Content Report	Allows Listeners to submit, view processing status, update details, or withdraw copyright and policy violation reports against published content.
UC-23	System Interaction	Manage System Review	Allows authenticated Listeners to perform CR operations (submit a rating with feedback comments, and view personal review history).
UC-24	Content Moderation	Manage Track Moderation	Allows Staff to review pending submissions, stream audio previews, inspect history of moderated tracks (Read), and update status to Approved or Rejected

			with mandatory rejection feedback (Update). (Includes: UC-31)
UC-25	Content Moderation	Resolve Content Reports	Allows Staffs to review pending community reports, inspect submitted violation proof, view resolved log history (Read), and update reports by dismissing false alarms or confirming policy breaches (Update).
UC-26	Content Moderation	Take Down Published Track	Allows Staffs to immediately take down, suspend, or revoke public access to any infringing track on the live catalog.
UC-27	System Administration	Manage User Accounts	Allows Admins to inspect registered accounts, monitor platform access, and apply administrative restrictions by banning or unbanning users. (Extension Point: Role Modification).
UC-28	System Administration	Manage Genres	Allows Admins to perform CRUD operations (create, read, update, delete) on the platform's music genre hierarchy and classification tags.
UC-29	System Administration	View System Statistics	Allows Admins to view dashboard metrics, monitor aggregate listening activity, track moderation turnarounds, and inspect server traffic statistics.
UC-30	Cloud Media Feature	Process Cloud Media	Secondary system use case executed by Cloud Storage to ingest new audio/image files (Create), retrieve media chunks (Read), and permanently purge deleted assets (Delete).

2. Overall Functionalities

2.1 Screens Flow





2.2 Screen Descriptions

#	Feature	Screen	Description
1	UC-01: Login	Login	Authenticates a registered account and grants access according to the role returned by the backend.
2	UC-02: Register Account	Register Account	Creates a new Listener account and starts the mandatory email-verification process.
3	UC-03: Verify Email	Email Verification Result	Validates a single-use token for account activation or password-reset authorization.
4	UC-04: Forgot Password	Forgot and Reset Password	Requests a password-reset link, validates a single-use password-reset token, and securely creates a new password.
5	UC-05: Logout	User Menu and Logout Confirmation	Ends the current authenticated session and returns the user to public access .
6	UC-06: View Recommended Song	Explore Music	Displays recommended, trending, and newly published public tracks .
7	UC-07: Search Public Catalog	Search Results	Searches published tracks, albums, and creator profiles by keyword .
8	UC-08: Filter Public Catalog	Filtered Public Catalog	Filters eligible public tracks without exposing unpublished content .
9	UC-09: Listen to Public Song	Track Details and Music Player	Streams one complete published track for a Guest and shows a login or registration prompt when it ends .

10	UC-10: Stream Music	Music Player	Streams complete tracks continuously through HTTP Range Requests and records listening activity .
11	UC-11: Control Playback	Music Player and Queue	Controls play, pause, seek, previous, next, volume, queue, shuffle, and repeat behavior .
12	UC-12: Manage Personal Profile	Personal Profile	Allows Listeners to view and update their personal account details, biography, and profile avatar .
13	UC-13: View Listening History	Listening History	Displays recorded listening activity in reverse chronological order .
14	UC-14: Manage My Favorite Tracks	Favorite Tracks	Creates a favorite relationship for one eligible published track, views favorites, and removes favorites .
15	UC-15: Manage My Playlist	Playlist Management	Performs full CRUD operations (create, view, edit title/description, delete) on custom playlists .
16	UC-16: View Lyric	Lyrics Viewer	Displays an eligible synchronized official or private personal lyric version for the current track .
17	UC-17: Switch Lyrics Language	Lyrics Language Selector	Switches the displayed lyric to another approved/public or owned private language version without restarting playback .
18	UC-18: Create Custom Lyric	Custom Lyric Editor	Creates or updates a private personal lyric, translation, or transliteration for the selected track .
19	UC-19: Manage My Tracks	Track Management	Performs CRUD operations on owned track drafts, processes track media, and submits eligible tracks for moderation .
20	UC-20: View Rejection Reason	Track Rejection Details	Displays the latest Staff rejection feedback and available correction actions for a rejected track .
21	UC-21: Manage My Album	Album Management	Performs CRUD operations on owned albums and maintains eligible owned tracks in each album .
22	UC-22: Manage Content Report	Content Report Management	Submits a new policy or copyright report against a published track, views status, updates details, or withdraws reports .
23	UC-23: Manage Track Moderation	Track Moderation	Allows Staff to review pending submissions, stream audio previews, inspect moderation history (Read), and update status to Approved or Rejected with mandatory rejection feedback (Update) .
24	UC-24: Resolve Content Reports	Report Moderation	Allows Staff to review pending community reports, inspect submitted violation proof, view resolved log history (Read), and update reports by dismissing false alarms or confirming policy breaches (Update) .
25	UC-25: Take	Track Takedown	Removes public access to an infringing track and completes the

	Down Published Track	Confirmation	related report resolution atomically .
26	UC-27: Manage User Accounts	User Management	Allows Admins to inspect registered accounts, monitor platform access, and apply administrative restrictions by banning or unbanning users .
27	UC-28: Assign User Roles	Assign Role	Assigns, elevates, or revokes the account's single operational system role .
28	UC-29: Manage Genres	Genre Management	Performs CRUD operations (create, read, update, delete) on the platform's music genre hierarchy and classification tags .
29	UC-30: View System Statistics	System Statistics	Displays aggregate account, catalog, listening, moderation, report, and traffic metrics for Admin .
30	UC-31: Process Cloud Media	Background Service - No Standalone UI	Secondary system use case executed by Cloud Storage to ingest new audio/image files (Create), retrieve media chunks (Read), and permanently purge deleted assets (Delete) .

2.3 Screen Authorization

Screen / Function	Guest	User	Staff	Admin
UC-01 Login	x	-	-	-
UC-02 Register Account	x	-	-	-
UC-03 Email Verification Result	x	-	-	-
UC-04 Forgot and Reset Password	x	-	-	-
UC-05 Logout	-	x	x	x
UC-06 Explore Music	x	x	x	x
UC-07 Search Results	x	x	x	x
UC-08 Filtered Public Catalog	x	x	x	x
UC-09 Track Details and Public Playback	x	x	x	x
UC-10 Music Player	-	x	-	-
UC-11 Music Player and Queue	-	x	-	-
UC-12 Personal Profile	-	x	-	-
UC-13 Listening History	-	x	-	-
UC-14 Favorite Tracks	-	x	-	-
UC-15 Playlist Management	-	x	-	-
UC-16 Lyrics Viewer	-	x	-	-

UC-17 Lyrics Language Selector	-	x	-	-
UC-18 Custom Lyric Editor	-	x	-	-
UC-19 Track Management	-	x	-	-
UC-20 Track Rejection Details	-	x	-	-
UC-21 Album Management	-	x	-	-
UC-22 Content Report Management	-	x	-	-
Staff Dashboard	-	-	x	-
UC-23 Track Moderation	-	-	x	-
UC-24 Report Moderation	-	-	x	-
UC-25 Track Takedown Confirmation	-	-	x	-
UC-26 User Management	-	-	-	x
UC-27 Assign Role	-	-	-	x
UC-28 Genre Management	-	-	-	x
UC-29 System Statistics	-	-	-	x
UC-30 Process Cloud Media				

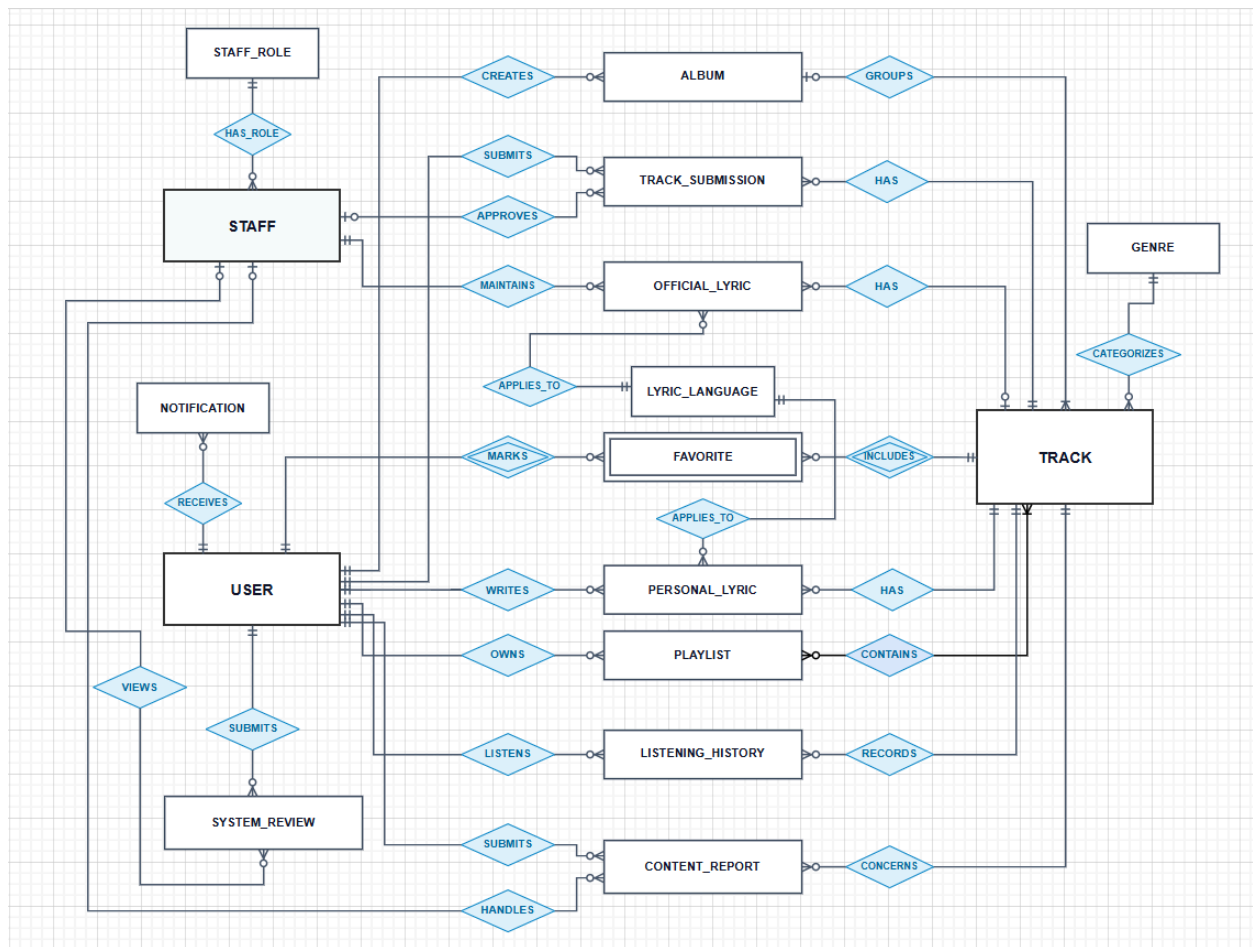
2.4 Non-UI Functions

#	Feature	System Function	Description
1	Email Verification	Send Verification Email	Generates a verification token and sends an account verification link to the registered email address.
2	Email Verification	Verify Email Token	Validates the verification token and activates the user account if the token is valid.
3	Password Recovery	Send Password Reset Email	Generates a password-reset token and sends a reset link to the user's email address.
4	Password Recovery	Validate Password Reset Token	Checks whether the password-reset token is valid, unused, and not expired.
5	Authentication	Generate Access Token	Generates a JWT access token after successful authentication.
6	Authentication	Refresh Access Token	Validates a refresh token and generates a new access token.
7	Authentication	Revoke Refresh Token	Revokes the current refresh token when the user logs out or when a session is invalidated.
8	Music Streaming	Stream Audio	Validates playback permissions and returns the direct Cloudinary secure audio URL supporting client-side HTTP Range Requests.
9	Listening Statistics	Record Play Count	Records a valid listening event and updates the track's play count.

10	Listening History	Record Listening History	Saves the authenticated listener's track-listening activity.
11	Cloud Media Processing	Upload Media	Uploads audio files, cover images, avatars, or lyric resources to cloud storage.
12	Cloud Media Processing	Delete Media	Deletes an unused or replaced media resource from cloud storage.
13	Cloud Media Processing	Generate Media URL	Returns the secure URL and public identifier of an uploaded media resource.
14	Track Moderation	Send Moderation Notification	Notifies the track owner when a submitted track is approved or rejected.
15	Content Report	Send Report Resolution Notification	Notifies the report submitter when a content report has been resolved.
16	Track Takedown	Unpublish Violated Track	Removes a violated track from the public catalog after the takedown decision is confirmed.
17	System Statistics	Aggregate System Statistics	Calculates administrative statistics for users, tracks, reports, and listening activity.
18	Data Maintenance	Delete Expired Tokens	Periodically removes expired or used email-verification and password-reset tokens.

3. System High Level Design

3.1 ERD



3.2. Database Design

a. Database Schema



b. Table Descriptions

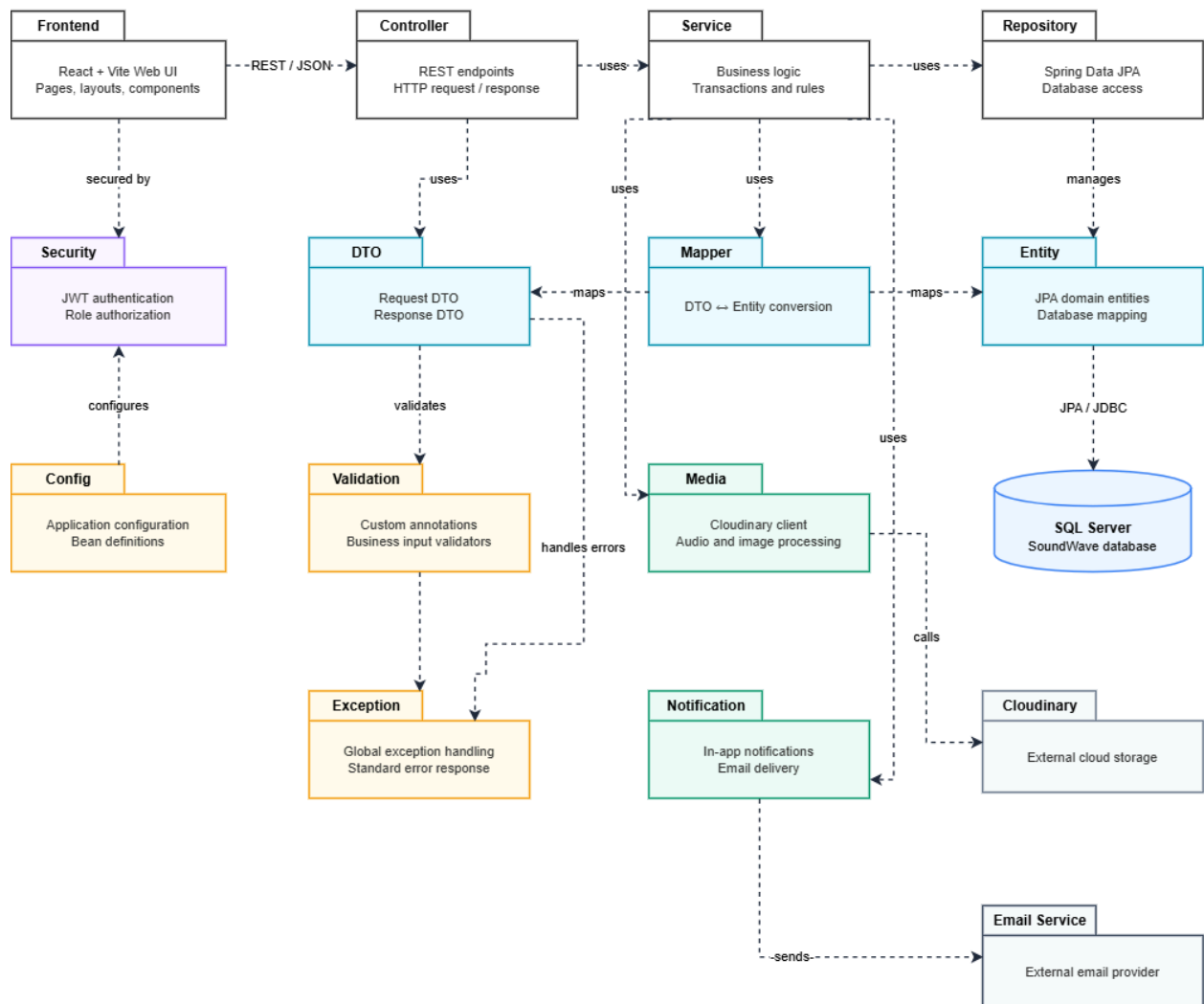
No.	Table	Description
1	roles	Stores the system roles used for authentication, authorization, and role-based navigation. The default roles are Listener, Staff, and Admin. Attributes: id, code, name, description, created_at. Primary key: id. Foreign keys: None.
2	app_users	Stores authentication credentials, account status, and role information for all registered accounts. Listener, Staff, and Admin accounts are stored in this table and distinguished by role_id. Attributes: id, role_id, email, password_hash, status, email_verified_at, last_login_at, deleted_at, created_at, updated_at. Primary key: id. Foreign keys: role_id references roles.id
3	user_profiles	Stores the public and personal profile information associated with an account, including username, display name, biography, avatar, date of birth, and country. Each account has one profile. Attributes: user_id, username, display_name, bio, avatar_public_id, avatar_url, date_of_birth, country_code, created_at, updated_at. Primary key: user_id. Foreign keys: user_id references app_users.id.
4	email_verification_tokens	Stores hashed one-time tokens used to verify an email address after account registration. A token has an expiration time and can only be used once. Attributes: id, user_id, token_hash, expires_at, used_at, created_at. Primary key: id. Foreign keys: user_id references app_users.id.
5	password_reset_tokens	Stores hashed one-time tokens used in the forgot-password and reset-password process. A valid token allows the account owner to create a new password before the token expires. Attributes: id, user_id, token_hash, expires_at, used_at, created_at. Primary key: id. Foreign keys: user_id references app_users.id.
6	refresh_tokens	Stores hashed refresh tokens used to maintain authenticated sessions and issue new access tokens. Tokens can be revoked during logout, account banning, password reset, or role modification. Attributes: id, user_id, token_hash, expires_at, revoked_at, created_at. Primary key: id. Foreign keys: user_id references app_users.id.
7	genres	Stores music genres used to categorize and filter tracks. An Admin can create, update, activate, or deactivate a genre without removing existing track relationships. Attributes: id, name, slug, description, is_active, created_by_user_id, created_at, updated_at. Primary key: id. Foreign keys: created_by_user_id references app_users.id.
8	albums	Stores albums, EPs, and singles created by registered listeners. It contains album metadata, publication status, cover references, release date, and publication time. Attributes: id, created_by_user_id, title, slug, description, status, cover_public_id, cover_url, release_date, published_at, created_at,

		updated_at. Primary key: id. Foreign keys: created_by_user_id references app_users.id.
9	tracks	Stores track metadata, publication status, playback information, and references to audio and cover files hosted on Cloudinary. Each track belongs to an uploader and a genre and may optionally belong to an album. Attributes: id, uploader_user_id, album_id, genre_id, title, slug, description, track_number, publication_status, approved_at, latest_rejection_reason, audio_public_id, audio_url, audio_format, duration_ms, cover_public_id, cover_url, play_count_cache, created_at, updated_at. Primary key: id. Foreign keys: uploader_user_id → app_users.id; album_id references albums.id (ON DELETE SET NULL, Nullable); genre_id references genres.id.
10	track_submissions	Stores each request to submit a track for Staff review. It records the submitter, reviewer, submission status, notes, rejection reason, submission time, and review time. Attributes: id, track_id, submitted_by_user_id, reviewer_user_id, status, submitter_note, reviewer_note, rejection_reason, submitted_at, reviewed_at. Primary key: id. Foreign keys: track_id references tracks.id; submitted_by_user_id references app_users.id; reviewer_user_id references app_users.id.
11	official_lyrics	Stores official time-synchronized lyrics for published tracks. Lyrics can be maintained in multiple languages, with one official version for each track and language combination. Attributes: id, track_id, language_code, lyric_content, status, created_by_user_id, created_at, updated_at. Primary key: id. Foreign keys: track_id references tracks.id; created_by_user_id references app_users.id.
12	personal_lyrics	Stores private lyrics, translations, or transliterations created by listeners for specific tracks. Personal lyrics are only accessible to the account that created them. Attributes: id, user_id, track_id, language_code, lyric_content, created_at, updated_at. Primary key: id. Foreign keys: user_id references app_users.id; track_id references tracks.id.
13	listening_history	Stores valid playback events for authenticated listeners. It records the selected track, playback time, listened duration, and whether the track was completed. Attributes: id, user_id, track_id, listened_duration_ms, completed, played_at. Primary key: id. Foreign keys: user_id references app_users.id; track_id references tracks.id.
14	favorites	Represents the many-to-many relationship between users and their favorite tracks. The composite primary key prevents the same user from favoriting the same track more than once. Attributes: user_id, track_id, created_at. Primary key: user_id, track_id. Foreign keys: user_id references app_users.id; track_id references tracks.id.
15	playlists	Stores playlists created by registered listeners. It contains the playlist name, description, visibility, cover references, owner, and creation and modification

		times. Attributes: id, owner_user_id, name, description, visibility, cover_public_id, cover_url, created_at, updated_at. Primary key: id. Foreign keys: owner_user_id references app_users.id.
16	playlist_tracks	Represents the many-to-many relationship between playlists and tracks. It also stores the playback position, the account that added the track, and the time the track was added. Attributes: playlist_id, track_id, added_by_user_id, position, added_at. Primary key: playlist_id, track_id. Foreign keys: playlist_id references playlists.id (ON DELETE CASCADE); track_id references tracks.id (ON DELETE CASCADE); added_by_user_id references app_users.id.
17	content_reports	Stores reports submitted by authenticated listeners about published tracks that may violate platform policies. It records the category, description, processing status, assigned Staff member, resolution note, and processing time. Attributes: id, track_id, submitted_by_user_id, handled_by_user_id, category, description, status, resolution_note, created_at, handled_at, updated_at. Primary key: id. Foreign keys: track_id references tracks.id; submitted_by_user_id references app_users.id; handled_by_user_id references app_users.id.
18	notifications	Stores in-application notifications sent to registered accounts. Notifications may contain track review results, report resolution results, account-status changes, role changes, or general system messages. Attributes: id, user_id, type, title, message, action_url, read_at, created_at. Primary key: id. Foreign keys: user_id references app_users.id.
19	lyric_languages	Stores the supported languages for track lyrics (e.g., Vietnamese, English, Japanese). It serves as a standardized reference catalog for both official and personal lyric translations. Attributes: id, code, name, created_at. Primary key: id. Foreign keys: None.
20	system_reviews	Stores application ratings (1 to 5 stars) and feedback comments submitted by authenticated listeners. It allows users to rate the platform and enables Staff to review user feedback. Attributes: id, user_id, rating, comment, status, created_at, updated_at. Primary key: id. Foreign keys: user_id references app_users.id.

3.3 Code Packages

SoundWave Code Package Diagram



Package descriptions

No	Package	Description
1	frontend	Contains the React and Vite user interface, including pages, layouts, components, routes, hooks, and API clients.
2	controller	Receives HTTP requests, validates request data, calls the appropriate service, and returns HTTP responses.
3	dto.request	Contains request objects used to receive and validate data sent from the frontend.
4	dto.response	Contains response objects used to return necessary data without exposing entities directly.

5	service	Contains business logic, business rules, authorization checks, and transaction processing.
6	repository	Provides database access through Spring Data JPA and performs entity persistence and query operations.
7	entity	Contains JPA entities representing the tables and relationships in the SQL Server database.
8	mapper	Converts Request DTOs and entities into the required entity or Response DTO format.
9	validation	Contains custom validation annotations and validators for business-specific input rules.
10	security	Handles JWT authentication, access authorization, password encoding, and role-based permissions.
11	config	Contains application configuration for security, CORS, database, Cloudinary, email, and shared beans.
12	exception	Contains custom exceptions and global exception handlers that produce standardized error responses.
13	media	Handles audio and image validation, upload, replacement, deletion, and processing through Cloudinary.
14	notification	Handles in-app notifications and email delivery for authentication, moderation, and account-management events.

II. Requirement Specifications

1. User Authentication Feature

1.1 UC-1: Login

UC ID and Name:	UC-1 Login		
Created By:	DuyLH	Date Created:	26 /09/2026
Primary Actor:	Guest	Secondary Actors:	None
Trigger:	A Guest submits an email address and password on the Login screen.		
Description:	Authenticates a registered account and grants access according to the role returned by the backend.		
Preconditions:	PRE-1: The account exists. PRE-2: The user is not currently authenticated.		
Postconditions:	POST-1: A valid session is created. POST-2: The user is redirected to the role-appropriate area.		
Normal Flow:	NF01: Successful login 1. The Guest opens the Login screen. 2. The Guest enters email and password. 3. The system validates the input and credentials. 4. The system verifies email and account status. 5. The system creates access and refresh tokens. 6. The backend identifies the account role and redirects the user.		
Alternative Flows:	AF01: Unverified account 1. At step 4 of NF01, the account email is unverified. 2. The system denies authenticated access and offers to resend verification. AF02: Role-specific redirection 1. At step 6 of NF01, Listener goes to Explore, Staff to Staff Dashboard, and Admin to Admin Dashboard.		
Exceptions:	EX01: Invalid credentials 1. At step 3 of NF01, the credentials do not match. 2. The system shows a generic inline error and keeps the user on Login. EX02: Authentication service unavailable 1. The system cannot create the session. 2. The system logs the failure and asks the user to retry.		
Priority:	Must Have		
Frequency of Use:	High		

Business Rules:	BR-01, BR-05, BR-06
Other Information:	The login form never asks the user to select a role.
Assumptions:	The authentication service and database are available.

1.2 UC-2: Register Account

UC ID and Name:	UC-2 Register Account		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Guest	Secondary Actors:	None
Trigger:	A Guest submits the Register Account form.		
Description:	Creates a new Listener account and starts the mandatory email-verification process.		
Preconditions:	PRE-1: The Guest is not authenticated. PRE-2: The email is not assigned to another account.		
Postconditions:	POST-1: A pending Listener account is created. POST-2: A verification token and email are generated.		
Normal Flow:	NF01: Register account 1. The Guest opens Register Account. 2. The Guest enters the required account information. 3. The system validates email uniqueness and password complexity. 4. The system hashes the password and creates an unverified Listener account. 5. The system invokes UC-03 Verify Email. 6. The system displays the verification instructions.		
Alternative Flows:	AF01: Cancel registration 1. Before step 4 of NF01, the Guest selects Login. 2. The system discards unsaved input and opens Login.		
Exceptions:	EX01: Duplicate email 1. At step 3 of NF01, the email already exists. 2. The system shows a custom inline message and creates no account.		

	EX02: Invalid registration data 1. Required data or password complexity is invalid. 2. The system marks the affected fields and requests correction. EX03: Verification email failure 1. The account is created but Email Service fails. 2. The system records the failure and offers resend.
Priority:	Must Have
Frequency of Use:	High
Business Rules:	BR-01, BR-02, BR-03, BR-34
Other Information:	Includes UC-03 Verify Email.
Assumptions:	The Guest can access the supplied email address.

1.3 UC-3: Verify Email

UC ID and Name:	UC-3 Verify Email		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Service
Trigger:	The user opens a one-time verification link sent by the system.		
Description:	Validates a single-use token for account activation or password-reset authorization.		
Preconditions:	PRE-1: A matching verification or password-reset token exists. PRE-2: The token has not been used.		

Postconditions:	POST-1: Registration verification activates the email, or password-reset verification authorizes UC-04. POST-2: The token is marked used.
Normal Flow:	NF01: Verify account email 1. The user opens the registration verification link. 2. The system hashes and locates the token in email_verification_tokens. 3. The system checks purpose, expiry, and used status. 4. The system marks the account email verified and consumes the token. 5. The system displays a success result and a Login action. NF02: Verify password-reset token 1. The user opens the reset link. 2. The system validates the token in password_reset_tokens. 3. The system consumes or reserves the token for one reset operation. 4. The system opens the Reset Password form.
Alternative Flows:	AF01: Request replacement link 1. The user selects Resend. 2. The system invalidates outstanding tokens of the same purpose. 3. The system creates and emails a replacement token.
Exceptions:	EX01: Invalid token 1. The token does not match a stored record. 2. The system displays an invalid-link result. EX02: Expired token 1. The token is expired. 2. The system displays an expired result and offers resend. EX03: Used token 1. The token was already consumed. 2. The system performs no state change and shows the appropriate result.
Priority:	Must Have
Frequency of Use:	Medium
Business Rules:	BR-01, BR-04, BR-34

Other Information:	Supports the two token stores defined by the approved use case.
Assumptions:	System clocks are synchronized and Email Service can deliver links.

1.4 UC-4: Forgot and Reset Password

UC ID and Name:	UC-4 Forgot and Reset Password		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Service
Trigger:	An account owner selects Forgot Password or opens a valid password-reset link.		
Description:	Requests a reset link, validates its single-use token, and creates a new password.		
Preconditions:	PRE-1: The user is not authenticated or cannot remember the password. PRE-2: For reset, a valid token has been supplied.		
Postconditions:	POST-1: The password is securely replaced. POST-2: The reset token and existing sessions are invalidated.		
Normal Flow:	NF01: Request password reset 1. The user opens Forgot Password and enters an email. 2. The system validates the input. 3. The system creates a time-limited password-reset token. 4. Email Service sends the reset link. 5. The system displays a generic confirmation. NF02: Reset password 1. The user opens the valid reset link. 2. UC-03 verifies the password-reset token. 3. The user enters and confirms a new password. 4. The system validates complexity and equality.		

	5. The system hashes and saves the password, consumes the token, and revokes existing sessions. 6. The system confirms success and opens Login.
Alternative Flows:	AF01: Email not registered 1. At step 2 of NF01, no account matches. 2. The system still displays the same generic confirmation and creates no token. AF02: Request another link 1. The user requests resend after expiry. 2. The system creates a replacement token and invalidates the prior token.
Exceptions:	EX01: Invalid or expired token 1. At step 2 of NF02, token verification fails. 2. The system denies reset and offers a new request. EX02: Non-compliant password 1. At step 4 of NF02, the password fails BR-03. 2. The system shows inline requirements without consuming the token. EX03: Email delivery failure 1. Email Service fails at step 4 of NF01. 2. The system logs the failure and schedules or offers retry.
Priority:	Must Have
Frequency of Use:	Medium
Business Rules:	BR-03, BR-04, BR-34
Other Information:	Uses password_reset_tokens; it does not activate an unverified account.
Assumptions:	The account owner can access the registered mailbox.

1.5 UC-5: Logout

UC ID and Name:	UC-5 Logout		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Listener, Staff, Admin	Secondary Actors:	None
Trigger:	An authenticated user selects Logout.		
Description:	Ends the current authenticated session and returns the user to public access.		
Preconditions:	PRE-1: An authenticated session exists.		
Postconditions:	POST-1: The current access and refresh tokens can no longer be used. POST-2: The user is returned to a public screen.		
Normal Flow:	NF01: Logout 1. The user selects Logout. 2. The system revokes the current refresh token. 3. The client clears authentication state. 4. The system displays confirmation and opens Explore as Guest.		
Alternative Flows:	AF01: Session already expired 1. The session expires before step 2 of NF01. 2. The client clears local state and opens Explore without calling protected functions.		
Exceptions:	EX01: Revocation service unavailable 1. Server-side revocation fails. 2. The client still clears local credentials; the server logs the failure for follow-up.		
Priority:	Must Have		
Frequency of Use:	Medium		
Business Rules:	None		

Other Information:	Logout is available from authenticated menus and dashboards.
Assumptions:	Refresh-token rotation or revocation is enabled.

2. Music search & Browsing Feature

2.1 UC-6: View Recommended Song

UC ID and Name:	UC-6 View Recommended Song		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Guest	Secondary Actors:	None
Trigger:	A user opens Explore Music.		
Description:	Displays recommended, trending, and newly published public tracks.		
Preconditions:	PRE-1: Published tracks exist.		
Postconditions:	POST-1: Eligible recommendation sections are displayed; no catalog data is modified.		
Normal Flow:	NF01: View recommendations 1. The user opens Explore Music. 2. The system queries eligible PUBLISHED tracks. 3. The system orders tracks for recommended, trending, and new-release sections. 4. The system displays track cards and public metadata. 5. The user may select a track and continue with UC-10.		
Alternative Flows:	AF01: Empty recommendation section 1. A section has no eligible track. 2. The system hides that section or shows an empty state while keeping other sections available.		
Exceptions:	EX01: Catalog retrieval failure		

	1. The query fails. 2. The system displays a retry state and logs the error.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-09
Other Information:	The ranking strategy may use release date and cached play counts.
Assumptions:	Recommendation data can be produced from existing catalog data.

2.2 UC-7: View Ranking Board

UC ID and Name:	UC-7: View Ranking Board		
Created By:	HaoNTT	Date Created:	08/10/2026
Primary Actor:	Guest, Listener	Secondary Actors:	None
Trigger:	A user clicks on the "Ranking Board" tab in the navigation bar or on the Explore Music screen.		
Description:	Displays the ranked list of published tracks with the highest play counts on the platform, sorted in descending order based on aggregate streaming statistics.		
Preconditions:	PRE-1: At least one track with PUBLISHED status exists in the catalog..		
Postconditions:	POST-1: The ranked leaderboard is displayed; no catalog data is modified.		
Normal Flow:	NF01: View ranking board 1. The user navigates to the Top Charts screen.		

	2. The system queries tracks with PUBLISHED status, ordered by play_count_cache descending (and by release date descending for ties). 3. The system renders the ranked list displaying rank positions (#1, #2, #3...), track titles, creator names, cover art, duration, and total play count. 4. The user may select any track to start streaming directly via UC-10.		
Alternative Flows:	AF01: Filter charts by genre: 1. At step 1 of NF01, the user selects a specific active genre. 2. The system filters the top charts to display only qualifying tracks within that genre.		
Exceptions:	EX01: No published tracks available: The catalog has no eligible published tracks; the system renders an empty state illustration. EX02: Query execution failure: The database query fails; the system displays retry controls and logs the error.		
Priority:	Should Have		
Frequency of Use:	High		
Business Rules:	BR-09		
Other Information:	The leaderboard ranking algorithm relies primarily on aggregated play counters (play_count_cache) and may apply secondary ordering based on publication date. Future enhancements may support daily, weekly, or monthly filtering based on specific ranges in listening_history		
Assumptions:	The platform maintains accurate streaming event logs, and the database cache for track play counts is kept up to date.		

2.3 UC-8: Filter Public Catalog

UC ID and Name:	UC-8 Filter Public Catalog		
Created By:	TuanTA	Date Created:	26/09/2026

Primary Actor:	Guest/Listener	Secondary Actors:	None
Trigger:	A user selects a genre or supported sorting criterion.		
Description:	Filters eligible public tracks without exposing unpublished content.		
Preconditions:	PRE-1: At least one active genre or sorting option exists.		
Postconditions:	POST-1: The filtered public catalog is displayed.		
Normal Flow:	NF01: Filter catalog 1. The user opens the filter controls. 2. The user selects genre and optional sort criteria. 3. The system validates the selected criteria. 4. The system queries PUBLISHED tracks matching the criteria. 5. The system displays the filtered results and updates the URL state.		
Alternative Flows:	AF01: Clear filters 1. The user selects Clear. 2. The system removes active criteria and restores the unfiltered public catalog. AF02: No matching track 1. The query returns no eligible track. 2. The system displays an empty state and Clear Filters action.		
Exceptions:	EX01: Invalid or inactive genre 1. A stale URL references an unavailable genre. 2. The system ignores the invalid value and informs the user. EX02: Catalog retrieval failure 1. The filtered query fails. 2. The system displays retry controls.		
Priority:	Should Have		
Frequency of Use:	High		

Business Rules:	BR-09
Other Information:	Filtering may be combined with a public search keyword.
Assumptions:	Genre activation state is current.

2.4 UC-9: Search Public Catalog

UC ID and Name:	UC-9 Search Public Catalog		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Guest	Secondary Actors:	None
Trigger:	A user submits a keyword from the search bar.		
Description:	Searches published tracks, albums, and creator profiles by keyword.		
Preconditions:	PRE-1: The public catalog is available.		
Postconditions:	POST-1: Matching eligible public results are displayed; no data is changed.		
Normal Flow:	NF01: Search catalog 1. The user enters a keyword. 2. The system normalizes and validates the keyword. 3. The system searches eligible tracks, albums, and creators. 4. The system applies publication and visibility rules. 5. The system displays grouped results with pagination.		
Alternative Flows:	AF01: No result 1. Step 3 of NF01 returns no match. 2. The system displays a no-results message and search suggestions. AF02: Lyric search 1. The user selects lyric search.		

	2. The system searches only tracks with approved official lyrics.
Exceptions:	EX01: Empty or invalid keyword 1. The keyword is blank or below the configured minimum. 2. The system shows inline guidance and does not execute the search. EX02: Search service failure 1. The search query fails. 2. The system displays a retry state.
Priority:	Must Have
Frequency of Use:	High
Business Rules:	BR-09, BR-10
Other Information:	Search is case-insensitive and may support partial matching.
Assumptions:	Search indexes are synchronized with catalog data.

UC ID and Name:	UC-9: View Ranking Board		
Created By:	HaoNTT	Date Created:	08/10/2026
Primary Actor:	Guest, Listener	Secondary Actors:	None
Trigger:	A user clicks on the "Ranking Board" tab in the navigation bar or on the Explore Music screen.		
Description:	Displays the ranked list of published tracks with the highest play counts on the platform, sorted in descending order based on aggregate streaming statistics.		
Preconditions:	PRE-1: At least one track with PUBLISHED status exists in the catalog..		

Postconditions:	POST-1: The ranked leaderboard is displayed; no catalog data is modified.
Normal Flow:	NF01: View ranking board 1. The user navigates to the Top Charts screen. 2. The system queries tracks with PUBLISHED status, ordered by play_count_cache descending (and by release date descending for ties). 3. The system renders the ranked list displaying rank positions (#1, #2, #3...), track titles, creator names, cover art, duration, and total play count. 4. The user may select any track to start streaming directly via UC-10.
Alternative Flows:	AF01: Filter charts by genre: 1. At step 1 of NF01, the user selects a specific active genre. 2. The system filters the top charts to display only qualifying tracks within that genre.
Exceptions:	EX01: No published tracks available: The catalog has no eligible published tracks; the system renders an empty state illustration. EX02: Query execution failure: The database query fails; the system displays retry controls and logs the error.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-09
Other Information:	The leaderboard ranking algorithm relies primarily on aggregated play counters (play_count_cache) and may apply secondary ordering based on publication date. Future enhancements may support daily, weekly, or monthly filtering based on specific ranges in listening_history
Assumptions:	The platform maintains accurate streaming event logs, and the database cache for track play counts is kept up to date.

3. Audio Playback Feature

3.1 UC-10: Stream Music

UC ID and Name:	UC-10 Stream Music		
Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	An authenticated Listener selects Play on an eligible track.		
Description:	Streams complete tracks continuously through HTTP Range Requests and records listening activity.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The track is eligible and has media.		
Postconditions:	POST-1: Requested audio ranges are delivered. POST-2: Qualified playback is recorded in listening history.		
Normal Flow:	NF01: Stream track 1. The Listener selects Play on an eligible track. 2. The system validates access rights and verifies that the track status is PUBLISHED. 3. The backend returns the track's secure audio CDN URL and playback metadata. 4. The client's HTML5 audio player requests and buffers audio byte ranges (Range: bytes=0-) directly from Cloudinary CDN. 5. The client player monitors continuous playback duration. 6. When the listening threshold is reached, the client emits a play-count log event to the backend.		
Alternative Flows:	AF01: Resume or seek 1. The Listener resumes or seeks to another position. 2. The player requests the corresponding byte range and continues playback. AF02: No next track 1. The track ends and the queue is empty. 2. The player stops at the completed track.		
Exceptions:	EX01: Unauthorized or expired session		

	1. Access validation fails. 2. The system pauses playback and requests login. EX02: Range request failure 1. Cloud Storage does not return the requested range. 2. The player retries within policy, then displays a playback error.
Priority:	Must Have
Frequency of Use:	Very High
Business Rules:	BR-09, BR-12, BR-13
Other Information:	Includes UC-30 Process Cloud Media.
Assumptions:	The browser supports HTML5 audio and Range Requests.

3.2 UC-11: Control Playback

UC ID and Name:	UC-11 Control Playback		
Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener uses a player or queue control during an active session.		
Description:	Controls play, pause, seek, previous, next, volume, queue, shuffle, and repeat behavior.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The player has an active or queued track.		
Postconditions:	POST-1: Player and queue state reflect the selected control.		

Normal Flow:	NF01: Basic playback control 1. The Listener selects play, pause, seek, volume, previous, or next. 2. The player validates the requested action. 3. The player updates audio state and visible controls. NF02: Manage queue 1. The Listener opens the queue. 2. The Listener adds, removes, or reorders eligible tracks. 3. The system updates the ordered queue. NF03: Shuffle and repeat 1. The Listener toggles shuffle or repeat. 2. The player recalculates the next-track rule and displays the active mode.
Alternative Flows:	AF01: Seek outside duration 1. The requested time is outside the track duration. 2. The player clamps the position to the nearest valid boundary. AF02: Previous at queue start 1. No previous item exists. 2. The player restarts the current track or performs no navigation according to player policy.
Exceptions:	EX01: Active track becomes unavailable 1. The current track is removed or inaccessible. 2. The player stops it and advances to the next eligible item. EX02: Player state error 1. The requested control cannot be applied. 2. The player restores the last stable state and displays a message.
Priority:	Must Have
Frequency of Use:	Very High
Business Rules:	BR-12
Other Information:	Queue state is session-scoped unless persistence is implemented.

Assumptions:	The client can maintain player state while navigating music screens.
---------------------	--

4. Profile Management

4.1 UC-12.1: Manage Personal Profile (Read)

UC ID and Name:	UC-12.1 View Personal Profile		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener opens Read Personal Profile. .		
Description:	Retrieves and displays the Listener's own profile information.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: A profile exists for the account.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow:	NF01: Read profile 1. The Listener opens Personal Profile. 2. The system validates ownership. 3. The system retrieves and displays the current profile.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		

Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-07, BR-08
Other Information:	Sub-function of UC-12 Manage Personal Profile. The approved parent Use Case remains unchanged.
Assumptions:	The account has one associated user profile.

4.2 UC-12.2: Manage Personal Profile (Update)

UC ID and Name:	UC-12.2 Update Personal Profile		
Created By:	DuyLH	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Update Personal Profile on an eligible record.		
Description:	Updates permitted fields and the avatar of the Listener's own profile.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: A profile exists for the account.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow:	NF01: Update profile 1. The Listener selects Edit Profile. 2. The Listener changes supported fields and optionally selects an avatar. 3. The system validates the data and avatar constraints. 4. When an avatar changed, the system invokes UC-30. 5. The system updates the owned profile and displays confirmation.		
Alternative Flows:	AF01: Cancel update		

	1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-07, BR-08
Other Information:	Sub-function of UC-12 Manage Personal Profile. The approved parent Use Case remains unchanged.
Assumptions:	The account has one associated user profile.

5. Music Interaction Feature

5.1 UC-13: View Listening History

UC ID and Name:	UC-13 View Listening History
------------------------	------------------------------

Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener selects Listening History from the profile or library.		
Description:	Displays recorded listening activity in reverse chronological order.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: The history is displayed without changing catalog data.		
Normal Flow:	NF01: View history 1. The Listener opens Listening History. 2. The system retrieves the Listener's history records. 3. The system excludes unavailable content where required. 4. The system displays the results in reverse chronological order with pagination. 5. The Listener may select an eligible track for UC-10.		
Alternative Flows:	AF01: Empty history 1. No history record exists. 2. The system displays an empty state and a link to Explore Music.		
Exceptions:	EX01: History retrieval failure 1. The database query fails. 2. The system displays retry controls and logs the error.		
Priority:	Should Have		
Frequency of Use:	Medium		
Business Rules:	BR-13		
Other Information:	Extends UC-12 at Profile Dashboard Displayed.		

Assumptions:	Playback-history creation uses a configured listening threshold.
---------------------	--

5.2 UC-14.1: Manage my Favorite Tracks (Create)

UC ID and Name:	UC-14.1 Add Track to Favorites		
Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Add Track to Favorites (Create).		
Description:	Creates a favorite relationship for one eligible published track.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: A target track is PUBLISHED for Create.		
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.		
Normal Flow:	NF01: Create favorite 1. The Listener selects the empty Favorite control on a published track. 2. The system confirms that no relation exists. 3. The system creates the favorite relation and updates the UI.		
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.		
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure		

	1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-09, BR-16, BR-17, BR-36
Other Information:	Sub-function of UC-14 Manage My Favorite Tracks. The approved parent Use Case remains unchanged.
Assumptions:	The database enforces uniqueness by user and track.

5.3 UC-14.2: Manage my Favorite Tracks (Read)

UC ID and Name:	UC-14.2 View Favorite Tracks		
Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener opens Read Favorite Tracks.		
Description:	Retrieves and displays the Listener's favorite tracks.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow:	NF01: Read favorite tracks 1. The Listener opens Favorite Tracks. 2. The system retrieves the Listener's favorite relations and eligible track data. 3. The system displays the collection.		
Alternative Flows:	AF01: No matching records		

	1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-09, BR-16, BR-17, BR-36
Other Information:	Sub-function of UC-14 Manage My Favorite Tracks. The approved parent Use Case remains unchanged.
Assumptions:	The database enforces uniqueness by user and track.

5.4 UC-14.3: Manage my Favorite Tracks (Delete)

UC ID and Name:	UC-14.3 Remove Song from Favorites		
Created By:	HaoNTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Remove Track from Favorites (Delete) and confirms the action.		
Description:	Deletes one owned favorite-track relationship.		
Preconditions:	PRE-1: UC-01 has completed.		

Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.
Normal Flow:	NF01: Delete favorite 1. The Listener selects the active Favorite control. 2. The system removes the owned favorite relation. 3. The system updates the collection and UI.
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged.
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-09, BR-16, BR-17, BR-36
Other Information:	Sub-function of UC-14 Manage My Favorite Tracks. The approved parent Use Case remains unchanged.
Assumptions:	The database enforces uniqueness by user and track.

6. Playlist Management

6.1 UC-15.1 Manage my Playlist (Create)

UC ID and Name:	UC-15.1 Create Playlist		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Create Playlist.		
Description:	Creates a new playlist owned by the Listener.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.		
Normal Flow	NF01: Create playlist 1. The Listener selects Create Playlist. 2. The Listener enters name, description, visibility, and optional cover. 3. The system validates the data and processes the cover through UC-30 when supplied. 4. The system creates the owned playlist.		
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.		
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.		
Priority:	Must Have		
Frequency of Use:	High		
Business Rules:	BR-14, BR-15, BR-35, BR-36		

Other Information:	Sub-function of UC-15 Manage My Playlist. The approved parent Use Case remains unchanged.
Assumptions:	Only the playlist owner can maintain a private playlist.

6.2 UC-15.2 Manage my Playlist (Read)

UC ID and Name:	UC-15.2 View My Playlists		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener opens Read Playlist.		
Description:	Retrieves owned playlists and playlist details.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow	NF01: Read playlists 1. The Listener opens My Playlists or an owned playlist. 2. The system retrieves permitted playlist metadata and ordered tracks. 3. The system displays the list or details.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		
Priority:	Must Have		
Frequency of Use:	High		
Business Rules:	BR-14, BR-15, BR-35, BR-36		

Other Information:	Sub-function of UC-15 Manage My Playlist. The approved parent Use Case remains unchanged.
Assumptions:	Only the playlist owner can maintain a private playlist.

6.3 UC-15.3: Manage my Playlist (Update)

UC ID and Name:	UC-15.3 Update Playlist		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Update Playlist on an eligible record.		
Description:	Updates owned playlist metadata, visibility, cover, tracks, or order.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow	NF01: Update playlist 1. The Listener selects Edit on an owned playlist. 2. The Listener changes metadata, visibility, cover, or track order. 3. The system validates ownership and input. 4. The system saves the changes and refreshes the playlist.		
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.		
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update		

	1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Must Have
Frequency of Use:	High
Business Rules:	BR-14, BR-15, BR-35, BR-36
Other Information:	Sub-function of UC-15 Manage My Playlist. The approved parent Use Case remains unchanged.
Assumptions:	Only the playlist owner can maintain a private playlist.

6.4 UC-15.4 Manage my Playlist (Delete)

UC ID and Name:	UC-15.4 Delete Playlist		
Created By:	TuanTA	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Delete Playlist and confirms the action.		
Description:	Deletes an owned playlist and its playlist-track relationships.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.		
Normal Flow	NF01: Delete playlist 1. The Listener selects Delete and confirms. 2. The system validates ownership. 3. The system deletes playlist-track relations and the playlist. 4. The system removes obsolete cover media when applicable.		
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged.		

Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.
Priority:	Must Have
Frequency of Use:	High
Business Rules:	BR-14, BR-15, BR-35, BR-36
Other Information:	Sub-function of UC-15 Manage My Playlist. The approved parent Use Case remains unchanged.
Assumptions:	Only the playlist owner can maintain a private playlist.

7. Lyrics Interaction

7.1 UC-16 View Lyrics

UC ID and Name:	UC-16 View Lyrics		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener opens Lyrics while an eligible track is active.		
Description:	Displays an eligible synchronized official or private personal lyric version for the current track.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: A lyric version is available for the active track.		
Postconditions:	POST-1: Lyrics are displayed and synchronized; no content is changed.		
Normal Flow	NF01: View synchronized lyric 1. The Listener opens Lyrics. 2. The system retrieves eligible official and owned personal versions. 3. The system selects the default permitted version. 4. The player displays timestamped lines and highlights the active line.		

	5. Lyrics continue to synchronize with playback position.
Alternative Flows:	AF01: No lyric available 1. Step 2 of NF01 returns no version. 2. The system displays a no-lyrics state and the Custom Lyric action. AF02: Unsynchronized text 1. A permitted lyric has no timestamps. 2. The system displays it as scrollable static text.
Exceptions:	EX01: Invalid lyric data 1. The selected lyric cannot be parsed. 2. The system hides the invalid version and shows another permitted version or an error. EX02: Lyric retrieval failure 1. Retrieval fails. 2. The system keeps playback active and offers retry.
Priority:	Should Have
Frequency of Use:	High
Business Rules:	BR-27, BR-28, BR-29
Other Information:	Extension point: Lyrics Displayed; UC-17 may extend this use case.
Assumptions:	Playback time is available to the lyric renderer.

7.2 UC-17 Switch Lyrics Language

UC ID and Name:	UC-17 Switch Lyrics Language		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener selects another language in the Lyrics Viewer.		
Description:	Switches the displayed lyric to another approved/public or owned private language version without restarting playback.		
Preconditions:	PRE-1: UC-16 is active. PRE-2: At least two permitted language versions exist.		
Postconditions:	POST-1: The selected language is displayed at the current playback position.		

Normal Flow	NF01: Switch language 1. The Listener opens the language selector. 2. The system lists permitted lyric versions. 3. The Listener selects a language. 4. The system loads the selected version. 5. The viewer aligns it to the current playback time without restarting audio.
Alternative Flows:	AF01: Return to original 1. The Listener selects Original. 2. The system restores the approved original version at the current playback time.
Exceptions:	EX01: Selected version unavailable 1. The version is removed or deactivated before step 4 of NF01. 2. The system refreshes the list and keeps the current version. EX02: Version load failure 1. The selected lyric cannot be loaded. 2. The system keeps the prior language and offers retry.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-27, BR-28
Other Information:	Extends UC-16 at Lyrics Displayed.
Assumptions:	Language codes are stored consistently.

7.3 UC-18 Create Custom Lyric

UC ID and Name:	UC-18 Create Custom Lyric		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener selects Create Custom Lyric for a track.		
Description:	Creates or updates a private personal lyric, translation, or transliteration for the selected track.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The target track exists.		
Postconditions:	POST-1: The owned private lyric is stored and available only to its creator.		

Normal Flow	NF01: Create custom lyric 1. The Listener selects Create Custom Lyric. 2. The Listener chooses a language and enters lyric content with optional timestamps. 3. The system validates the language and content. 4. The system creates the personal_lyrics record for the Listener and track. 5. The system confirms success and opens the private lyric. NF02: Update custom lyric 1. The Listener opens an owned custom lyric. 2. The Listener edits content or timestamps. 3. The system validates ownership and data. 4. The system updates the record and refreshes Lyrics Viewer.
Alternative Flows:	AF01: Save draft content 1. The Listener saves incomplete timestamps. 2. The system stores permitted static lyric text for later editing. AF02: Cancel editing 1. The Listener cancels before saving. 2. The system discards unsaved changes.
Exceptions:	EX01: Duplicate personal lyric language 1. A personal version already exists for the Listener, track, and language. 2. The system offers to open the existing version instead of creating a duplicate. EX02: Unauthorized update 1. The Listener attempts to edit another user's lyric. 2. The system denies the operation. EX03: Invalid content 1. The content exceeds limits or cannot be parsed. 2. The system displays field-level errors.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-29, BR-36
Other Information:	Personal lyrics are never published or shared.
Assumptions:	The selected track remains available to the Listener.

8. Track Management

8.1 UC-19.1 Manage my Tracks (Create)

UC ID and Name:	UC-19.1 Create Track Draft		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Create Track Draft.		
Description:	Uploads media and creates an owned track in Draft status.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target track.		
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.		
Normal Flow	NF01: Create track draft 1. The Listener selects Upload Track. 2. The Listener enters title, genre, metadata, audio, and optional cover. 3. The system validates mandatory data and media constraints. 4. The system invokes UC-31 to upload media. 5. The system creates the owned track with DRAFT status.		
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.		
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.		
Priority:	Must Have		
Frequency of Use:	Medium		
Business Rules:	BR-21, BR-23, BR-24, BR-36		
Other Information:	Sub-function of UC-19 Manage My Tracks. The approved parent Use Case remains unchanged.		

Assumptions:	The Listener holds rights to uploaded content.
---------------------	--

8.2 UC-19.2 Manage my Tracks (View)

UC ID and Name:	UC-19.2 View My Tracks		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener opens View My Tracks		
Description:	Retrieves owned tracks, metadata, and workflow status .		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target track.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow	NF01: Read owned tracks 1. The Listener opens Track Management or a track detail. 2. The system retrieves owned tracks and their submission status. 3. The system displays permitted actions.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		
Priority:	Must Have		
Frequency of Use:	Meidum		
Business Rules:	BR-21, BR-23, BR-24, BR-36		
Other Information:	Sub-function of UC-19 Manage My Tracks. The approved parent Use Case remains unchanged.		
Assumptions:	The Listener holds rights to uploaded content.		

8.3 UC-19.3 Manage my Tracks (Update Status)

UC ID and Name:	UC-19.3 Update Track		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Update Track on an eligible record.		
Description:	Updates an eligible owned Draft or Rejected track.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target track.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow	NF01: Update track 1. The Listener selects Edit on an owned DRAFT or REJECTED track. 2. The Listener changes metadata or media. 3. The system validates ownership, status, and input. 4. The system processes replacement media through UC-31 when required. 5. The system saves the changes.		
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.		
Exceptions:	1EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.		
Priority:	Must Have		

Frequency of Use:	Medium
Business Rules:	BR-21, BR-23, BR-24, BR-36
Other Information:	Sub-function of UC-19 Manage My Tracks. The approved parent Use Case remains unchanged.
Assumptions:	The Listener holds rights to uploaded content.

8.4 UC-19.4 Manage my Tracks (Delete)

UC ID and Name:	UC-19.4 Delete Track		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Delete Track and confirms the action.		
Description:	Deletes an eligible owned unapproved track and obsolete media.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target track.		
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.		
Normal Flow	NF01: Delete track 1. The Listener selects Delete on an eligible owned unapproved track and confirms. 2. The system validates status and dependencies. 3. The system deletes related draft data and invokes UC-31 for obsolete media.		
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged.		
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.		

Priority:	Must Have
Frequency of Use:	Medium
Business Rules:	BR-21, BR-23, BR-24, BR-36
Other Information:	Sub-function of UC-19 Manage My Tracks. The approved parent Use Case remains unchanged.
Assumptions:	The Listener holds rights to uploaded content.

8.5 UC-20 View Rejection Reason

UC ID and Name:	UC-20 View Rejection Reason		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	A Listener opens rejection details for an owned rejected track.		
Description:	Displays the latest Staff rejection feedback and available correction actions.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The owned track or submission is REJECTED and contains feedback.		
Postconditions:	POST-1: The reason is displayed without modifying moderation data.		
Normal Flow	NF01: View rejection reason 1. The Listener opens a REJECTED track. 2. The system validates ownership. 3. The system retrieves the latest rejection reason, reviewer time, and permitted actions. 4. The system displays the details. 5. The Listener may continue to UC-19 Update Track.		
Alternative Flows:	AF01: View prior submission feedback 1. More than one rejected submission exists. 2. The Listener selects a prior entry and the system displays its read-only feedback.		
Exceptions:	EX01: Reason unavailable 1. The selected rejection has no readable feedback. 2. The system displays a generic moderation notice and logs the data inconsistency. EX02: Unauthorized access		

	1. Ownership validation fails. 2. The system denies access.
Priority:	Should Have
Frequency of Use:	Low.
Business Rules:	BR-20, BR-24
Other Information:	The displayed reason is Staff-authored moderation feedback.
Assumptions:	Rejected records are retained for correction and resubmission.

9. Album Management

9.1 UC-21.1 Manage my Album (Create)

UC ID and Name:	UC-21.1 Create Album		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Create Album.		
Description:	Creates a new owned album or EP in Draft status.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target album.		
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.		
Normal Flow	NF01: Create album 1. The Listener selects Create Album. 2. The Listener enters title, description, release date, and optional cover. 3. The system validates the data and invokes UC-31 for the cover when supplied. 4. The system creates the owned album with DRAFT status.		
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.		
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation.		

	EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-18, BR-22, BR-23
Other Information:	Sub-function of UC-21 Manage My Album. The approved parent Use Case remains unchanged.
Assumptions:	Album visibility is derived from approved component tracks.

9.2 UC-21.2 Manage my Album (Read)

UC ID and Name:	UC-21.2 View Albums		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener opens the Read Album.		
Description:	Retrieves owned albums, metadata, and track membership.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target album.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow	NF01: Read albums 1. The Listener opens My Albums or an owned album. 2. The system retrieves owned album metadata and track membership. 3. The system displays details and permitted actions.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		

Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-18, BR-22, BR-23
Other Information:	Sub-function of UC-21 Manage My Album. The approved parent Use Case remains unchanged.
Assumptions:	Album visibility is derived from approved component tracks.

9.3 UC-21.3 Manage my Album (Update)

UC ID and Name:	UC-21.3 Update Albums		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	Cloud Storage
Trigger:	The Listener selects Update Album on an eligible record.		
Description:	Updates owned album metadata, cover, track membership, and order.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target album.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow	NF01: Update album 1. The Listener selects Edit on an owned album. 2. The Listener changes metadata, cover, track membership, or track order. 3. The system validates ownership and that selected tracks are owned. 4. The system saves the changes and recalculates public eligibility.		
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.		
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails.		

	2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-18, BR-22, BR-23
Other Information:	Sub-function of UC-21 Manage My Album. The approved parent Use Case remains unchanged.
Assumptions:	Album visibility is derived from approved component tracks.

9.4 UC-21.4 Manage my Album (Delete)

UC ID and Name:	UC-21.4 Delete Albums		
Created By:	PhucDTT	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Delete Album and confirms the action.		
Description:	Deletes an owned album while retaining and unlinking its tracks.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: The Listener owns any existing target album.		
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.		
Normal Flow	NF01: Delete album 1. The Listener selects Delete and confirms. 2. The system validates ownership. 3. The system unlinks tracks without deleting them, deletes the album, and removes obsolete cover media.		
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation.		

	2. The system leaves stored data unchanged.
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-18, BR-22, BR-23
Other Information:	Sub-function of UC-21 Manage My Album. The approved parent Use Case remains unchanged.
Assumptions:	Album visibility is derived from approved component tracks.

10. Content Reporting Feature

10.1 UC-22.1 Manage Content Report (Create)

UC ID and Name:	UC-22.1 Create Content Report		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Create Content Report.		
Description:	Submits a new policy or copyright report against a published track.		
Preconditions:	PRE-1: UC-01 has completed. PRE-2: A new report targets a PUBLISHED track.		
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.		
Normal Flow	NF01: Create report 1. The Listener selects Report on a PUBLISHED track. 2. The Listener selects a predefined category and enters details. 3. The system validates eligibility and required data. 4. The system creates the report with PENDING status. 5. The system confirms submission.		

Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.
Priority:	Should Have
Frequency of Use:	Low
Business Rules:	BR-30, BR-31, BR-32, BR-36
Other Information:	Sub-function of UC-22 Manage Content Report. The approved parent Use Case remains unchanged.
Assumptions:	Predefined report categories are configured.

10.2 UC-22.2 Manage Content Report (Read)

UC ID and Name:	UC-22.2 View Content Report		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener opens Read Content Report.		
Description:	Retrieves reports submitted by the current Listener and their status.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow	NF01: Read reports 1. The Listener opens Content Report Management or a report detail. 2. The system retrieves reports submitted by the Listener. 3. The system displays status, target, timestamps, and resolution when available.		

Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.
Priority:	Should Have
Frequency of Use:	Low
Business Rules:	BR-30, BR-31, BR-32, BR-36
Other Information:	Sub-function of UC-22 Manage Content Report. The approved parent Use Case remains unchanged.
Assumptions:	Predefined report categories are configured.

10.3 UC-22.3 Manage Content Report (Update)

UC ID and Name:	UC-22.3 Update Content Report		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Update Content Report on an eligible record.		
Description:	Updates category or description of an owned Pending report.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow:	NF01: Update pending report 1. The Listener selects Edit on an owned PENDING report.		

	2. The Listener changes category or description. 3. The system validates ownership and current status. 4. The system saves the changes.
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Should Have
Frequency of Use:	Low
Business Rules:	BR-30, BR-31, BR-32, BR-36
Other Information:	Sub-function of UC-22 Manage Content Report. The approved parent Use Case remains unchanged.
Assumptions:	Predefined report categories are configured.

10.4 UC-22.4 Manage Content Report (Delete)

UC ID and Name:	UC-22.4 Withdraw Content Report (Delete)		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects Withdraw Content Report (Delete) and confirms the action.		
Description:	Withdraws an owned Pending report from moderation.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.		
Normal Flow:	NF01: Withdraw report 1. The Listener selects Withdraw on an owned PENDING report and confirms. 2. The system marks the report withdrawn or removes it according to retention policy. 3. The system removes it from the Staff pending queue.		
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged.		
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.		

Priority:	Should Have
Frequency of Use:	Low
Business Rules:	BR-30, BR-31, BR-32, BR-36
Other Information:	Sub-function of UC-22 Manage Content Report. The approved parent Use Case remains unchanged.
Assumptions:	Predefined report categories are configured.

11. System Interaction

11.1 UC-23.1: Manage System Review (Create)

UC ID and Name:	UC-23.1 Create Submit System Review		
Created By:		Date Created:	08/10/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener selects "Feedback / Rate Us" in the footer, account dropdown, or feedback prompt.		
Description:	Allows an authenticated user to submit an application rating and feedback comments. The system validates input and applies an automated profanity filter to reject inappropriate or vulgar language.		
Preconditions:	PRE-1: UC-01 has completed with Listener role. PRE-2: The account is verified (email_verified_at is not null) and status is ACTIVE.		
Postconditions:	POST-1: The verified review record is saved to the database (system_reviews). POST-2: A confirmation message is displayed to the Listener.		

Normal Flow:	NF01: Submit review 1. The Listener opens the System Review dialog. 2. The Listener chooses a star rating (1 to 5) and enters optional feedback text. 3. The Listener clicks "Submit Review". 4. The system validates basic constraints (star rating selected, text within allowed character limit). 5. The backend validation module passes the feedback comment through the Profanity Filter. 6. The system confirms the text is compliant, stores the record into system_reviews, and displays a thank-you confirmation.
Alternative Flows:	AF01: Cancel submission 1. The Listener closes the dialog before clicking Submit. 2. The system discards input values without persisting any record.
Exceptions:	EX01: Profanity detected 1. At step 5 of NF01, the comment contains prohibited words, vulgarities, or abusive terms. 2. The system rejects persistence, highlights the text area, and shows an inline error: <i>"Your feedback contains terms that violate community standards. Please rephrase before submitting."</i> 3. The Listener edits the text and may re-attempt submission. EX02: Missing required rating 1. The star rating is not selected; the system blocks submission and indicates the required field. EX03: Database persistence failure 1. The database fails to write the review record; the system alerts the user and preserves the form input for retry.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-07
Other Information:	The automated profanity filter operates as a server-side validation step before database persistence. Each active user may be restricted to a single active system review or a rate limit to prevent spam.

Assumptions:	The profanity dictionary/filter service covers common offensive terms and inappropriate phrasing in the supported system languages (e.g., Vietnamese and English).
---------------------	--

11.2 UC-23.2: Manage System Review (Create)

UC ID and Name:	UC-23.1 View My System Reviews		
Created By:		Date Created:	08/10/2026
Primary Actor:	Listener	Secondary Actors:	None
Trigger:	The Listener opens "My Feedback & Reviews" in their personal Profile or Library menu.		
Description:	Retrieves and displays historical system ratings and comments submitted by the authenticated Listener.		
Preconditions:	PRE-1: UC-01 has completed.		
Postconditions:	POST-1: Authorized review records are displayed in read-only format without modifying business data.		
Normal Flow:	NF01: Read review history 1. The Listener navigates to the "My Reviews" section. 2. The system verifies identity and queries system_reviews for records where user_id = current_user_id. 3. The system returns the reviews in reverse chronological order (newest first). 4. The system renders each entry showing: star rating, comment text, submission timestamp, and moderation/acknowledgment status.		
Alternative Flows:	AF01: No reviews found 1. The query returns 0 records for the current user. 2. The system displays an empty state with a button linking to UC-23.1 to submit a new review.		

Exceptions:	EX01: Unauthorized access 1. Session authentication expires or ownership validation fails. 2. The system redirects to Login (UC-01). EX02: Retrieval failure 1. The database query fails. 2. The system displays retry controls and logs the error.
Priority:	Should Have
Frequency of Use:	Low
Business Rules:	BR-07
Other Information:	Sub-function of UC-23 Manage System Review. The approved parent Use Case remains unchanged.
Assumptions:	Submitted reviews are preserved persistently even if related profile details change.

12. Content Moderation

12.1 UC-24.1: Manage Tracks Moderation (Read)

UC ID and Name:	UC-24.1 View Track Moderation		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Staff	Secondary Actors:	Cloud Storage
Trigger:	The Staff opens View Track Moderation.		
Description:	Retrieves pending or historical submissions and displays review information.		
Preconditions:	PRE-1: UC-01 has completed with Staff permission.		

	PRE-2: A pending submission exists for decision flows.
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.
Normal Flow:	NF01: Read moderation queue 1. Staff opens Track Moderation. 2. The system retrieves pending submissions and permitted moderation history. 3. The system displays status, uploader, metadata, and submission time. NF02: Review submission 1. Staff opens a PENDING submission. 2. The system displays metadata and invokes UC-30 for controlled audio playback. 3. Staff assesses quality and policy compliance.
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.
Priority:	Must Have
Frequency of Use:	High
Business Rules:	BR-19, BR-20, BR-21, BR-24, BR-33, BR-34, BR-38
Other Information:	Sub-function of UC-24 Manage Track Moderation. The approved parent Use Case remains unchanged.
Assumptions:	Staff follows the approved moderation policy.

12.2 UC-24.2 Manage Tracks Moderation (Update)

UC ID and Name:	UC-24.2 Update Track Moderation		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Staff	Secondary Actors:	Cloud Storage
Trigger:	The Staff selects Update Track Moderation on an eligible record.		
Description:	Updates a Pending submission with an approval or rejection decision.		
Preconditions:	PRE-1: UC-01 has completed with Staff permission. PRE-2: A pending submission exists for decision flows.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow:	NF01: Approve track 1. After NF02, Staff selects Approve and confirms. 2. The system rechecks that the submission is PENDING. 3. The system sets the track to PUBLISHED and records reviewer and time. 4. The system creates in-app and email notifications. NF02: Reject track 1. After NF02, Staff selects Reject. 2. Staff enters the mandatory rejection reason and confirms. 3. The system validates that the submission is PENDING and the reason is present. 4. The system sets the track to REJECTED and stores feedback, reviewer, and time.		

	5. The system creates in-app and email notifications.
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Must Have

Frequency of Use:	High
Business Rules:	BR-19, BR-20, BR-21, BR-24, BR-33, BR-34, BR-38
Other Information:	Sub-function of UC-24 Manage Track Moderation. The approved parent Use Case remains unchanged.
Assumptions:	Staff follows the approved moderation policy.

12.3 UC-25.1: Resolve Content Reports (Read)

UC ID and Name:	UC-25.1 View Content Reports		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	The Staff opens Read Content Reports.		
Description:	Retrieves pending and resolved reports for Staff assessment.		
Preconditions:	PRE-1: UC-01 has completed with Staff permission. PRE-2: A pending report exists for decision flows.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow:	NF01: Read report queue 1. Staff opens Report Moderation. 2. The system lists pending reports and permits access to resolved history.		

	3. Staff opens a report and reviews reporter, category, description, and target track.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		
Priority:	Must Have		
Frequency of Use:	High		
Business Rules:	BR-32, BR-33, BR-34, BR-38		
Other Information:	Sub-function of UC-25 Resolve Content Reports. The approved parent Use Case remains unchanged.		
Assumptions:	Staff can inspect the target track and applicable policy.		

12.4 UC-25.2 Resolve Content Reports (Update)

UC ID and Name:	UC-25.2 Update Content Report Resolution		
Created By:	TriNM	Date Created:	26/09/2026

Primary Actor:	Staff	Secondary Actors:	None
Trigger:	The Staff selects Update Content Report Resolution on an eligible record.		
Description:	Updates a Pending report with a dismissed or confirmed-violation outcome.		
Preconditions:	PRE-1: UC-01 has completed with Staff permission. PRE-2: A pending report exists for decision flows.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow:	NF01: Dismiss report 1. After NF01, Staff determines that no violation exists. 2. Staff enters a resolution note and confirms Dismiss. 3. The system rechecks PENDING status. 4. The system sets the report to REJECTED or the approved dismissed status and records handler details. 5. The system notifies the reporter. NF02: Confirm policy breach 1. After NF01, Staff determines that the track violates policy. 2. Staff enters the resolution note and selects Confirm Violation. 3. The system preserves the report as pending action and opens UC-26. 4. Only after UC-26 succeeds, the system marks the report RESOLVED. 5. The system notifies the reporter.		

Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values. 2. The system returns the current record without creating a duplicate change.
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Must Have
Frequency of Use:	High

Business Rules:	BR-32, BR-33, BR-34, BR-38
Other Information:	Sub-function of UC-25 Resolve Content Reports. The approved parent Use Case remains unchanged.
Assumptions:	Staff can inspect the target track and applicable policy.

12.5 UC-26: Take Down Published Track

UC ID and Name:	UC-26: Take Down Published Track		
Created By:	TriNM	Date Created:	26/09/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	Authorized Staff confirms takedown for a violating published track.		
Description:	Removes public access to an infringing track and completes the related report resolution atomically.		
Preconditions:	PRE-1: UC-01 has completed with Staff permission. PRE-2: The target track is PUBLISHED. PRE-3: The action is supported by a confirmed violation or authorized moderation reason.		
Postconditions:	POST-1: The track is no longer publicly discoverable or playable. POST-2: The related report is RESOLVED when supplied. POST-3: Notifications and audit data are recorded.		
Normal Flow:	NF01: Take down track		

	1. Staff opens Track Takedown Confirmation. 2. The system displays the track, reason, impact, and related report. 3. Staff confirms the action. 4. In one transaction, the system changes the track to the approved removed/suspended status, resolves the report, records handler data, and creates notifications. 5. The system invalidates public catalog/cache access. 6. The system confirms completion.
Alternative Flows:	AF01: Cancel takedown 1. Staff cancels at step 3 of NF01. 2. The system makes no track or report change and returns to review. AF02: Takedown without report 1. An authorized moderation process supplies a direct policy reason. 2. The system performs the track-status and notification parts of NF01 without a report update.
Exceptions:	EX01: Track already unavailable 1. The track is no longer PUBLISHED at step 4. 2. The system does not duplicate the action and reloads current status. EX02: Transaction failure 1. Any track, report, or audit update fails. 2. The system rolls back all transactional changes and offers retry. EX03: Unauthorized action 1. Staff permission validation fails.

	2. The system denies the request and records the attempt. EX04: Notification failure 1. The transaction commits but email delivery fails. 2. The system keeps the takedown, logs the failure, and retries email.
Priority:	Must Have
Frequency of Use:	Medium
Business Rules:	BR-09, BR-32, BR-33, BR-34, BR-38
Other Information:	Uses soft takedown; physical media may be retained as moderation evidence.
Assumptions:	A defined track status exists for removed or suspended content.

13. System Administration

13.1 UC-27.1: Manage User Account (Ban)

UC ID and Name:	UC-27.1: Ban User Account		
Created By:	DuyLH	Date Created:	02/10/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin clicks Ban User on an Active user's account details.		
Description:	Suspends an Active user account and immediately revokes its active sessions.		
Preconditions:	PRE-1: UC-01 has completed with Admin permission.		

	PRE-2: The target account is currently ACTIVE. PRE-3: The target account is eligible for administrative suspension.
Postconditions:	POST-1: The target account status is BANNED. POST-2: All active access and refresh tokens of the target account are revoked. POST-3: The action and notification are recorded.
Normal Flow:	NF01: Ban user account 1. Admin views the user list and selects an ACTIVE user. 2. Admin clicks Ban User. 3. The system displays a dialog requesting a reason for the ban. 4. Admin enters the reason and clicks Confirm Ban. 5. The system updates the account status in the database, revokes the user's access tokens, and updates the UI. 6. The system sends an automated email notifying the user of the suspension. 7. The use case ends.
Alternative Flows:	AF01: Cancel ban 1. Admin closes the confirmation dialog before confirming. 2. The account remains ACTIVE and no session is revoked.
Exceptions:	EX01: Missing ban justification 1. Admin leaves the ban reason empty. 2. The system blocks confirmation and displays that a reason is required. 3. Admin enters a reason and may continue NF01. EX02: Account status changed 1. The account is no longer ACTIVE when the update is submitted. 2. The system rejects the stale action and reloads the current status. EX03: Notification failure 1. The ban and token revocation succeed but email delivery fails. 2. The system keeps the account BANNED, logs the failure, and retries delivery.
Priority:	Must Have
Frequency of Use:	As needed
Business Rules:	BR-06, BR-33, BR-34, BR-37

Other Information:	Sub-function of UC-27 Manage User Accounts. Banning an account does not automatically remove its published tracks.
Assumptions:	Admin has a valid policy reason for suspending the account.

13.2 UC-27.2 Manage User Account (Unban)

UC ID and Name:	UC-27.2: Unban User Account		
Created By:	DuyLH	Date Created:	02/10/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin clicks Unban User on a Banned user's account details.		
Description:	Restores platform access for a previously Banned user account.		
Preconditions:	PRE-1: UC-01 has completed with Admin permission. PRE-2: The target account is currently BANNED. PRE-3: The account is eligible to have access restored.		
Postconditions:	POST-1: The target account status is ACTIVE. POST-2: The user can authenticate again with a new session. POST-3: The action and notification are recorded.		
Normal Flow:	NF01: Unban user account 1. Admin filters the user list to show suspended accounts and selects a BANNED user. 2. Admin clicks Unban User. 3. The system asks for confirmation: Are you sure you want to restore access for this user? 4. Admin clicks Confirm. 5. The system updates the account status to ACTIVE in the database. 6. The system sends an email notifying the user that the account is restored. 7. The use case ends		

Alternative Flows:	AF01: Cancel unban 1. Admin closes the confirmation dialog before confirming. 2. The account remains BANNED.
Exceptions:	EX01: Account status changed 1. The account is no longer BANNED when the update is submitted. 2. The system rejects the stale action and reloads the current status. EX02: Persistence failure 1. The database update fails. 2. The system keeps the account BANNED and asks Admin to retry. EX03: Notification failure 1. The account becomes ACTIVE but email delivery fails. 2. The system keeps the account ACTIVE, logs the failure, and retries delivery.
Priority:	Must Have
Frequency of Use:	Rare - normally after an approved appeal or administrative review
Business Rules:	BR-06, BR-33, BR-34, BR-37
Other Information:	Sub-function of UC-27 Manage User Accounts.
Assumptions:	Admin has completed the required appeal or administrative review.

13.3 UC-28.1 Manage Genre (Create)

UC ID and Name:	UC-28.1 Create Genre		
Created By:	DuyLH	Date Created:	10/02/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	The Admin selects Create Genre.		

Description:	Creates a new music genre.
Preconditions:	PRE-1: UC-01 has completed with Admin permission.
Postconditions:	POST-1: A valid new record or relationship is stored and displayed.
Normal Flow:	NF01: Create genre 1. Admin selects Add Genre. 2. Admin enters name, slug, description, and active state. 3. The system validates required data and uniqueness. 4. The system creates the genre and refreshes the list.
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record.
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.
Priority:	Must Have
Frequency of Use:	Low
Business Rules:	BR-37
Other Information:	Sub-function of UC-28 Manage Genres. The approved parent Use Case remains unchanged.
Assumptions:	Inactive genres are excluded from new public filtering selections.

13.4 UC-28.2 Manage Genre (Read)

UC ID and Name:	UC-28.2 View Genres		
Created By:	DuyLH	Date Created:	10/02/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	The Admin opens View Genres.		
Description:	Retrieves genres, active state, and usage information.		
Preconditions:	PRE-1: UC-01 has completed with Admin permission.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow:	NF01: Read genres 1. Admin opens Genre Management. 2. The system retrieves genres, active state, and usage count. 3. The system displays searchable and sortable results.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure 1. The data or media query fails. 2. The system displays retry controls and logs the error.		
Priority:	Must Have		
Frequency of Use:	Low		

Business Rules:	BR-37
Other Information:	Sub-function of UC-28 Manage Genres. The approved parent Use Case remains unchanged.
Assumptions:	Inactive genres are excluded from new public filtering selections.

13.5 UC-28.3 Manage Genre (Update)

UC ID and Name:	UC-28.3 Update Genre		
Created By:	DuyLH	Date Created:	10/02/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	The Admin selects Update Genre on an eligible record.		
Description:	Updates permitted genre fields or active state.		
Preconditions:	PRE-1: UC-01 has completed with Admin permission.		
Postconditions:	POST-1: The authorized record reflects the confirmed update and required notifications are recorded.		
Normal Flow:	NF01: Update genre 1. Admin selects Edit on a genre. 2. Admin changes permitted fields. 3. The system validates uniqueness and references. 4. The system saves and displays the updated genre.		
Alternative Flows:	AF01: Cancel update 1. The actor cancels before persistence. 2. The system keeps the current stored state. AF02: No effective change 1. Submitted values match current values.		

	2. The system returns the current record without creating a duplicate change.
Exceptions:	EX01: Invalid update data 1. Required input or a business constraint fails. 2. The system blocks the update and requests correction. EX02: Unauthorized or invalid-status update 1. Permission, ownership, or workflow-state validation fails. 2. The system denies the update. EX03: Concurrent update 1. The record changed after it was loaded. 2. The system rejects stale data and reloads the current record. EX04: Persistence, media, or notification failure 1. A required operation fails. 2. The system rolls back transactional changes or logs and retries a non-transactional notification.
Priority:	Must Have
Frequency of Use:	Low
Business Rules:	BR-37
Other Information:	Sub-function of UC-28 Manage Genres. The approved parent Use Case remains unchanged.
Assumptions:	Inactive genres are excluded from new public filtering selections.

13.6 UC-28.4 Manage Genre (Delete)

UC ID and Name:	UC-28.4 Delete or Deactivate Genre		
Created By:	DuyLH	Date Created:	02/10/2026
Primary Actor:	Admin	Secondary Actors:	None

Trigger:	The Admin selects Delete or Deactivate Genre and confirms the action.
Description:	Deletes an unreferenced genre or deactivates a referenced genre.
Preconditions:	PRE-1: UC-01 has completed with Admin permission.
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.
Normal Flow:	NF01: Delete genre 1. Admin selects Delete and confirms. 2. The system verifies that no protected track reference prevents deletion. 3. The system deletes the unreferenced genre and refreshes filters.
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged. AF02: Deactivate referenced genre 1. Track references prevent physical deletion. 2. Admin confirms Deactivate. 3. The system sets the genre inactive and retains historical references.
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.
Priority:	Must Have
Frequency of Use:	Low
Business Rules:	BR-37

Other Information:	Sub-function of UC-28 Manage Genres. The approved parent Use Case remains unchanged.
Assumptions:	Inactive genres are excluded from new public filtering selections.

13.7 UC-29 View System Statistics

UC ID and Name:	UC-29 View System Statistics		
Created By:	DuyLH	Date Created:	02/10/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin opens System Statistics.		
Description:	Displays aggregate account, catalog, listening, moderation, report, and traffic metrics.		
Preconditions:	PRE-1: UC-01 has completed with Admin permission.		
Postconditions:	POST-1: Authorized aggregate statistics are displayed; operational data is unchanged.		
Normal Flow:	NF01: View statistics 1. Admin opens System Statistics. 2. The system validates Admin permission. 3. The system retrieves aggregate metrics for the default period. 4. The system displays summary cards, trends, and category breakdowns. 5. Admin may change the supported date range or filter. 6. The system refreshes the aggregates.		
Alternative Flows:	AF01: No data for period 1. The selected period contains no records. 2. The system displays zero values and an explanatory empty state. AF02: Cached statistics 1. Live aggregation is expensive.		

	2. The system displays the latest valid cached snapshot with its update time.
Exceptions:	EX01: Statistics query failure 1. An aggregate query or analytics source fails. 2. The system displays unavailable widgets separately and offers retry. EX02: Unauthorized access 1. Permission validation fails. 2. The system denies access and records the attempt.
Priority:	Should Have
Frequency of Use:	Medium
Business Rules:	BR-37
Other Information:	Metrics contain aggregates rather than exposed private content.
Assumptions:	Required event and operational data are retained.

14. System Process

14.1 UC-30.1: Process Cloud Media (Create)

UC ID and Name:	UC-30.1 Upload Cloud Media		
Created By:	TriNM	Date Created:	02/10/2026
Primary Actor:	System	Secondary Actors:	Cloud Storage
Trigger:	The System selects Upload Cloud Media.		
Description:	Uploads and verifies a new authorized media asset.		
Preconditions:	PRE-1: The parent use case authorized the operation. PRE-2: Required provider credentials and target configuration are available.		

Postconditions:	POST-1: A valid new record or relationship is stored and displayed.
Normal Flow:	NF01: Create media 1. The parent use case submits a file and target metadata. 2. The system validates type, size, and target folder. 3. The system uploads the asset to Cloud Storage. 4. The system verifies the provider response and returns public ID, secure URL, format, and relevant metadata.
Alternative Flows:	AF01: Cancel creation 1. The actor cancels before persistence. 2. The system discards unsaved input and creates no record. AF02: Replace media 1. The parent submits new media and the current media identifier. 2. The system completes NF01 for the new asset. 3. After the new asset is verified and referenced, the system deletes or schedules deletion of the old asset. 4. The system returns the replacement result.
Exceptions:	EX01: Invalid or duplicate data 1. Validation or uniqueness checking fails. 2. The system identifies the affected fields and creates no record. EX02: Unauthorized or ineligible target 1. Permission, ownership, status, or eligibility validation fails. 2. The system denies creation. EX03: Persistence or media failure 1. A database or Cloud Storage operation fails. 2. The system avoids a partial record and offers retry.
Priority:	Must Have
Frequency of Use:	Very High
Business Rules:	BR-08, BR-12, BR-26
Other Information:	Sub-function of UC-30 Process Cloud Media. The approved parent Use Case remains unchanged.

Assumptions:	Cloud Storage supports secure upload, deletion, CDN delivery, and HTTP Range Requests.
---------------------	--

14.2 UC-30.2: Process Cloud Media (Read)

UC ID and Name:	UC-30.2 Retrieve/Stream Cloud Media		
Created By:	TriNM	Date Created:	02/10/2026
Primary Actor:	System	Secondary Actors:	Cloud Storage
Trigger:	The System opens Retrieve/Stream Cloud Media.		
Description:	Retrieves an authorized resource, delivery URL, or audio byte range.		
Preconditions:	PRE-1: The parent use case authorized the operation. PRE-2: Required provider credentials and target configuration are available.		
Postconditions:	POST-1: Authorized current data is displayed without modifying stored business data.		
Normal Flow:	NF01: Read or stream media 1. The parent use case submits an authorized media identifier or range request. 2. The system validates access and media state. 3. Cloud Storage returns the resource, CDN URL, or requested byte range. 4. The system returns the result to the parent use case.		
Alternative Flows:	AF01: No matching records 1. The query returns no permitted record. 2. The system displays an empty state and the appropriate navigation action.		
Exceptions:	EX01: Unauthorized access 1. Permission or ownership validation fails. 2. The system denies access. EX02: Retrieval failure		

	1. The data or media query fails. 2. The system displays retry controls and logs the error.
Priority:	Must Have
Frequency of Use:	Very High
Business Rules:	BR-08, BR-12, BR-26
Other Information:	Sub-function of UC-30 Process Cloud Media. The approved parent Use Case remains unchanged.
Assumptions:	Cloud Storage supports secure upload, deletion, CDN delivery, and HTTP Range Requests.

14.3 UC-30.3: Process Cloud Media (Delete)

UC ID and Name:	UC-30.3 Delete Cloud Media		
Created By:	TriNM	Date Created:	02/10/2026
Primary Actor:	System	Secondary Actors:	Cloud Storage
Trigger:	The System selects Delete Cloud Media and confirms the action.		
Description:	Deletes an authorized media asset or confirms the desired deleted state.		
Preconditions:	PRE-1: The parent use case authorized the operation. PRE-2: Required provider credentials and target configuration are available.		
Postconditions:	POST-1: The eligible owned record or relationship is removed, withdrawn, or deactivated according to the approved rule.		
Normal Flow:	NF01: Delete media		

	1. The parent submits an authorized media identifier. 2. The system verifies that deletion is permitted. 3. Cloud Storage removes the asset or confirms it is already absent. 4. The system returns the deletion result.
Alternative Flows:	AF01: Cancel deletion 1. The actor cancels the confirmation. 2. The system leaves stored data unchanged
Exceptions:	EX01: Unauthorized or ineligible deletion 1. Permission, ownership, status, or dependency validation fails. 2. The system denies deletion. EX02: Record already absent 1. The target was already removed. 2. The system returns the current state without repeating destructive work. EX03: Persistence or media cleanup failure 1. The delete transaction or required cleanup fails. 2. The system rolls back inconsistent changes or queues safe cleanup.
Priority:	Must Have
Frequency of Use:	Very High
Business Rules:	BR-08, BR-12, BR-26
Other Information:	Sub-function of UC-30 Process Cloud Media. The approved parent Use Case remains unchanged.
Assumptions:	Cloud Storage supports secure upload, deletion, CDN delivery, and HTTP Range Requests.

III. Design Specifications

1. User Authentication

1.1 System Access

a. User Login

This screen allows registered users to log in to SoundWave using their email and password. The system automatically identifies the user's role and grants the appropriate permissions.

UI Design

Login

Email

user@example.com

Password

.....

☒ Remember Me

[Forgot Password](#)

Login

Don't have an account?

[Register](#)

Field Name	Field Type	Description
Email	Textbox	Allows the user to enter their registered email.
Password	Password field	Allows the user to enter their password.
Show Password	Icon button	Shows or hides the entered password.
Remember Me	Checkbox	Keeps the user signed in.
Login	Button	Submits the login information.
Forgot Password	Link	Opens the password recovery screen.
Register	Link	Opens the account registration screen.
Error Message	Message	Displays a custom validation or login error.

Database Access

Table	CRUD	Description
app_users	R, U	Reads account information and updates the last login time.
roles	R	Reads the role assigned to the account.
refresh_tokens	C	Stores the refresh token after successful login.

SQL Commands

- Find account
SELECT u.id, u.email, u.password_hash, u.status, u.email_verified_at, u.deleted_at, r.code AS

```
role_code FROM app_users AS u INNER JOIN roles AS r ON r.id = u.role_id WHERE u.email = ? AND u.deleted_at IS NULL;
```

2. Update login time
UPDATE app_users SET last_login_at = SYSUTCDATETIME(), updated_at = SYSUTCDATETIME() WHERE id = ?;
3. Store refresh-token hash
INSERT INTO refresh_tokens (user_id, token_hash, expires_at, created_at) VALUES (?, ?, ?, SYSUTCDATETIME());

#note

The password is verified against password_hash by Spring Security PasswordEncoder. Account status and email verification are checked by the service layer.

b. Register Account

This screen allows guests to create a SoundWave account. A verification email is sent after successful registration.

UI Design

Register

Display Name

John Doe

Email

john.doe@example.com

Password

.....

Confirm Password

.....

Password Rules:

- Minimum 8 characters, at least 1 number, and 1 symbol

☒ Accept Terms

Register

Already have an account? [Login](#)

Field Name	Field Type	Description
Display Name	Textbox	Allows the user to enter their display name.
Email	Email field	Allows the user to enter a valid email address.
Password	Password field	Allows the user to create a password.
Confirm Password	Password field	Confirms the entered password.
Show Password	Icon button	Shows or hides the password.
Password Rules	Indicator	Displays the password requirements.
Accept Terms	Checkbox	Confirms acceptance of the terms and policies.
Register	Button	Submits the registration information.
Login	Link	Opens the login screen.
Validation Message	Message	Displays invalid or missing information.

Database Access

Table	CRUD	Description
roles	R	Reads the default Listener role.
app_users	R, C	Checks the email and creates the account.
user_profiles	C	Creates the initial user profile.
email_verification_tokens	C	Stores the email verification token hash.

SQL Command

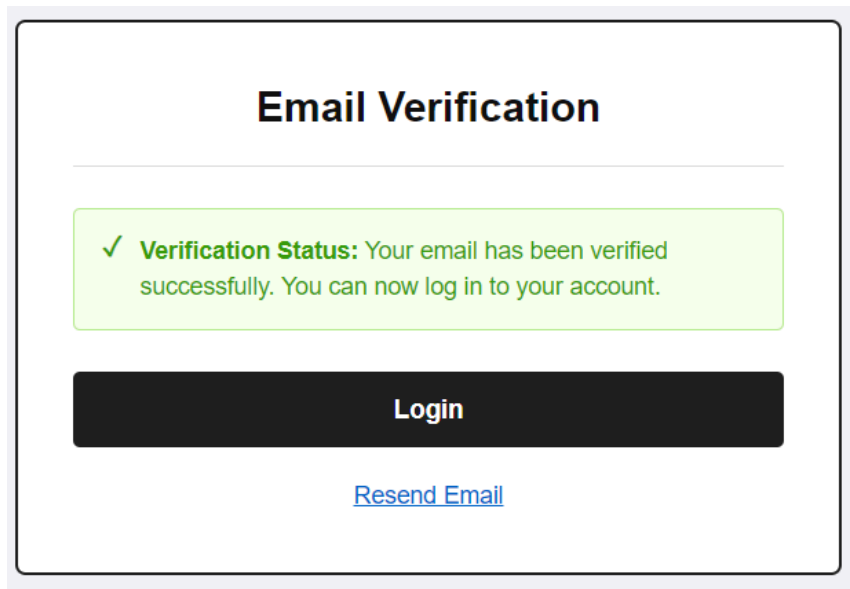
1. Check registered email
SELECT id FROM app_users WHERE email = ? AND deleted_at IS NULL;
2. Find default role
SELECT id FROM roles WHERE code = ?;
3. Create user account
role_id, email, password_hash, status, created_at, updated_at) OUTPUT [INSERTED.id](#)
VALUES (?, ?, ?, SYSUTCDATETIME(), SYSUTCDATETIME());
4. Create user profile
INSERT INTO user_profiles (user_id, username, display_name, created_at, updated_at)
VALUES (?, ?, ?, SYSUTCDATETIME(), SYSUTCDATETIME());
5. Store verification-token hash
INSERT INTO email_verification_tokens (user_id, token_hash, expires_at, created_at) VALUES
(?, ?, ?, SYSUTCDATETIME());

c. Verify Email

This screen displays the result of email verification. A verified account can access authenticated features.

1. UC-03: Verify Email

UI Design



Field Name	Field Type	Description
Verification Status	Message	Displays the verification result.
Login	Button	Opens the login screen after verification.
Resend Email	Link	Sends a new verification email if needed.
Error Message	Message	Displays an expired or invalid link error.

Database Access

Table	CRUD	Description
email_verification_tokens	R, U	Validates the token and marks it as used.
app_users	R, U	Verifies the email and activates the account.

SQL Command

1. Validate verification token
SELECT t.id, t.user_id FROM email_verification_tokens AS t INNER JOIN app_users AS u ON u.id = t.user_id WHERE t.token_hash = ? AND t.used_at IS NULL AND t.expires_at > SYSUTCDATETIME() AND u.deleted_at IS NULL;
2. Verify user email
UPDATE app_users SET email_verified_at = SYSUTCDATETIME(), status = ?, updated_at = SYSUTCDATETIME() WHERE id = ?;
3. Mark token as used
UPDATE email_verification_tokens SET used_at = SYSUTCDATETIME() WHERE id = ?;

d. Forgot Password

This screen allows users to request a password-reset link using their registered email.

UI Design

Forgot Password

mail

user@example.com

Send Reset Link

[Back to Login](#)

✓ **Result Message:** A password reset link has been sent to your email address.

mail

user@example.com

Send Reset Link

[Back to Login](#)

Forgot Password

⚠ **Validation Message:** Please enter a valid registered email address.

mail

user@example.com

Send Reset Link

[Back to Login](#)

Field Name	Field Type	Description
mail	Email field	Allows the user to enter their registered email.
Send Reset Link	Button	Sends a password-reset email.
Back to Login	Link	Returns to the login screen.
Result Message	Message	Displays the request result.
Validation Message	Message	Displays an invalid or missing email error.

Database Access

Table	CRUD	Description
app_users	R	Finds the account associated with the email.
password_reset_tokens	C, U	Invalidates old tokens and stores a new token hash.

SQL Command

1. Find user account
SELECT id, email, status FROM app_users WHERE email = ? AND deleted_at IS NULL;
2. Invalidate previous reset tokens
UPDATE password_reset_tokens SET used_at = SYSUTCDATETIME() WHERE user_id = ? AND used_at IS NULL;
3. Store password-reset token hash
INSERT INTO password_reset_tokens (user_id, token_hash, expires_at, created_at)VALUES (?, ?, SYSUTCDATETIME());

e. Reset Password

This screen allows users to create a new password using a valid password-reset link.

UI Design

Reset Password

New Password

.....

Confirm Password

.....

Save Password

Reset Password

Validation Message: Passwords do not match or the token is invalid.

New Password

.....

Confirm Password

.....

Save Password

Field Name	Field Type	Description
------------	------------	-------------

New Password	Password field	Allows the user to enter a new password.
Confirm Password	Password field	Confirms the new password.
Show Password	Icon button	Shows or hides the entered password.
Save Password	Button	Updates the account password.
Validation Message	Message	Displays password or token errors.

Database Access

Table	CRUD	Description
password_reset_tokens	R, U	Validates and marks the reset token as used.
app_users	U	Updates the account password hash.
refresh_tokens	U	Revokes existing login sessions.

SQL Command

1. Validate password-reset token
SELECT id, user_id FROM password_reset_tokens WHERE token_hash = ? AND used_at IS NULL AND expires_at > SYSUTCDATETIME();
2. Update password hash
UPDATE app_users SET password_hash = ?, updated_at = SYSUTCDATETIME() WHERE id = ? AND deleted_at IS NULL;
3. Mark reset token as used
UPDATE password_reset_tokens SET used_at = SYSUTCDATETIME() WHERE id = ?;
4. Revoke existing sessions
UPDATE refresh_tokens SET revoked_at = SYSUTCDATETIME() WHERE user_id = ? AND revoked_at IS NULL;

f. Logout

This function allows authenticated users to securely end their current session.

UI Design



Logout



Are you sure you want to end your current session?
You will need to log in again to access your account.

Confirm Logout

Field Name	Field Type	Description
Logout	Button	Ends the current user session.
Confirm Logout	Dialog button	Confirms the logout action, if required.
Logout	Button	Ends the current user session.

Database Access

Table	CRUD	Description
refresh_tokens	U	Revokes the refresh token of the current session.

SQL Command

1. Revoke current refresh token
UPDATE refresh_tokens SET revoked_at = SYSUTCDATETIME() WHERE user_id = ? AND token_hash = ? AND revoked_at IS NULL;

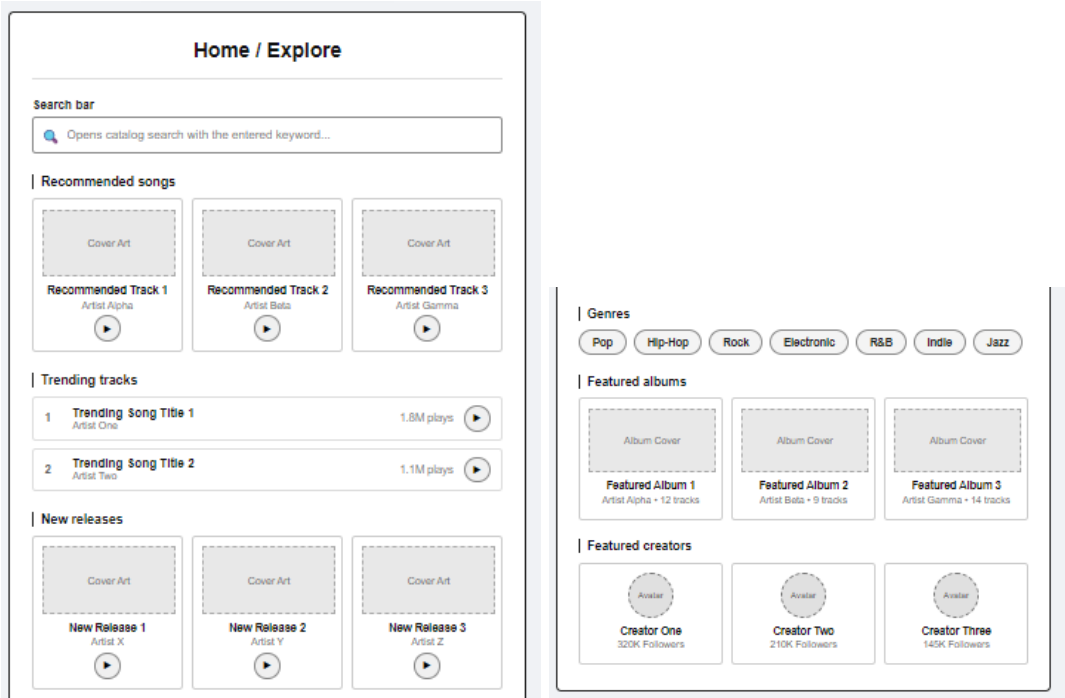
2. Music Discovery

2.1 Public Catalog

a. Explore Music

This screen presents recommended songs, trending tracks, new releases, genres, albums, and creators for discovery.

UI Design



Field Name	Field Type	Description
Search bar	Search input	Opens catalog search with the entered keyword.
Recommended songs	Track card list	Shows suggested public tracks.
Trending tracks	Track list	Shows popular tracks ordered by play count.
New releases	Track card list	Shows recently published tracks.
Genres	Genre chips	Opens music filtered by the selected genre.
Featured albums	Album card list	Opens the selected album details.
Featured creators	Creator card list	Opens the selected creator profile.
Play	Icon button	Starts playback for the selected track.

Database Access

Table	CRUD	Description
tracks	R	Reads approved public tracks and playback statistics.
albums	R	Reads published album information.
genres	R	Reads active genres.

app_users, user_profiles	R	Reads public creator information.
-----------------------------	---	-----------------------------------

SQL Commands

1. Load trending tracks

```
SELECT TOP (12) * FROM tracks WHERE publication_status = 'PUBLISHED' ORDER BY play_count_cache DESC, published_at DESC;
```

2. Load new releases

```
SELECT TOP (12) * FROM tracks WHERE publication_status = 'PUBLISHED' ORDER BY published_at DESC;
```

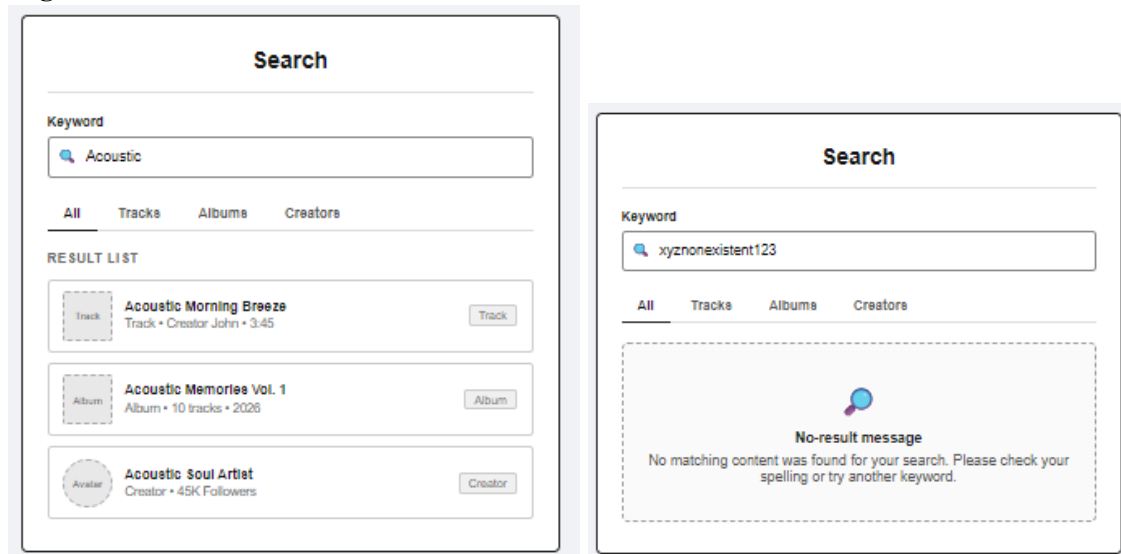
3. Load active genres

```
SELECT id, name, slug FROM genres WHERE is_active = 1 ORDER BY name;
```

b. Search Results

This screen displays public tracks, albums, and creators matching the entered keyword.

UI Design



Field Name	Field Type	Description
Keyword	Search input	Accepts a song, album, or creator keyword.
All	Tab	Shows combined search results.
Tracks	Tab	Shows matching public tracks.
Albums	Tab	Shows matching published albums.
Creators	Tab	Shows matching public creator profiles.
Result list	Card list	Displays results for the selected category.
No-result message	Message	Appears when no matching content is found.

Database Access

Table	CRUD	Description
tracks	R	Searches published tracks.

albums	R	Searches published albums.
app_users, user_profiles	R	Searches creator profiles.

SQL Commands

1. Search tracks

SELECT TOP (@limit) id, title, slug, cover_url FROM tracks WHERE publication_status = 'PUBLISHED' AND title LIKE '%' + @keyword + '%' ORDER BY play_count_cache DESC;

2. Search albums

SELECT TOP (@limit) id, title, slug, cover_url FROM albums WHERE status = 'PUBLISHED' AND title LIKE '%' + @keyword + '%' ORDER BY published_at DESC;

3. Search creators

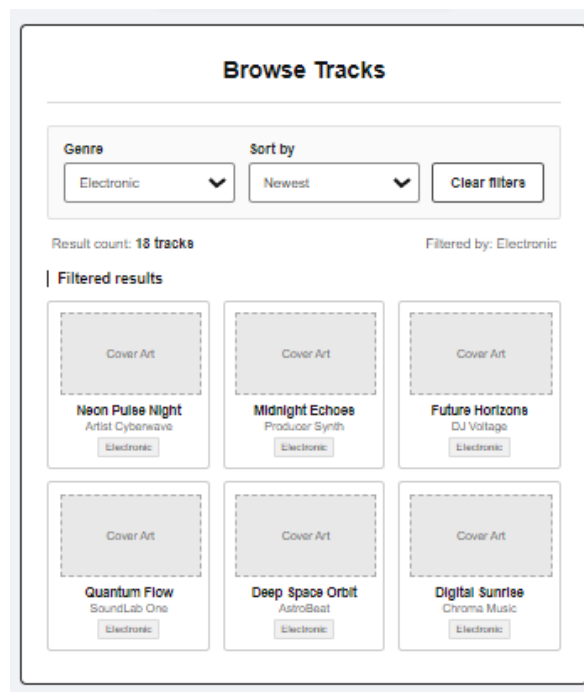
SELECT TOP (@limit) u.id, p.username, p.display_name, p.avatar_url FROM app_users u JOIN user_profiles p ON p.user_id = u.id WHERE u.deleted_at IS NULL AND (p.username LIKE '%' + @keyword + '%' OR p.display_name LIKE '%' + @keyword + '%');

c. Filtered Public Catalog

This screen displays public tracks matching the selected genre and sorting criteria.

UC-08: Filter Public Catalog

UI Design



Field Name	Field Type	Description
Genre	Dropdown	Filters tracks by an active genre.
Sort by	Dropdown	Sorts by newest, most played, or title.
Clear filters	Button	Restores the default catalog view.

Filtered results	Track card list	Displays tracks matching the selected conditions.
Result count	Text	Shows the number of matching tracks.

Database Access

Table	CRUD	Description
genres	R	Loads available filter values.
tracks	R	Reads approved tracks matching the filter.
albums	R	Reads album data displayed on track cards.

SQL Commands

1. Load genre options

```
SELECT id, name FROM genres WHERE is_active = 1 ORDER BY name;
```

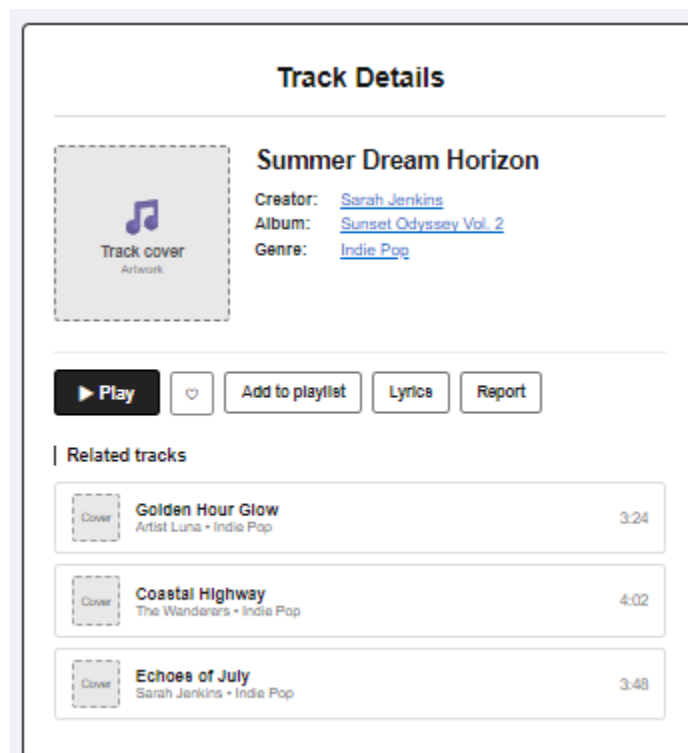
2. Filter tracks

```
SELECT t.id, t.title, t.cover_url, t.duration_ms, a.title AS album_title FROM tracks t LEFT JOIN albums a ON a.id = t.album_id WHERE t.publication_status = 'PUBLISHED' AND (@genreId IS NULL OR t.genre_id = @genreId) ORDER BY t.published_at DESC OFFSET @offset ROWS FETCH NEXT @limit ROWS ONLY;
```

d. Track Details

This screen displays public information about the selected track and provides actions for playback, favorites, playlists, lyrics, and content reporting.

UI Design



Field Name	Field Type	Description
Track cover	Image	Displays the track artwork.

Title	Text	Displays the track title.
Creator	Link	Open the creator profile.
Album	Link	Opens album details when available.
Genre	Link	Opens the corresponding filtered catalog.
Play	Primary button	Starts playback for the selected public track.
Favorite	Icon button	Adds or removes the track from the listener's favorites.
Add to playlist	Action button	Adds the track to a playlist owned by the listener.
Lyrics	Action button	Open the Lyrics Viewer.
Report	Action button	Open the content report form.
Related tracks	Track list	Shows other published tracks in the same genre.

Database Access

Table	CRUD	Description
tracks	R	Reads published track metadata and playable media references.
albums, genres	R	Reads related album and genre information.
app_users, user_profiles	R	Reads public creator information.
favorites	R	Reads the authenticated listener's favorite state.
playlists, playlist_tracks	R	Reads playlists available to the authenticated listener.

SQL Commands

1. Load track details

```
SELECT t.*, a.title AS album_title, g.name AS genre_name, p.display_name AS creator_name FROM
tracks t LEFT JOIN albums a ON a.id = t.album_id JOIN genres g ON g.id = t.genre_id JOIN
user_profiles p ON p.user_id = t.uploader_user_id WHERE t.id = @trackId AND t.publication_status =
'PUBLISHED';
```

2. Load related tracks

```
SELECT TOP (8) id, title, cover_url FROM tracks WHERE genre_id = @genreId AND id <> @trackId
AND publication_status = 'PUBLISHED' ORDER BY play_count_cache DESC;
```

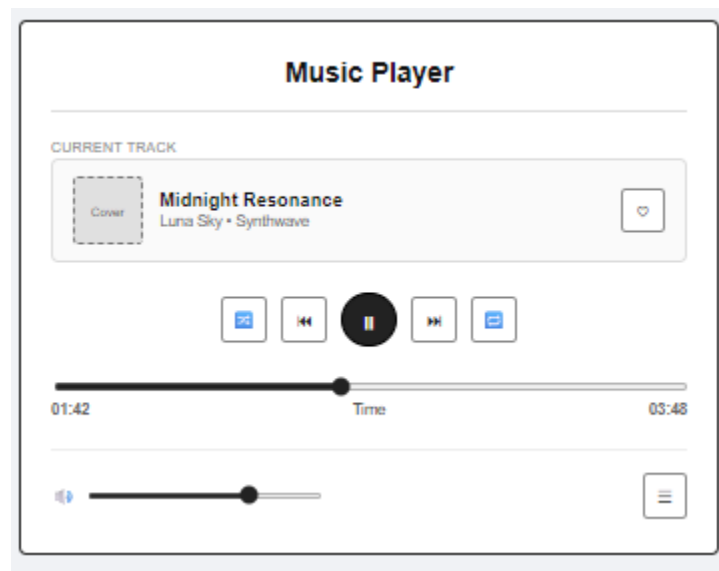
3. Music Playback

3.1 Audio Playback

a. Music Player

This persistent player streams the selected track and provides the main playback controls.

UI Design



Field Name	Field Type	Description
Current track	Track summary	Shows cover, title, and creator.
Previous	Icon button	Plays the previous queue item.
Play or pause	Icon button	Starts or pauses playback.
Next	Icon button	Plays the next queue item.
Progress	Slider	Shows and changes playback position.
Time	Text	Shows elapsed and total duration.
Volume	Slider	Changes playback volume.
Repeat and shuffle	Icon buttons	Changes queue playback behavior.
Favorite	Icon button	Updates the favorite state.
Queue	Icon button	Opens or closes the queue drawer.

Database Access

Table	CRUD	Description
tracks	R, U	Reads playable media and increments the cached play count.
favorites	R, C, D	Reads and updates the favorite state.

SQL Commands

1. Load playable track

```
SELECT id, title, audio_url, audio_format, duration_ms, cover_url FROM tracks WHERE id = @trackId AND publication_status = 'PUBLISHED';
```

2. Record a valid play event and update history

```
BEGIN TRANSACTION;
```

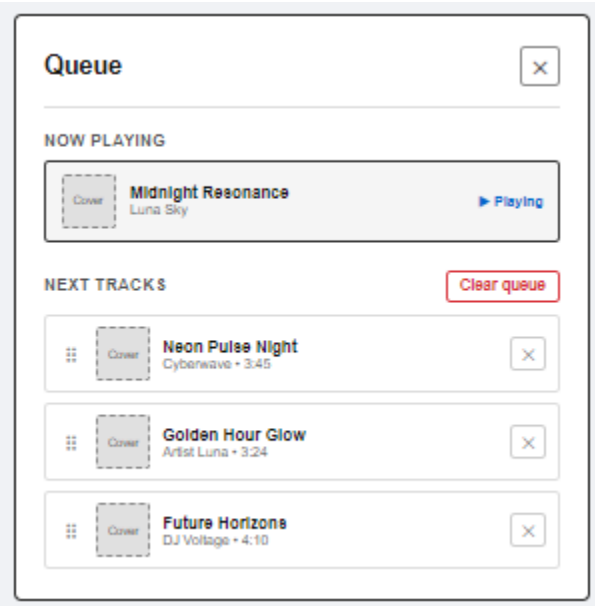
2.1: Save user listening history

```
INSERT INTO listening_history (user_id, track_id, listened_duration_ms, completed, played_at)
VALUES (@userId, @trackId, @listenedDurationMs, CASE WHEN @listenedDurationMs >=
@totalDurationMs * 0.9 THEN 1 ELSE 0 END, SYSUTCDATETIME());
2.2: Increment track play count
UPDATE tracks SET play_count_cache = play_count_cache + 1, updated_at = SYSUTCDATETIME()
WHERE id = @trackId AND publication_status = 'PUBLISHED';
COMMIT TRANSACTION;
```

b. Playback Queue

This drawer displays the current and upcoming tracks and allows the listener to control their playback order.

UI Design



Field Name	Field Type	Description
Now playing	Track row	Displays the current track.
Next tracks	Sortable list	Displays upcoming queue items.
Move item	Drag control	Changes a track's queue position.
Remove item	Icon button	Removes a track from the queue.
Clear queue	Button	Removes all upcoming items.
Close	Icon button	Closes the queue drawer.

Database Access

Table	CRUD	Description
tracks	R	Loads metadata for queue items.
Client playback state	R, U	Keeps queue order in the current client session.

SQL Commands

1. Load queue item metadata

```
SELECT id, title, audio_url, duration_ms, cover_url FROM tracks WHERE id IN (@trackIds) AND publication_status = 'PUBLISHED';
```

4. Listener Profile and Library

4.1 Personal Profile

a. Manage Personal Profile

This screen allows the authenticated listener to view and update their personal profile.

UI Design

The 'Edit Profile' form is a vertical layout with a light gray background. At the top, the title 'Edit Profile' is centered in a bold, dark blue font. Below the title is a horizontal line. The form contains several sections: 1. Avatar section: A circular placeholder labeled 'Avatar' next to a button labeled 'Upload new image'. Below the button is the text 'Uploads or changes the profile image'. 2. Username section: A text input field containing 'john_doe_music'. 3. Display name section: A text input field containing 'John Doe'. 4. Biography section: A text area containing 'Independent music lover, guitarist and electronic beats enthusiast based in Hanoi.' with a small edit icon (pencil) at the bottom right. 5. Date of birth section: A date input field containing '05/15/1998' with a calendar icon at the bottom right. 6. Country section: A dropdown menu showing 'VN - Vietnam' with a downward arrow icon. At the bottom of the form is a large, dark gray button labeled 'Save changes'.

Field Name	Field Type	Description
Avatar	Image upload	Uploads or changes the profile image.
Username	Text input	Stores the unique public username.
Display name	Text input	Stores the public display name.
Biography	Textarea	Stores a short profile introduction.
Date of birth	Date input	Stores the listener's date of birth.
Country	Dropdown	Stores a two-letter country code.

Save changes	Primary button	Validates and saves profile changes.
--------------	----------------	--------------------------------------

Database Access

Table	CRUD	Description
user_profiles	R, U	Reads and updates profile information.
app_users	R	Reads account email and status.

SQL Commands

1. Load profile

SELECT u.email, u.status, p.* FROM app_users u JOIN user_profiles p ON p.user_id = u.id WHERE u.id = @userId AND u.deleted_at IS NULL;

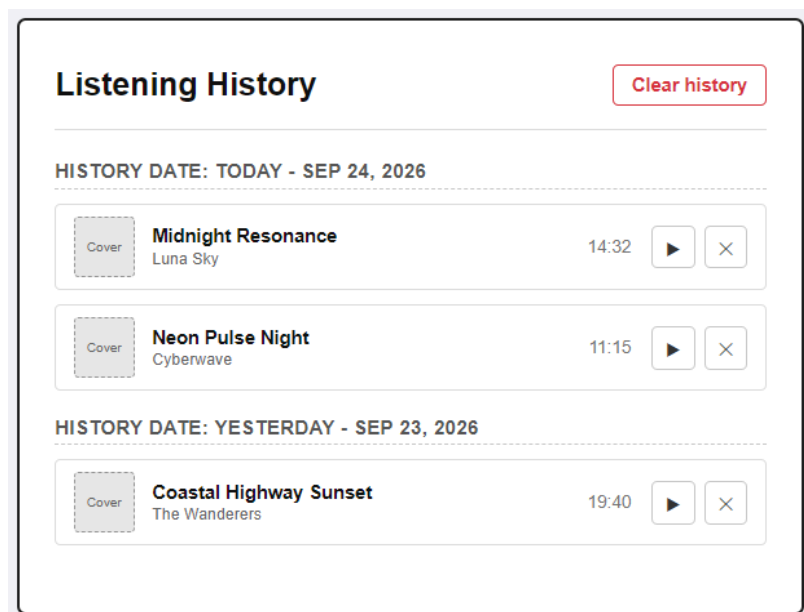
2. Update profile

UPDATE user_profiles SET username = @username, display_name = @displayName, bio = @bio, avatar_public_id = @avatarPublicId, avatar_url = @avatarUrl, date_of_birth = @dateOfBirth, country_code = @countryCode, updated_at = SYSUTCDATETIME() WHERE user_id = @userId;

b. Listening History

This screen displays the authenticated listener's recorded playback history.

UI Design



Field Name	Field Type	Description
History date	Section label	Groups listening activity by date.
Track	Track row	Displays the played track and creator.
Played at	Date and time	Shows when the track was played.
Play again	Icon button	Starts the selected track.
Remove item	Icon button	Removes one history record.
Clear history	Button	Removes all history records for the user.

Database Access

Table	CRUD	Description
listening_history (planned)	R, C, D	Stores user, track, and playback time. This table is not yet present in the supplied ERD.
tracks	R	Reads track metadata for history rows.

SQL Commands

1. Load listening history

```
SELECT h.id, h.played_at, t.id AS track_id, t.title, t.cover_url FROM listening_history h JOIN tracks t ON t.id = h.track_id WHERE h.user_id = @userId ORDER BY h.played_at DESC OFFSET @offset ROWS FETCH NEXT @limit ROWS ONLY;
```

2. Clear listening history

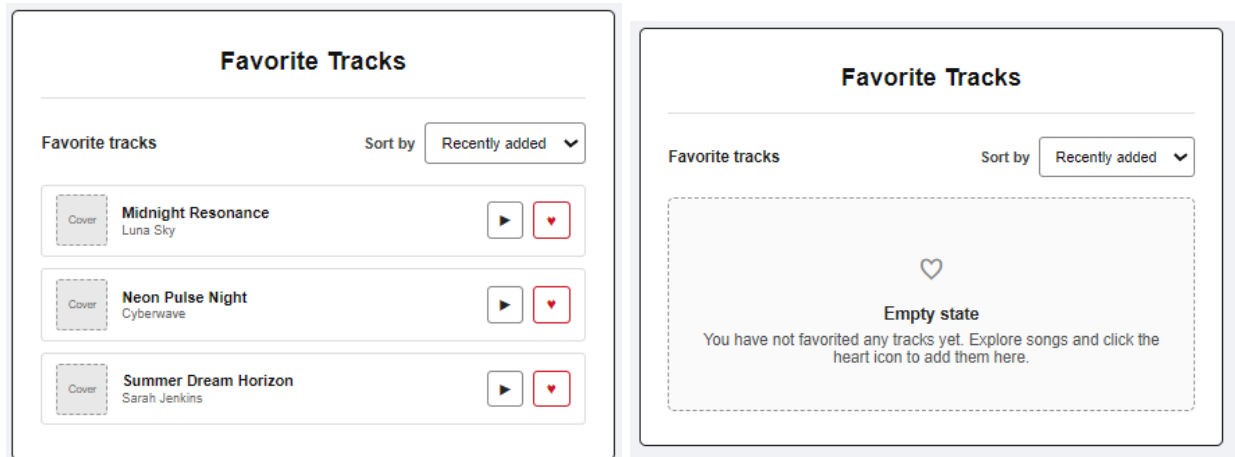
```
DELETE FROM listening_history WHERE user_id = @userId;
```

4.2 Favorite Management

a. Favorite Tracks

This screen displays and manages tracks favorited by the authenticated listener.

UI Design



Field Name	Field Type	Description
Favorite tracks	Track list	Displays tracks favorited by the current user.
Play	Icon button	Starts the selected track.
Remove favorite	Icon button	Removes the selected track from favorites.
Sort by	Dropdown	Sorts by recently added, title, or creator.
Empty state	Message	Appears when the user has no favorites.

Database Access

Table	CRUD	Description
favorites	R, C, D	Reads, creates, and removes favorite relationships.
tracks	R	Reads track information.
user_profiles	R	Reads creator names.

SQL Commands

1. Load favorites

```
SELECT f.created_at, t.id, t.title, t.cover_url, p.display_name FROM favorites f JOIN tracks t ON t.id = f.track_id JOIN user_profiles p ON p.user_id = t.uploader_user_id WHERE f.user_id = @userId ORDER BY f.created_at DESC;
```

2. Add favorite

```
INSERT INTO favorites (user_id, track_id, created_at) VALUES (@userId, @trackId, SYSUTCDATETIME());
```

3. Remove favorite

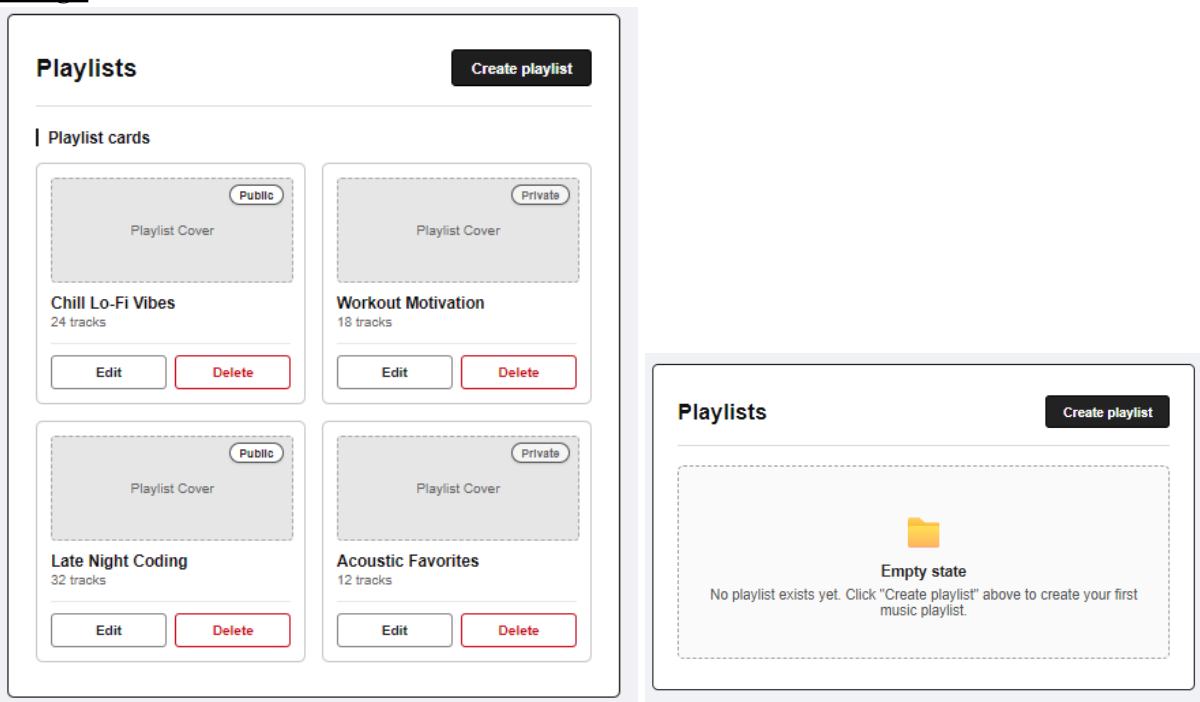
```
DELETE FROM favorites WHERE user_id = @userId AND track_id = @trackId;
```

4.3 Playlist Management

a. Playlist List

This screen displays playlists owned by the authenticated listener and provides playlist management actions.

UI Design



Field Name	Field Type	Description
Create playlist	Primary button	Opens the playlist form.
Playlist cards	Card grid	Displays owned playlists and track counts.
Visibility	Badge	Shows whether a playlist is public or private.
Edit	Action button	Opens the selected playlist for editing.
Delete	Action button	Deletes the selected playlist after confirmation.
Empty state	Message	Appears when no playlist exists.

Database Access

Table	CRUD	Description
-------	------	-------------

playlists	R, D	Reads and deletes playlists owned by the user.
playlist_tracks	R, D	Reads track counts and removes deleted playlist items.

SQL Commands

1. Load owned playlists

```
SELECT p.*, COUNT(pt.track_id) AS track_count FROM playlists p LEFT JOIN playlist_tracks pt ON
pt.playlist_id = p.id WHERE p.owner_user_id = @userId GROUP BY p.id, p.owner_user_id, p.name,
p.description, p.visibility, p.cover_url, p.created_at ORDER BY p.created_at DESC;
```

2. Delete playlist

```
BEGIN TRANSACTION;
```

2.1: Delete all track associations from the playlist first

```
DELETE FROM playlist_tracks WHERE playlist_id = @playlistId;
```

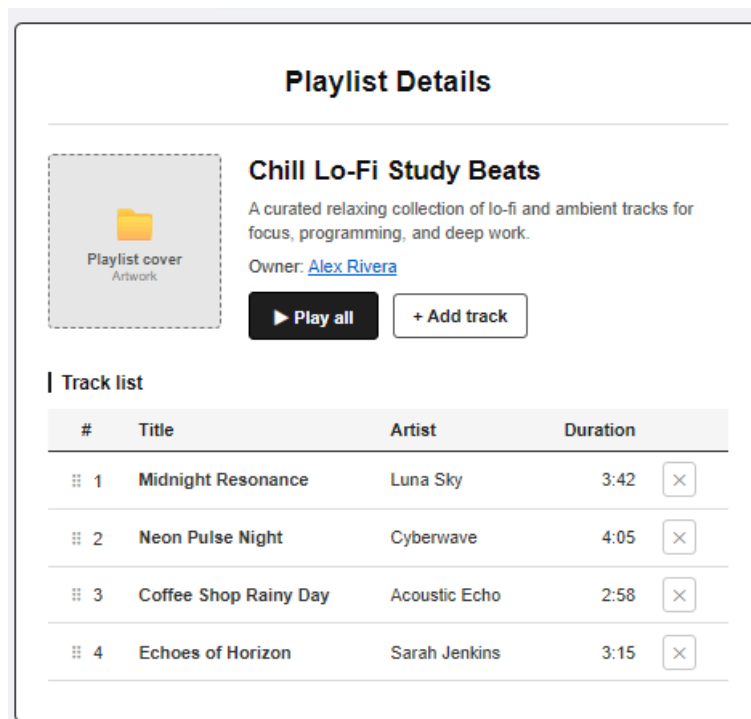
2.2: Delete the playlist itself

```
DELETE FROM playlists WHERE id = @playlistId AND owner_user_id = @userId; IF
@@ROWCOUNT = 0 BEGIN ROLLBACK TRANSACTION; THROW 50004, 'Playlist not found or
unauthorized.', 1; END;
COMMIT TRANSACTION;
```

b. Playlist Details

This screen displays a playlist and its ordered tracks and allows the owner to manage its contents.

UI Design



Field Name	Field Type	Description
Playlist cover	Image	Displays playlist artwork.
Name	Text	Displays the playlist name.

Description	Text	Displays the playlist description.
Owner	Link	Opens the owner's profile.
Play all	Primary button	Adds all public tracks to the queue.
Track list	Sortable table	Displays playlist tracks in order.
Add track	Button	Opens track selection for the owner.
Remove track	Icon button	Removes a selected track for the owner.

Database Access

Table	CRUD	Description
playlists	R	Reads playlist information and visibility.
playlist_tracks	R, C, U, D	Reads and manages ordered playlist items.
tracks	R	Reads public track information.

SQL Commands

1. Load playlist

```
SELECT p.*, up.display_name AS owner_name FROM playlists p JOIN user_profiles up ON up.user_id = p.owner_user_id WHERE p.id = @playlistId AND (p.visibility = 'PUBLIC' OR p.owner_user_id = @userId);
```

2. Load playlist tracks

```
SELECT pt.position, t.id, t.title, t.duration_ms, t.cover_url FROM playlist_tracks pt JOIN tracks t ON t.id = pt.track_id WHERE pt.playlist_id = @playlistId ORDER BY pt.position;
```

3. Remove playlist track

```
DELETE FROM playlist_tracks WHERE playlist_id = @playlistId AND track_id = @trackId;
```

4. Add track to playlist

```
INSERT INTO playlist_tracks (
    playlist_id,
    track_id,
    added_by_user_id,
    position,
    added_at
)
VALUES (
    @playlistId,
    @trackId,
    @userId,
    COALESCE((SELECT MAX(position) FROM playlist_tracks WHERE playlist_id = @playlistId), 0)
    + 1,
    SYSUTCDATETIME()
);
```

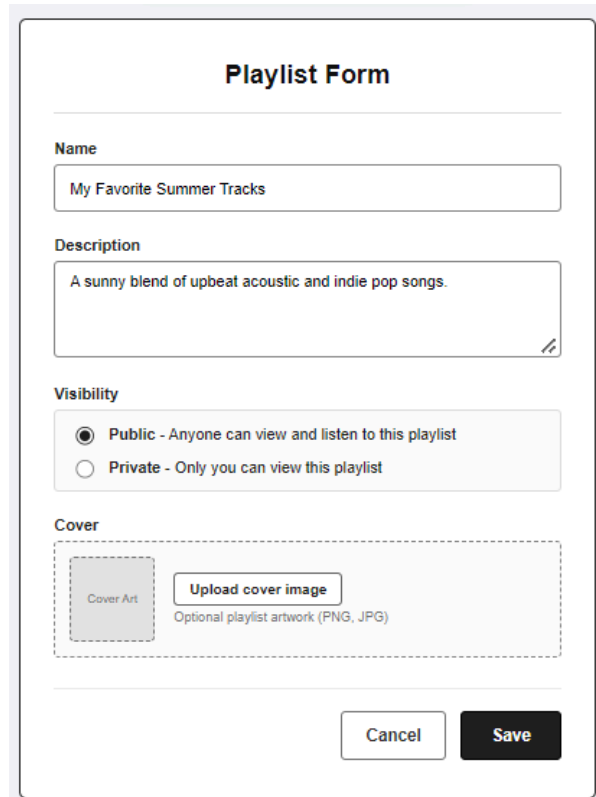
5. Reorder playlist track position

```
UPDATE playlist_tracks
SET position = @newPosition
WHERE playlist_id = @playlistId
AND track_id = @trackId;
```

c. Create or Edit Playlist

This screen allows the authenticated listener to create a playlist or update a playlist they own.

UI Design



The image shows a 'Playlist Form' with the following fields and controls:

- Name:** A text input field containing 'My Favorite Summer Tracks'.
- Description:** A text area containing 'A sunny blend of upbeat acoustic and indie pop songs.' with a small edit icon at the bottom right.
- Visibility:** A radio button group with two options:
 - ☒ **Public** - Anyone can view and listen to this playlist
 - ☐ **Private** - Only you can view this playlist
- Cover:** A dashed box containing a placeholder image labeled 'Cover Art' and a button labeled 'Upload cover image'. Below the button, it says 'Optional playlist artwork (PNG, JPG)'.
- Buttons:** At the bottom right, there are two buttons: 'Cancel' and 'Save'.

Field Name	Field Type	Description
Name	Text input	Required playlist name.
Description	Textarea	Optional playlist description.
Visibility	Radio group	Selects public or private visibility.
Cover	Image upload	Optional playlist artwork.
Save	Primary button	Creates or updates the playlist.
Cancel	Secondary button	Returns without saving.

Database Access

Table	CRUD	Description
playlists	C, U	Creates or updates a playlist owned by the current user.

SQL Commands

1. Create playlist

```
INSERT INTO playlists (owner_user_id, name, description, visibility, cover_url, created_at) VALUES (@userId, @name, @description, @visibility, @coverUrl, SYSUTCDATETIME());
```

2. Update playlist

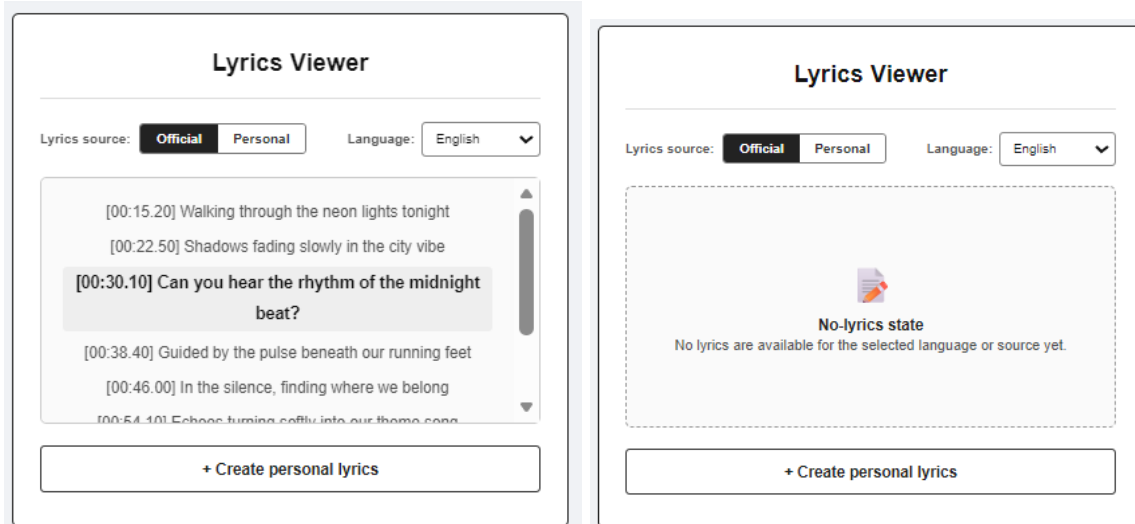
```
UPDATE playlists SET name = @name, description = @description, visibility = @visibility, cover_url = @coverUrl WHERE id = @playlistId AND owner_user_id = @userId;
```

4.4 Lyrics

a. Lyrics Viewer

This screen displays available lyrics and allows the listener to switch between available lyric languages.

UI Design



Field Name	Field Type	Description
Lyrics source	Tab	Switches between official and personal lyrics when available.
Language	Dropdown	Selects an available lyric language.
Lyric content	Scrollable text	Displays the selected lyrics.
Create personal lyrics	Button	Opens the custom lyric editor.
No-lyrics state	Message	Appears when the selected language has no lyrics.

Database Access

Table	CRUD	Description
official_lyrics	R	Reads published official lyrics.
personal_lyrics	R	Reads the current user's personal lyrics.

SQL Commands

1. Load official lyrics

```
SELECT language_code, lyric_content FROM official_lyrics WHERE track_id = @trackId AND status = 'PUBLISHED' ORDER BY language_code;
```

2. Load personal lyrics

```
SELECT language_code, lyric_content FROM personal_lyrics WHERE user_id = @userId AND track_id = @trackId ORDER BY language_code;
```

b. Custom Lyric Editor

This screen allows the authenticated listener to create and maintain personal lyrics for a track.

UI Design

Personal Lyrics Editor

Track

Cover

Midnight Resonance
Luna Sky • Synthwave

Read-only

Language

en - English

Lyric content

[00:15.20] Walking through the neon lights tonight
[00:22.50] Shadows fading slowly in the city vibe
[00:30.10] Can you hear the rhythm of the midnight beat?
[00:38.40] Guided by the pulse beneath our running feet
[00:46.00] In the silence, finding where we belong
[00:54.10] Echoes turning softly into our theme song

Delete

Cancel

Save

Field Name	Field Type	Description
Track	Read-only track summary	Identifies the track being edited.
Language	Dropdown	Selects the lyric language code.
Lyric content	Textarea	Accepts the listener's personal lyrics.
Save	Primary button	Creates or updates the personal lyrics.
Delete	Danger button	Deletes the selected personal lyric version.
Cancel	Secondary button	Returns without saving.

Database Access

Table	CRUD	Description
personal_lyrics	R, C, U, D	Manages lyrics owned by the current user.
tracks	R	Checks that the target track exists and is accessible.

SQL Commands

1. Create personal lyrics

```
INSERT INTO personal_lyrics (user_id, track_id, language_code, lyric_content, created_at, updated_at)
VALUES (@userId, @trackId, @languageCode, @lyricContent, SYSUTCDATETIME(),
SYSUTCDATETIME());
```

2. Update personal lyrics

```
UPDATE personal_lyrics SET lyric_content = @lyricContent, updated_at = SYSUTCDATETIME()
WHERE user_id = @userId AND track_id = @trackId AND language_code = @languageCode;
```

3. Delete personal lyrics

```
DELETE FROM personal_lyrics WHERE user_id = @userId AND track_id = @trackId AND
language_code = @languageCode;
```

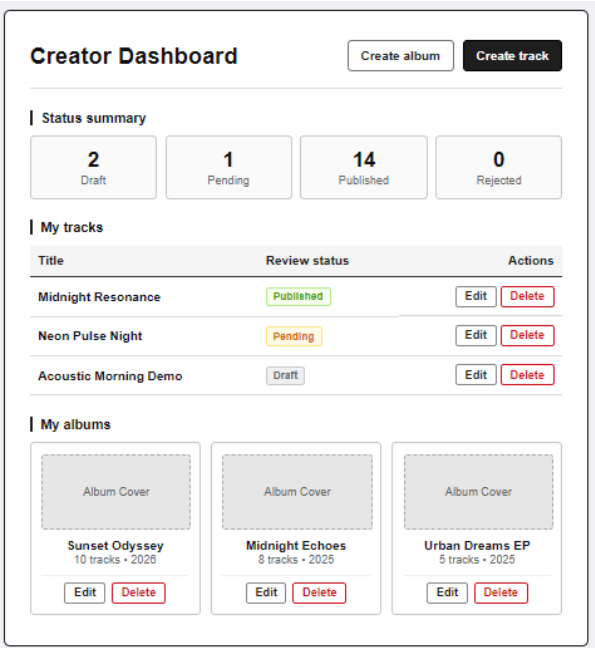
5. User Content Management

5.1 Track Management

a. Track Management

This screen displays tracks owned by the authenticated listener and provides track creation, editing, deletion, and review-submission actions.

UI Design



Field Name	Field Type	Description
Create track	Primary button	Opens the track form.
Create album	Secondary button	Opens the album form.
Status summary	Statistic cards	Shows draft, pending, published, and rejected counts.
My tracks	Table	Lists owned tracks and review status.
My albums	Card list	Lists albums created by the user.
Edit	Action button	Opens the selected item for editing.
Delete	Action button	Deletes an eligible item after confirmation.

Database Access

Table	CRUD	Description
tracks	R, D	Reads and deletes eligible owned tracks.
albums	R, D	Reads and deletes eligible owned albums.
track_submissions	R	Reads the latest review status.

SQL Commands

1. Load owned tracks

```
SELECT t.*, s.status AS submission_status FROM tracks t OUTER APPLY (SELECT TOP (1) status
FROM track_submissions WHERE track_id = t.id ORDER BY submitted_at DESC) s WHERE
t.uploader_user_id = @userId ORDER BY t.updated_at DESC;
```

2. Load owned albums

```
SELECT a.*, COUNT(t.id) AS track_count FROM albums a LEFT JOIN tracks t ON t.album_id = a.id
WHERE a.created_by_user_id = @userId GROUP BY a.id, a.created_by_user_id, a.title, a.slug,
a.description, a.status, a.cover_url, a.release_date, a.published_at, a.created_at ORDER BY a.created_at
DESC;
```

b. Upload or Edit Track

This screen allows the authenticated listener to upload a new track or edit an eligible track they own.

UI Design

Field Name	Field Type	Description
Title	Text input	Required track title.
Album	Dropdown	Optional album owned by the user.
Genre	Dropdown	Required active genre.
Track number	Number input	Optional position within the album.
Description	Textarea	Optional track description.

Audio file	File upload	Uploads the required audio file.
Cover image	Image upload	Uploads optional track artwork.
Save draft	Button	Saves the track without review submission.
Submit for review	Primary button	Creates a pending review submission.

Database Access

Table	CRUD	Description
tracks	C, U	Creates or updates track metadata and cloud media references.
albums	R	Loads albums owned by the user.
genres	R	Loads active genres.
track_submissions	C	Creates a review request.

SQL Commands

1. Create track draft

```
INSERT INTO tracks (uploader_user_id, album_id, genre_id, title, slug, description, track_number,
publication_status, audio_public_id, audio_url, audio_format, duration_ms, cover_public_id, cover_url,
play_count_cache, created_at, updated_at) VALUES (@userId, @albumId, @genreId, @title, @slug,
@description, @trackNumber, 'DRAFT', @audioPublicId, @audioUrl, @audioFormat, @durationMs,
@coverPublicId, @coverUrl, 0, SYSUTCDATETIME(), SYSUTCDATETIME());
```

2. Submit track for review

```
INSERT INTO track_submissions (track_id, submitted_by_user_id, status, submitter_note, submitted_at)
VALUES (@trackId, @userId, 'PENDING', @note, SYSUTCDATETIME());
```

3. Update track draft / rejected track

```
UPDATE tracks
```

```
SET title = @title,
```

```
    slug = @slug,
```

```
    genre_id = @genreId,
```

```
    album_id = @albumId,
```

```
    track_number = @trackNumber,
```

```
    description = @description,
```

```
    audio_public_id = COALESCE(@audioPublicId, audio_public_id),
```

```
    audio_url = COALESCE(@audioUrl, audio_url),
```

```
    audio_format = COALESCE(@audioFormat, audio_format),
```

```
    duration_ms = COALESCE(@durationMs, duration_ms),
```

```
    cover_public_id = COALESCE(@coverPublicId, cover_public_id),
```

```
    cover_url = COALESCE(@coverUrl, cover_url),
```

```
    updated_at = SYSUTCDATETIME()
```

```
WHERE id = @trackId
```

```
AND uploader_user_id = @userId
```

```
AND publication_status IN ('DRAFT', 'REJECTED');
```

c. Track Rejection Details

This screen displays the latest rejection reason and reviewer feedback for a rejected track owned by the listener.

UI Design

Track Review Details

Track

Cover

Summer Dream Horizon (Remix)
Sarah Jenkins • Electronic

Submission status

Rejected

Reviewed at
Sep 24, 2026 • 15:45 GMT+7

Rejection reason

Audio quality does not meet the platform standard / Detected severe clipping and distortion.

Reviewer note

The uploaded master track has significant clipping during the chorus at 01:23 - 01:45. Please check master limiter settings, re-export as 24-bit WAV without digital clipping, and submit again for review.

Field Name	Field Type	Description
Track	Track summary	Identifies the rejected track.
Submission status	Status badge	Shows the latest review state.
Rejection reason	Read-only text	Displays the reason entered by staff.
Reviewer note	Read-only text	Displays additional review feedback.
Reviewed at	Date and time	Shows when the decision was recorded.
Edit track	Primary button	Opens the track form for correction.

Database Access

Table	CRUD	Description
track_submissions	R	Reads the latest rejected submission and review details.
tracks	R	Reads track information.

SQL Commands

1. Load latest rejection

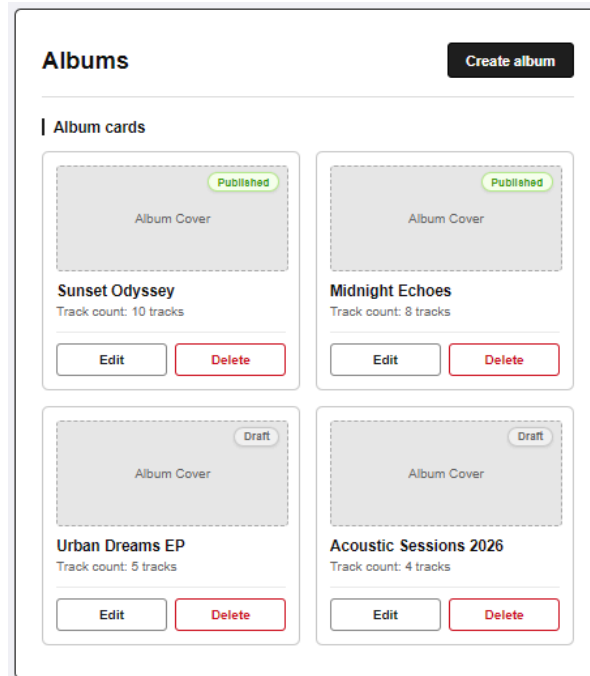
```
SELECT TOP (1) s.status, s.rejection_reason, s.reviewer_note, s.reviewed_at, t.id, t.title FROM
track_submissions s JOIN tracks t ON t.id = s.track_id WHERE s.track_id = @trackId AND
s.submitted_by_user_id = @userId AND s.status = 'REJECTED' ORDER BY s.reviewed_at DESC;
```

5.2 Album Management

a. Album Management

This screen displays albums owned by the authenticated listener and provides album management actions.

UI Design



Field Name	Field Type	Description
Create album	Primary button	Opens the album form.
Album cards	Card grid	Displays owned albums and status.
Track count	Text	Shows the number of tracks in each album.
Edit	Action button	Opens the selected album for editing.
Delete	Action button	Deletes an eligible album after confirmation.

Database Access

Table	CRUD	Description
albums	R, D	Reads and deletes albums owned by the user.
tracks	R, U	Counts album tracks and clears album references when required.

SOL Commands

1. Load owned albums

```
SELECT a.*, COUNT(t.id) AS track_count FROM albums a LEFT JOIN tracks t ON t.album_id = a.id
WHERE a.created_by_user_id = @userId GROUP BY a.id, a.created_by_user_id, a.title, a.slug,
a.description, a.status, a.cover_url, a.release_date, a.published_at, a.created_at ORDER BY a.created_at
DESC;
```

2. Delete album (Unlink tracks and remove album)

BEGIN TRANSACTION;

2.1: Unlink album from all tracks belonging to this album

```
UPDATE tracks SET album_id = NULL, track_number = NULL, updated_at = SYSUTCDATETIME()
WHERE album_id = @albumId AND uploader_user_id = @userId;
```

2.2: Delete album (only allowed for unpublished albums)

```
DELETE FROM albums WHERE id = @albumId AND created_by_user_id = @userId AND status <>
'PUBLISHED'; IF @@ROWCOUNT = 0 BEGIN ROLLBACK TRANSACTION; THROW 50003,
'Album not found, already published, or unauthorized.', 1; END;
COMMIT TRANSACTION;
```

b. Create or Edit Album

This screen allows the authenticated listener to create an album or edit an eligible album they own.

UI Design

Album Form

Title *

Release date

Description

A concept album exploring synthwave melodies and sunset vibes.

Cover image

Cover Art

Optional album artwork (Square PNG, JPG)

Tracks (Select owned tracks & arrange order)

⋮	☒ 1. Midnight Resonance	3:42
⋮	☒ 2. Neon Pulse Night	4:05
⋮	☒ 3. Digital Sunrise	3:18
⋮	☐ 4. Deep Space Orbit (Demo)	4:20

Field Name	Field Type	Description
Title	Text input	Required album title.
Description	Textarea	Optional album description.
Cover image	Image upload	Uploads optional album artwork.
Release date	Date input	Stores the intended release date.
Tracks	Selectable list	Selects owned tracks for the album.
Track order	Drag control	Changes track numbers within the album.
Save	Primary button	Creates or updates the album.

Database Access

Table	CRUD	Description
albums	C, U	Creates or updates the album.
tracks	R, U	Loads eligible tracks and updates album membership.

SQL Commands

1. Create album

```
INSERT INTO albums (created_by_user_id, title, slug, description, status, cover_public_id, cover_url, release_date, created_at) VALUES (@userId, @title, @slug, @description, 'DRAFT', @coverPublicId, @coverUrl, @releaseDate, SYSUTCDATETIME());
```

2. Update album track

```
UPDATE tracks SET album_id = @albumId, track_number = @trackNumber, updated_at = SYSUTCDATETIME() WHERE id = @trackId AND uploader_user_id = @userId;
```

3. Update album metadata

```
UPDATE albums SET title = @title, slug = @slug, description = @description, release_date = @releaseDate, cover_public_id = COALESCE(@coverPublicId, cover_public_id), cover_url = COALESCE(@coverUrl, cover_url), updated_at = SYSUTCDATETIME() WHERE id = @albumId AND created_by_user_id = @userId AND status <> 'PUBLISHED';
```

4. Delete album (Unlink tracks before deleting album)

```
BEGIN TRANSACTION;
```

4.1: Unlink album from tracks (do not delete tracks)

```
UPDATE tracks SET album_id = NULL, track_number = NULL, updated_at = SYSUTCDATETIME() WHERE album_id = @albumId AND uploader_user_id = @userId;
```

4.2: Delete unpublished album

```
DELETE FROM albums WHERE id = @albumId AND created_by_user_id = @userId AND status <> 'PUBLISHED';
```

```
COMMIT TRANSACTION;
```

5.3 Content Report Management

a. Content Report Management

This screen displays content reports submitted by the authenticated listener and their current resolution status.

UI Design

Submitted Reports				
Report list				
Track	Category	Status	Resolution note	Action
Neon Horizon (Rip)	Copyright infringement	Pending	Under review by staff	Delete
Spam Advertisement Audio	Spam / Misleading	Resolved	Track removed from public catalog.	Delete
Acoustic Dream No. 4	Inappropriate content	Rejected	No policy violation detected.	Delete

Field Name	Field Type	Description
Report list	Table	Displays submitted reports.
Track	Link	Opens the reported track.
Category	Text	Shows the selected report category.

Status	Status badge	Shows pending, resolved, or rejected status.
Resolution note	Text	Shows staff feedback when available.
Delete	Action button	Deletes a pending report owned by the user.

Database Access

Table	CRUD	Description
content_reports	R, D	Reads and deletes eligible reports submitted by the user.
tracks	R	Reads reported track information.

SQL Commands

1. Load submitted reports

```
SELECT r.*, t.title AS track_title FROM content_reports r JOIN tracks t ON t.id = r.track_id WHERE
r.submitted_by_user_id = @userId ORDER BY r.created_at DESC;
```

2. Delete pending report

```
DELETE FROM content_reports WHERE id = @reportId AND submitted_by_user_id = @userId AND
status = 'PENDING';
```

b. Create Content Report

This screen allows the authenticated listener to submit a report for a published track.

UI Design

Report Track

Track

Cover

Neon Pulse Night (Rip)
Suspected Re-uploader

Reported Target

Category

Copyright infringement

Description

This track is an unauthorized copy of our original song published last month. Please review and remove it from public catalog.

Cancel

Submit report

Field Name	Field Type	Description
Track	Read-only track summary	Identifies the reported track.
Category	Dropdown	Selects the report category.
Description	Textarea	Explains the suspected violation.
Submit report	Primary button	Validates and creates the report.

Cancel	Secondary button	Closes the dialog without saving.
--------	------------------	-----------------------------------

Database Access

Table	CRUD	Description
content_reports	C	Creates a pending report.
tracks	R	Validates the target public track.

SQL Commands

1. Create report

```
INSERT INTO content_reports (track_id, submitted_by_user_id, category, description, status, created_at, updated_at) VALUES (@trackId, @userId, @category, @description, 'PENDING', SYSUTCDATETIME(), SYSUTCDATETIME());
```

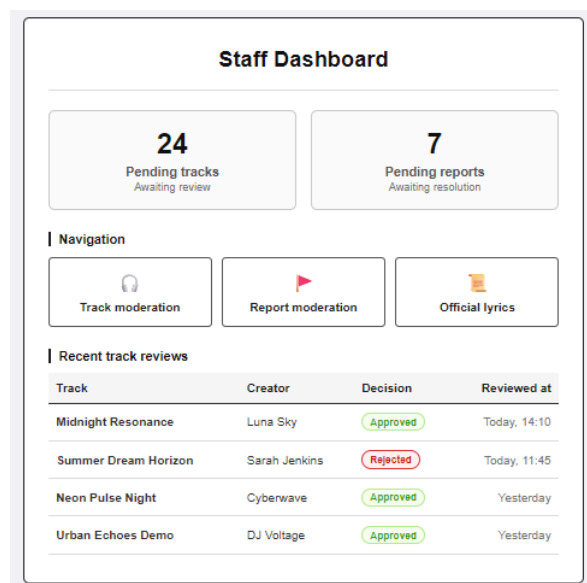
6. Staff Moderation

6.1 Staff Dashboard

a. Staff Dashboard

This screen summarizes the staff member's moderation workload and provides direct navigation to staff functions.

UI Design



Field Name	Field Type	Description
Pending tracks	Statistic card	Shows the number of track submissions awaiting review.
Pending reports	Statistic card	Shows the number of content reports awaiting resolution.
Recent track reviews	Table	Shows recently reviewed track submissions.
Track moderation	Navigation button	Opens Track Moderation.
Report moderation	Navigation button	Opens Report Moderation.
Official lyrics	Navigation button	Opens Official Lyrics Management.

Database Access

Table	CRUD	Description
track_submissions	R	Calculates pending and recently reviewed track submissions.
content_reports	R	Calculates pending and recently resolved content reports.
official_lyrics	R	Reads official lyric workload information.

SOL Commands

1. Count pending track submissions

```
SELECT COUNT(*) AS pending_tracks FROM track_submissions WHERE status = 'PENDING';
```

2. Count pending content reports

```
SELECT COUNT(*) AS pending_reports FROM content_reports WHERE status = 'PENDING';
```

3. Load recent track reviews

```
SELECT TOP (10) ts.*, t.title FROM track_submissions ts JOIN tracks t ON t.id = ts.track_id WHERE ts.reviewer_user_id = @staffUserId ORDER BY ts.reviewed_at DESC;
```

6.2 Track Moderation

a. Track Moderation

This screen lists track submissions available for staff review and provides search, filtering, and review navigation.

UI Design

Field Name	Field Type	Description
Search	Search input	Finds submissions by track or submitter.
Status	Dropdown	Filters pending or reviewed submissions.
Submission table	Table	Displays track, submitter, date, and status.
Open review	Action button	Opens the selected submission.
Pagination	Pagination	Moves between result pages.

Database Access

Table	CRUD	Description
-------	------	-------------

track_submissions	R	Reads submissions awaiting review.
tracks	R	Reads submitted track metadata.
user_profiles	R	Reads submitter information.

SQL Commands

1. Load pending submissions

```
SELECT s.id, s.submitted_at, s.status, t.id AS track_id, t.title, p.display_name AS submitter FROM
track_submissions s JOIN tracks t ON t.id = s.track_id JOIN user_profiles p ON p.user_id =
s.submitted_by_user_id WHERE s.status = 'PENDING' ORDER BY s.submitted_at ASC OFFSET
@offset ROWS FETCH NEXT @limit ROWS ONLY;
```

b. Track Review

This screen presents the submitted track and assessment information and allows staff to approve or reject it through confirmation dialogs.

UI Design

Review Track Submission

Cover preview

Track information

Title:Midnight Resonance (Synth Mix)

Submitter:Luna Sky (ID: usr_8921)

Album:Sunset Odyssey

Genre:Electronic / Synthwave

Submitted:Sep 24, 2026 • 10:15 GMT+7

Audio preview

01:18

midnight_resonance.wav

03:42

Submitter note

"This is the official lead single for my upcoming album. All stems and samples used are 100% original composition. Looking forward to review feedback."

Quality checklist

☒ Audio quality meets standard (no audible distortion, clipping, or phase issues)

☒ Cover artwork is appropriate and compliant with copyright guidelines

☒ Metadata (Title, Submitter, Genre) is accurate and formatted correctly

☐ No automated fingerprinting or sample clearance flags

Reject

Approve

Field Name	Field Type	Description
Track information	Details panel	Displays metadata and ownership.
Audio preview	Audio player	Streams the submitted audio for assessment.
Cover preview	Image	Displays the submitted artwork.
Submitter note	Read-only text	Displays the note from the uploader.
Quality checklist	Checklist	Records the staff assessment.

Approve	Primary button	Opens the approval confirmation.
Reject	Danger button	Opens the rejection form.

Database Access

Table	CRUD	Description
track_submissions	R, U	Read submission details; update review status (approved/rejected) and reviewer notes.
tracks	R, U	Read track details; update track status to PUBLISHED or REJECTED.
albums	R, U	Check and update album status to PUBLISHED if the first track is approved (per BR-18).

SQL Commands

1. Load review details

```
SELECT s.*, t.*, a.title AS album_title, g.name AS genre_name, p.display_name AS submitter FROM
track_submissions s JOIN tracks t ON t.id = s.track_id LEFT JOIN albums a ON a.id = t.album_id JOIN
genres g ON g.id = t.genre_id JOIN user_profiles p ON p.user_id = s.submitted_by_user_id WHERE s.id
= @submissionId;
```

6.3 Report Moderation

a. Report Moderation

This screen lists submitted content reports and provides search, filtering, and review navigation for staff.

UI Design

Report Moderation

Search
Finds reports by track or reporter.

Category

Copyright

Status

Pending

Report table

Track	Reporter	Category	Submission time	Status	Action
Neon Horizon (Rip)	alex_music	Copyright	Today, 14:15	Pending	Open review
Cyber Punk Audio Clip	user_99	Inappropriate	Today, 11:20	Pending	Open review
Casino Spam Track	moderator_bot	Spam	Yesterday, 19:40	Resolved	Open review
Acoustic Dream No. 4	listener_12	Copyright	Sep 22, 2026	Rejected	Open review

Field Name	Field Type	Description
Search	Search input	Finds reports by track or reporter.
Category	Dropdown	Filters by report category.
Status	Dropdown	Filters pending or completed reports.

Report table	Table	Displays report summary and submission time.
Open review	Action button	Opens the selected report.

Database Access

Table	CRUD	Description
content_reports	R	Reads reports requiring review.
tracks	R	Reads reported track information.
user_profiles	R	Reads reporter information.

SQL Commands

1. Load pending reports

```
SELECT r.id, r.category, r.description, r.created_at, t.id AS track_id, t.title, p.display_name AS reporter
FROM content_reports r JOIN tracks t ON t.id = r.track_id JOIN user_profiles p ON p.user_id =
r.submitted_by_user_id WHERE r.status = 'PENDING' ORDER BY r.created_at ASC;
```

2. Approve track submission

```
BEGIN TRANSACTION;
```

2.1: Update submission to APPROVED

```
UPDATE track_submissions SET status = 'APPROVED', reviewer_user_id = @staffUserId,
reviewer_note = @reviewerNote, reviewed_at = SYSUTCDATETIME() WHERE id = @submissionId
AND status = 'PENDING'; IF @@ROWCOUNT = 0 BEGIN ROLLBACK TRANSACTION; THROW
50001, 'The track submission is no longer pending or does not exist.', 1; END;
```

2.2: Update track status to PUBLISHED

```
UPDATE tracks SET publication_status = 'PUBLISHED', approved_at = SYSUTCDATETIME(),
latest_rejection_reason = NULL, updated_at = SYSUTCDATETIME() WHERE id = @trackId;
```

2.3: (Auto-trigger BR-18) If the track belongs to a DRAFT album, publish the album

```
UPDATE albums SET status = 'PUBLISHED', published_at = SYSUTCDATETIME(), updated_at =
SYSUTCDATETIME() WHERE id = (SELECT album_id FROM tracks WHERE id = @trackId) AND
status = 'DRAFT';
```

2.4: Create in-app notification for the uploader

```
INSERT INTO notifications (user_id, type, title, message, action_url, created_at) VALUES
(@uploaderUserId, 'TRACK_APPROVED', 'Track Approved', N'Congratulations! Your track "' +
@trackTitle + N'" has been approved and published.', '/tracks/' + @trackSlug, SYSUTCDATETIME());
COMMIT TRANSACTION;
```

3. Reject track submission (Reject track with a required reason)

```
BEGIN TRANSACTION;
```

3.1: Update submission to REJECTED with a required reason

```
UPDATE track_submissions SET status = 'REJECTED', reviewer_user_id = @staffUserId,
rejection_reason = @rejectionReason, reviewer_note = @reviewerNote, reviewed_at =
SYSUTCDATETIME() WHERE id = @submissionId AND status = 'PENDING'; IF @@ROWCOUNT =
0 BEGIN ROLLBACK TRANSACTION; THROW 50002, 'The track submission is no longer pending or
does not exist.', 1; END;
```

3.2: Update track status to REJECTED

```
UPDATE tracks SET publication_status = 'REJECTED', latest_rejection_reason = @rejectionReason,
updated_at = SYSUTCDATETIME() WHERE id = @trackId;
```

3.3: Create in-app notification for the uploader to revise submission

```
INSERT INTO notifications (user_id, type, title, message, action_url, created_at) VALUES
(@uploaderUserId, 'TRACK_REJECTED', 'Track Needs Revision', N'Your track "' + @trackTitle + N'"
```

was not approved. Reason: ' + @rejectionReason, '/creator/tracks/' + CAST(@trackId AS VARCHAR(20)) + '/rejection', SYSUTCDATETIME());
COMMIT TRANSACTION;

b. Content Report Review

This screen displays the selected content report and reported track. Staff can reject an invalid report or continue to the Track Takedown Confirmation when a violation is confirmed. Confirming a violation on this screen does not immediately change the track or report status.

UI Design

Field Name	Field Type	Description
Report details	Details panel	Displays the category, description, reporter, and submission time.
Reported track	Track preview	Displays and plays the reported track for assessment.
Resolution note	Textarea	Records the Staff member's assessment and decision reason.
Confirm violation	Danger button	Opens Track Takedown Confirmation without immediately changing the report or track status.
Reject report	Secondary button	Opens a confirmation popup and marks an invalid report as REJECTED after confirmation.

Database Access

Table	CRUD	Description
content_reports	R, U	Reads the selected report and updates it only when Staff rejects an invalid report.
tracks	R	Reads the reported track for review. UC-24 does not update the track status.

user_profiles	R	Reads the reporter and track-owner information.
notifications	C	Notifies the reporter when the report is rejected.

SQL Commands

1. Reject invalid report

```
BEGIN TRANSACTION;
```

```
UPDATE content_reports
SET handled_by_user_id = @staffUserId,
    status = 'REJECTED',
    resolution_note = @resolutionNote,
    handled_at = SYSUTCDATETIME(),
    updated_at = SYSUTCDATETIME()
WHERE id = @reportId
AND status = 'PENDING';
```

```
IF @@ROWCOUNT = 0
BEGIN
    ROLLBACK TRANSACTION;
    THROW 50001, 'The report has already been processed.', 1;
END;
```

```
INSERT INTO notifications (
    user_id,
    type,
    title,
    message,
    action_url,
    created_at
)
VALUES (
    @reporterUserId,
    'REPORT_RESOLVED',
    @notificationTitle,
    @notificationMessage,
    @actionUrl,
    SYSUTCDATETIME()
);
```

```
COMMIT TRANSACTION;
```

c. Track Takedown Confirmation

This confirmation dialog allows staff to remove a published track from public access after a content violation has been confirmed.

UI Design

Take Down Track

Reported track

Cover

Neon Horizon (Rip)
Published by: John Doe (ID: usr_4412)

Report category

Copyright infringement

Resolution note

Confirmed unauthorized reproduction of audio master. The track has been removed from public access and the uploader has been notified of the policy violation.

☒ **Confirmation:** I confirm that this published track violates platform policy and should be removed from public access immediately.

Cancel

Take down track

Field Name	Field Type	Description
Reported track summary	Read-only panel	Identifies the published track that will be removed.
Report category	Read-only text	Displays the confirmed violation category.
Resolution note	Textarea	Displays or completes the mandatory takedown reason.
Confirmation	Checkbox	Requires explicit confirmation before the track is taken down.
Take down track	Danger button	Updates the track and report in one transaction.
Cancel	Secondary button	Returns to Content Report Review without changing data.

Database Access

Table	CRUD	Description
content_reports	R, U	Validates and resolves the pending report.
tracks	R, U	Validates the published track and changes its status to TAKEN_DOWN.
notifications	C	Notifies the track owner and report submitter.
audit_logs	C	Records the sensitive moderation action if audit logging is implemented.

SQL Commands

```
1. Take down the published track
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE content_reports
```

```

SET status = 'RESOLVED',
    handled_by_user_id = @staffUserId,
    resolution_note = @resolutionNote,
    handled_at = SYSUTCDATETIME(),
    updated_at = SYSUTCDATETIME()
WHERE id = @reportId
    AND track_id = @trackId
    AND status = 'PENDING';

IF @@ROWCOUNT = 0
BEGIN
    THROW 50001, 'The report has already been processed.', 1;
END;

UPDATE tracks
SET publication_status = 'TAKEN_DOWN',
    updated_at = SYSUTCDATETIME()
WHERE id = @trackId
    AND publication_status = 'PUBLISHED';

IF @@ROWCOUNT = 0
BEGIN
    THROW 50002, 'The track is no longer available for takedown.', 1;
END;

INSERT INTO notifications (
    user_id,
    type,
    title,
    message,
    action_url,
    created_at
)
VALUES (
    @trackOwnerUserId,
    'TRACK_TAKEN_DOWN',
    @ownerNotificationTitle,
    @ownerNotificationMessage,
    @trackActionUrl,
    SYSUTCDATETIME()
);

INSERT INTO notifications (
    user_id,
    type,
    title,
    message,
    action_url,
    created_at
)
VALUES (

```

```

        @reporterUserId,
        'REPORT_RESOLVED',
        @reporterNotificationTitle,
        @reporterNotificationMessage,
        @reportActionUrl,
        SYSUTCDATETIME()
    );

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;

```

6.4 Official Lyrics

a. Official Lyrics Management

This screen allows staff to create, update, publish, and delete official lyric versions for published tracks.

UI Design

Field Name	Field Type	Description
Track search	Search input	Selects a published track.
Language	Dropdown	Selects the lyric language code.
Lyric content	Textarea	Edits official lyrics.
Status	Dropdown	Sets draft or published status.
Save	Primary button	Creates or updates the lyric version.
Delete	Danger button	Deletes the selected lyric version.

Database Access

Table	CRUD	Description
-------	------	-------------

official_lyrics	R, C, U, D	Manages official lyric versions.
tracks	R	Loads eligible published tracks.

SQL Commands

1. Create official lyrics

```
INSERT INTO official_lyrics (track_id, language_code, lyric_content, status, created_by_user_id,
created_at, updated_at) VALUES (@trackId, @languageCode, @lyricContent, @status, @staffId,
SYSUTCDATETIME(), SYSUTCDATETIME());
```

2. Update official lyrics

```
UPDATE official_lyrics SET lyric_content = @lyricContent, status = @status, updated_at =
SYSUTCDATETIME() WHERE id = @lyricId;
```

3. Delete official lyrics

```
DELETE FROM official_lyrics WHERE id = @lyricId;
```

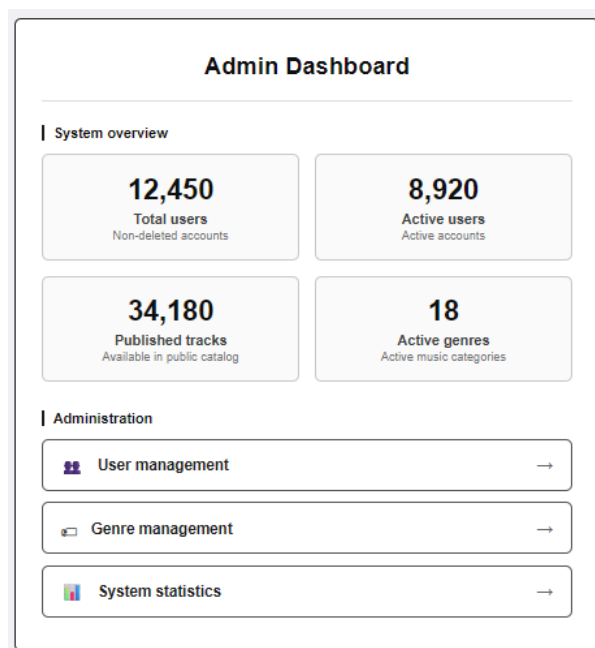
7. System Administration

7.1 Admin Dashboard

a. Admin Dashboard

This screen summarizes key system information and provides direct navigation to administrative functions.

UI Design



Field Name	Field Type	Description
Total users	Statistic card	Shows the number of non-deleted accounts.
Active users	Statistic card	Shows the number of active accounts.
Published tracks	Statistic card	Shows the number of tracks available in the public catalog.
Active genres	Statistic card	Shows the number of active genres.

User management	Navigation button	Opens User Management.
Genre management	Navigation button	Opens Genre Management.
System statistics	Navigation button	Opens System Statistics.

Database Access

Table	CRUD	Description
app_users, roles	R	Calculates account and role totals.
tracks	R	Calculates published track totals.
genres	R	Calculates active genre totals.
content_reports	R	Calculates pending report totals.

SQL Commands

1. Load account statistics

```
SELECT COUNT(*) AS total_users, SUM(CASE WHEN status = 'ACTIVE' THEN 1 ELSE 0 END) AS active_users FROM app_users WHERE deleted_at IS NULL;
```

2. Count published tracks

```
SELECT COUNT(*) AS published_tracks FROM tracks WHERE publication_status = 'PUBLISHED';
```

3. Count active genres

```
SELECT COUNT(*) AS active_genres FROM genres WHERE is_active = 1;
```

7.2 User Management

a. User Management

This screen allows administrators to search, review, ban, and unban system accounts.

UI Design

The mockup shows a 'User Management' interface. At the top, there's a search bar with a magnifying glass icon and placeholder text 'Finds accounts by email, username, c'. To the right of the search bar are two dropdown menus: 'ROLE' with 'All Roles' selected, and 'STATUS' with 'All Statuses' selected. Below these is a table titled 'USER TABLE'. The table has three columns: 'ACCOUNT', 'ROLE', and 'STATUS'. It lists five users: Alex Miller (Admin, Active), Sarah Chen (Staff, Active), John Doe (Listener, Active), Maya Kim (Listener, Pending), and Tom Riddle (Listener, Banned). Each user entry includes a circular profile picture icon, the user's name, email, role in a pill-shaped button, and status in a colored pill-shaped button.

Field Name	Field Type	Description
Search	Search input	Finds accounts by email, username, or display name.
Role	Dropdown	Filters accounts by role.
Status	Dropdown	Filters active, pending, or banned accounts.

User table	Table	Displays account and role information.
Ban	Action button	Changes an eligible account to banned.
Unban	Action button	Restores an eligible banned account.
View details	Action button	Opens the selected account.

Database Access

Table	CRUD	Description
app_users	R, U	Reads accounts and updates account status.
roles	R	Reads role information.
user_profiles	R	Reads public profile information.

SQL Commands

1. Load user accounts

```
SELECT u.id, u.email, u.status, u.email_verified_at, u.last_login_at, r.code AS role_code, p.username,
p.display_name FROM app_users u JOIN roles r ON r.id = u.role_id LEFT JOIN user_profiles p ON
p.user_id = u.id WHERE u.deleted_at IS NULL ORDER BY u.created_at DESC;
```

2. Change account status

```
UPDATE app_users SET status = @status, updated_at = SYSUTCDATETIME() WHERE id = @userId
AND deleted_at IS NULL;
```

b. User Details

This screen displays account, profile, role, activity, and content summary information for the selected user.

UI Design

The UI design for the 'User Details' screen for Alex Rivers is as follows:

- Header:**
 - Avatar placeholder (AVATAR)
 - Alex Rivers** (Name)
 - @alexrivers • User ID: #USR-84920 (Handle and ID)
 - CURRENT ROLE: **Creator** (button)
- Actions:**
 - Assign role (button)
 - Ban or unban (button)
- CONTENT SUMMARY:**
 - 14 Tracks
 - 2 Albums
 - 1 Reports
- Account information:**
 - Email: alex.rivers@soundwave.io
 - Status: **Active** (button)
 - Verification state: **Verified** (button)
- Profile information:**
 - Username: @alexrivers
 - Display name: Alex Rivers
 - Country: United States (US)

Field Name	Field Type	Description
------------	------------	-------------

Account information	Details panel	Displays email, status, and verification state.
Profile information	Details panel	Displays username, display name, and country.
Current role	Badge	Displays the assigned role.
Content summary	Statistic cards	Shows track, album, and report counts.
Ban or unban	Action button	Updates the account status.
Assign role	Action button	Opens the role assignment dialog.

Database Access

Table	CRUD	Description
app_users	R, U	Reads account information and changes status.
roles	R	Reads the current role.
user_profiles	R	Reads profile information.
tracks, albums, content_reports	R	Reads administrative summary counts.

SQL Commands

1. Load user details

```
SELECT u.*, r.code AS role_code, r.name AS role_name, p.* FROM app_users u JOIN roles r ON r.id = u.role_id LEFT JOIN user_profiles p ON p.user_id = u.id WHERE u.id = @userId;
```

2. Load content summary

```
SELECT (SELECT COUNT(*) FROM tracks WHERE uploader_user_id = @userId) AS track_count,
(SELECT COUNT(*) FROM albums WHERE created_by_user_id = @userId) AS album_count,
(SELECT COUNT(*) FROM content_reports WHERE submitted_by_user_id = @userId) AS report_count;
```

c. Assign Role

This dialog allows an administrator to assign an authorized role to the selected account.

UI Design

The 'Assign role' dialog box contains the following elements:

- USER:** A section showing a selected user 'Alex Rivers' with a circular profile picture containing the initials 'AR' and a text area displaying '@alexrivers • alex.rivers@soundwave.io'.
- CURRENT ROLE:** A dropdown menu currently showing 'Creator'.
- NEW ROLE:** A dropdown menu currently showing 'Staff'.
- Buttons:** 'Cancel' and 'Confirm' buttons at the bottom right.

Field Name	Field Type	Description
------------	------------	-------------

User	Read-only user summary	Identifies the account being changed.
Current role	Text	Displays the current role.
New role	Dropdown	Selects an available role.
Confirm	Primary button	Updates the user's role.
Cancel	Secondary button	Closes the dialog without saving.

Database Access

Table	CRUD	Description
roles	R	Loads available roles.
app_users	U	Updates the assigned role.

SQL Commands

1. Load roles

```
SELECT id, code, name, description FROM roles ORDER BY id;
```

2. Assign role

```
UPDATE app_users SET role_id = @roleId, updated_at = SYSUTCDATETIME() WHERE id = @userId  
AND deleted_at IS NULL;
```

7.3 Genre Management

a. Genre Management

This screen allows administrators to search, review, activate, and deactivate music genres.

UI Design

Field Name	Field Type	Description
Search	Search input	Finds genres by name or slug.
Status	Dropdown	Filters active or inactive genres.
Create genre	Primary button	Opens the genre form.
Genre table	Table	Displays genre information and track count.
Edit	Action button	Opens the selected genre.

Activate or deactivate	Action button	Changes genre availability.
------------------------	---------------	-----------------------------

Database Access

Table	CRUD	Description
genres	R, U	Reads genres and updates active status.
tracks	R	Counts tracks assigned to each genre.

SQL Commands

1. Load genres

```
SELECT g.*, COUNT(t.id) AS track_count FROM genres g LEFT JOIN tracks t ON t.genre_id = g.id
GROUP BY g.id, g.name, g.slug, g.description, g.is_active, g.created_by_user_id, g.created_at ORDER
BY g.name;
```

2. Change genre status

```
UPDATE genres SET is_active = @isActive WHERE id = @genreId;
```

b. Create or Edit Genre

This screen allows administrators to create a genre or update an existing genre.

UI Design

Field Name	Field Type	Description
Name	Text input	Required unique genre name.
Slug	Text input	Required unique URL-friendly value.
Description	Textarea	Optional genre description.
Active	Switch	Controls whether the genre is available for selection.
Save	Primary button	Creates or updates the genre.

Cancel	Secondary button	Returns without saving.
--------	------------------	-------------------------

Database Access

Table	CRUD	Description
genres	C, U	Creates or updates a genre.

SQL Commands

1. Create genre

```
INSERT INTO genres (name, slug, description, is_active, created_by_user_id, created_at) VALUES (@name, @slug, @description, @isActive, @adminId, SYSUTCDATETIME());
```

2. Update genre

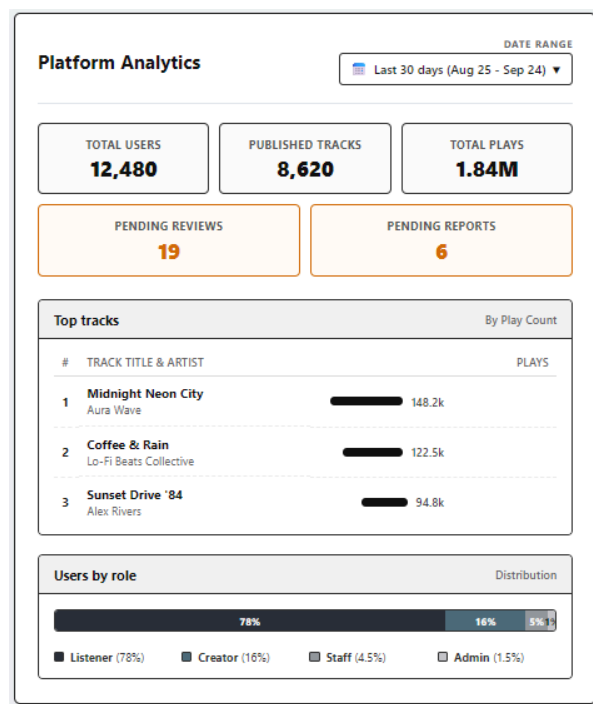
```
UPDATE genres SET name = @name, slug = @slug, description = @description, is_active = @isActive WHERE id = @genreId;
```

7.4 Statistics

a. System Statistics

This screen presents administrative statistics for accounts, catalog content, playback, moderation, and reports.

UI Design



Field Name	Field Type	Description
Date range	Date filter	Limits statistics to the selected period where supported.
Total users	Statistic card	Shows the number of non-deleted accounts.
Published tracks	Statistic card	Shows the number of public tracks.
Total plays	Statistic card	Shows the sum of cached track plays.
Pending reviews	Statistic card	Shows pending track submissions.

Pending reports	Statistic card	Shows reports awaiting staff action.
Top tracks	Chart or table	Shows tracks with the highest play counts.
Users by role	Chart	Shows the account distribution by role.

Database Access

Table	CRUD	Description
app_users, roles	R	Calculates account and role statistics.
tracks	R	Calculates catalog and playback statistics.
track_submissions	R	Calculates moderation workload.
content_reports	R	Calculates report workload.

SQL Commands

1. Load summary statistics

```
SELECT (SELECT COUNT(*) FROM app_users WHERE deleted_at IS NULL) AS total_users,
(SELECT COUNT(*) FROM tracks WHERE publication_status = 'PUBLISHED') AS published_tracks,
(SELECT COALESCE(SUM(play_count_cache), 0) FROM tracks) AS total_plays, (SELECT
COUNT(*) FROM track_submissions WHERE status = 'PENDING') AS pending_reviews, (SELECT
COUNT(*) FROM content_reports WHERE status = 'PENDING') AS pending_reports;
```

2. Load top tracks

```
SELECT TOP (10) id, title, play_count_cache FROM tracks WHERE publication_status = 'PUBLISHED'
ORDER BY play_count_cache DESC;
```

3. Load users by role

```
SELECT r.code, COUNT(u.id) AS user_count FROM roles r LEFT JOIN app_users u ON u.role_id = r.id
AND u.deleted_at IS NULL GROUP BY r.code ORDER BY r.code;
```

8. Non-UI Functions

a. Process Cloud Media

This backend service validates, uploads, replaces, and deletes audio and image resources in cloud storage and persists the resulting media references.

System Functions

Not applicable. This function runs in the background and has no standalone user interface.

Feature	System Function	Description
Cloud Media Processing	Validate media	Validates the authorized media type, size, format, and ownership context before upload.
Cloud Media Processing	Upload media	Uploads audio or image content and returns its public identifier and secure URL.
Cloud Media Processing	Replace media	Uploads a replacement resource and removes the superseded cloud object when safe.
Cloud Media Processing	Delete media	Deletes an unused media object after authorization and reference checks.
Cloud Media Processing	Persist media reference	Stores the cloud public identifier and secure URL on the related entity.

Database Access

Table	CRUD	Description
tracks	U	Stores audio and track-cover public identifiers and URLs.
albums	U	Stores album-cover public identifiers and URLs.
user_profiles	U	Stores avatar public identifiers and URLs.
Cloud Storage	C, U, D	Stores, replaces, and removes authorized media objects.

SQL Commands

1. Update track media reference

```
UPDATE tracks SET audio_public_id = @audioPublicId, audio_url = @audioUrl, audio_format = @audioFormat, duration_ms = @durationMs, cover_public_id = @coverPublicId, cover_url = @coverUrl, updated_at = SYSUTCDATETIME() WHERE id = @trackId;
```

2. Update profile avatar reference

```
UPDATE user_profiles SET avatar_public_id = @avatarPublicId, avatar_url = @avatarUrl, updated_at = SYSUTCDATETIME() WHERE user_id = @userId;
```

IV. Appendix

1. Assumptions & Dependencies

AS-1: The user must have a stable internet connection with a minimum speed of 1Mbps to ensure uninterrupted audio streaming.

AS-2: All audio files uploaded by creators are assumed to be legally owned or appropriately licensed, minimizing copyright liabilities for the platform.

DE-1: The platform relies heavily on a third-party Cloud Storage provider (e.g., AWS S3, Google Cloud Storage, or Firebase) for hosting and streaming large MP3 audio files and image assets.

DE-2: The system's email verification and password recovery functionalities depend entirely on an external SMTP Email Service (e.g., SendGrid, Mailgun).

2. Limitations & Exclusions

Offline Playback: The current scope of the project does not support downloading songs for offline listening. Users must be online to stream music.

Mobile Application: The platform is designed strictly as a responsive web application. A native mobile app (iOS/Android) is excluded from this phase.

Monetization/Payment: The platform will operate entirely free of charge. Subscription plans, payment gateways, and creator royalty payouts are not included in the initial release.

3. Business Rules

ID	Category	Rule Definition
BR-01	Constraint	Email must be verified (via OTP or link) before the account is fully activated; unverified accounts only have Guest

		privileges. The Email Service must trigger a welcome email upon successful registration.
BR-02	Constraint	Each email can only be registered for one unique account on the system.
BR-03	Constraint	Passwords must meet minimum complexity requirements (e.g., $\geq$ 8 characters, including uppercase/lowercase/numbers).
BR-04	Constraint	"Forgot password" requests trigger the Email Service to send a verification link/code, which is only valid for a limited time (e.g., 15 minutes).
BR-05	Constraint	An account can only hold exactly one system role at a time (Listener, Staff, Admin).
BR-06	Constraint	Accounts locked (banned) by an Admin cannot log in or perform any actions until they are unbanned.
BR-07	Constraint	Users are only permitted to edit their own personal information and avatar.
BR-08	Constraint	Profile pictures (avatars) must comply with format/size limits specified by the system before being securely saved to Cloud Storage.
BR-09	Constraint	Only content with an "Approved/Published" status will appear in search results and public browsing for Guests and Listeners.
BR-10	Constraint	Searching by lyrics (lyric search) is only applicable to songs that have an officially approved .lrc file.

BR-11	Constraint	Guests may stream complete tracks with PUBLISHED status. When a track reaches the end, the system must display a login or registration prompt. The Guest may dismiss the prompt and continue browsing public content, but authenticated features remain unavailable.
BR-12	Fact	Audio streaming is executed via direct HTTP Range Requests between the client's HTML5 Audio player and the Cloud Storage (Cloudinary) CDN. The backend application server is responsible only for verifying track playback authorization and returning the secure CDN URL; it does not proxy audio binary streams.
BR-13	Constraint	Listening History is only recorded and displayed for logged-in accounts (Listener).
BR-14	Constraint	Only the creator (owner) of a playlist has the authority to view private playlists, add/remove songs, edit details, or delete the playlist.
BR-15	Constraint	Playlists in the "private" state are not visible to any users other than the owner.
BR-16	Constraint	A song can only be "liked" once per user; pressing like again will be unlike (toggle).
BR-17	Inference	The "Liked Songs" list is automatically updated (add/remove) as soon as the user likes/unlikes, without requiring manual playlist creation.
BR-18	Inference	A newly created Album/EP by a Listener has the default status "Draft". An album is only eligible to be published and

		displayed publicly when it contains at least one track, and those component tracks have been approved by Staff.
BR-19	Constraint	Only users with the Staff role have the authority to access the moderation dashboard and change a track's submission status to Approve or Reject.
BR-20	Constraint	When Rejecting a track submission, a feedback reason is mandatory to notify the Listener (Uploader) so they can correct and resubmit.
BR-21	Constraint	Mandatory metadata (title, genre) must be fully entered before a track submission can be sent for review.
BR-22	Constraint	An Album/EP is displayed publicly as long as it contains at least one "Published" track. Any Draft, Pending, or Rejected tracks within that album remain hidden from the public and do not affect the visibility of the album itself.
BR-23	Constraint	Listeners (Uploaders) can only manage (edit/delete) songs and albums that they uploaded themselves.
BR-24	Constraint	Rejected content can be edited and resubmitted under the same record based on the feedback provided by Staff.
BR-25	Constraint	Only Staff members have the authority to add, edit, or remove official lyrics (.lrc). Listeners are restricted to managing their own private custom lyrics.
BR-26	Constraint	The .lrc file must be in a valid time-synced format to be accepted for storage and display.

BR-27	Constraint	A song can have multiple lyric translations (Original, Vietnamese, English...), but there can be only one "Original" version.
BR-28	Constraint	Users can only switch the lyric language among approved/public translations.
BR-29	Constraint	Custom personal lyrics, translations, or transcriptions (Romaji, Pinyin) created by a Listener are strictly private, stored and displayed only on the creator's account, and cannot be shared publicly.
BR-30	Constraint	Only logged-in users (Listeners) have the right to submit content reports. Reports can strictly be targeted only at tracks that are currently in the "Published" status.
BR-31	Constraint	Each report must be associated with a specific predefined reason (copyright infringement, inappropriate content, incorrect metadata...).
BR-32	Fact	Reports are routed to the Staff moderation dashboard for processing; the report status must be tracked and resolved.
BR-33	Action Enabler	The system must trigger email notifications when a track's submission status or a user's account status changes.
BR-34	Constraint	Email notifications are delegated through the Email Service; if sending fails, the system requires a retry mechanism or error logging.
BR-35	Constraint	Deep sharing links only function for public content; private playlists or pending content cannot generate valid sharing links.

BR-36	Constraint	Guests only have permission to view/search public content and stream music; all interactive actions (like, playlist, upload, report...) require logging in.
BR-37	Constraint	Admins have full override authority over system data (users, genres). Actions such as banning or unbanning a user take immediate effect and revoke active sessions.
BR-38	Constraint	Staff members do not have the authority to manage user accounts — they are strictly restricted to content moderation, report handling, and official lyric maintenance.